

PYTHON

django

MVC



Samit Koyom
สถาบันไอทีจีเนียส



วิทยากร



อ.สามิตร โกยม

ปริญญาโทคณะเทคโนโลยีสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

► Frontend

Angular, React, Vue, Next, Nuxt, Bootstrap, Tailwind CSS

► Backend

PHP, Python, Java, Kotlin, Go, Rust, NodeJS, NestJS, .NET

► Database

MySQL, PostgreSQL, MS SQL Server, MongoDB, Supabase

► Mobile

Java, Kotlin, Objective C, Swift, KMP, Cordova, Flutter ,
React Native, Expo

► DevOps

Git, Github, Gitlab, Docker, Kubernetes, Jenkins, CI/CD



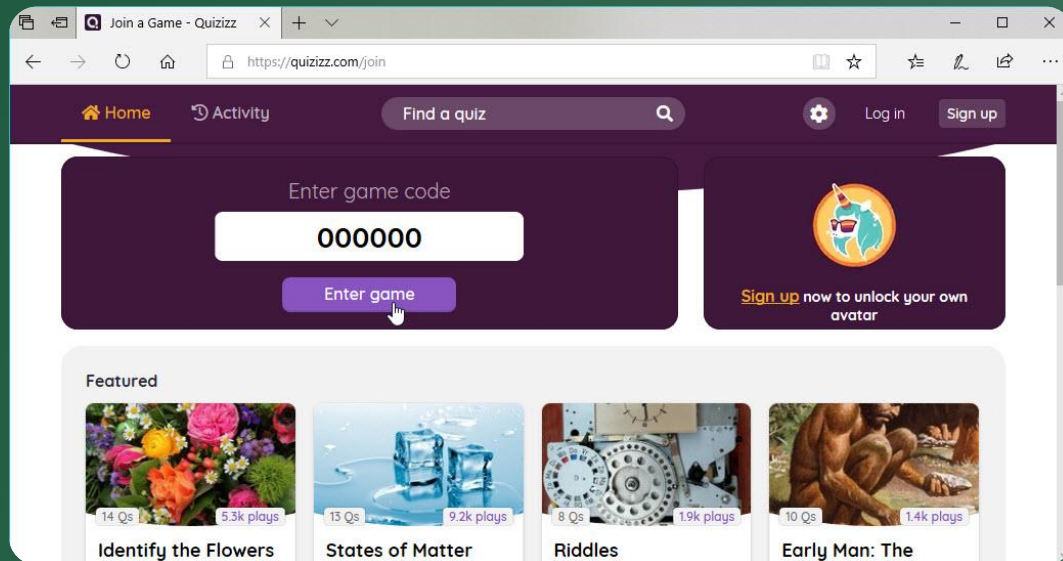
แบบทดสอบก่อนอบรม



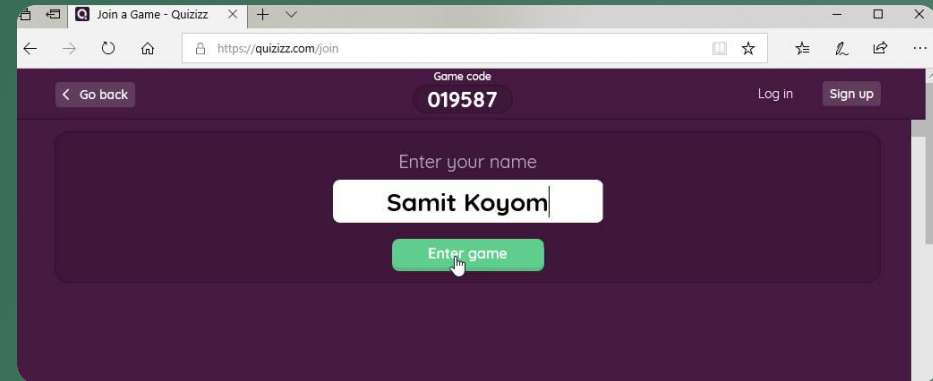
Pre-Test ทำแบบทดสอบก่อนอบรม

STEP 1: เข้าทำแบบทดสอบที่ลิงก์ ป้อนรหัสเข้าห้องสอบ

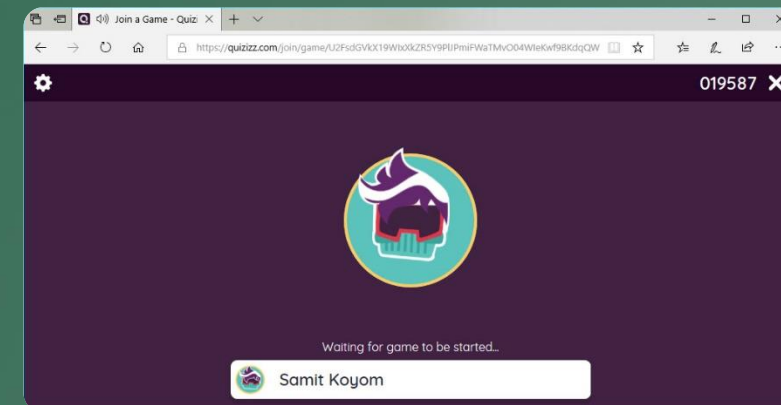
quizizz.com/join



STEP 2: ป้อนชื่อ



STEP 3: รอผู้สอน Start ข้อสอบ





ดาวน์โหลดเอกสารประกอบการอบรม

bit.ly/django-minebea

Course Outline



1. แนะนำ Django Framework
2. ติดตั้งและสร้างโปรเจกต์แรก
3. โครงสร้างไฟล์ของ Django
4. การสร้างแอปและการทำงานแบบ MVC
5. การกำหนดเส้นทาง URL
6. Template Engine ของ Django
7. Static Files และ Media
8. การทำงานกับ Model และฐานข้อมูล
9. Django Rest Framework (DRF)
10. การทำ CRUD Rest API
11. การทำ Authentication (JWT) ใน DRF
12. การ Deployed Project Django แบบต่างๆ



PYTHON django MVC

วันที่

1

5



5

dj

dj



Samit Koyom
สถาบันไอทีจีเนียส

MinebeaMitsumi
Passion to Create Value through Difference



Day 1



1. แนะนำ Django Framework
2. ติดตั้งและสร้างโปรเจกต์แรก
3. โครงสร้างไฟล์ของ Django
4. การสร้างแอปและการทำงานแบบ MVC
5. การกำหนดเส้นทาง URL
6. Template Engine ของ Django
7. Static Files และ Media
8. การทำงานกับ Model และฐานข้อมูล

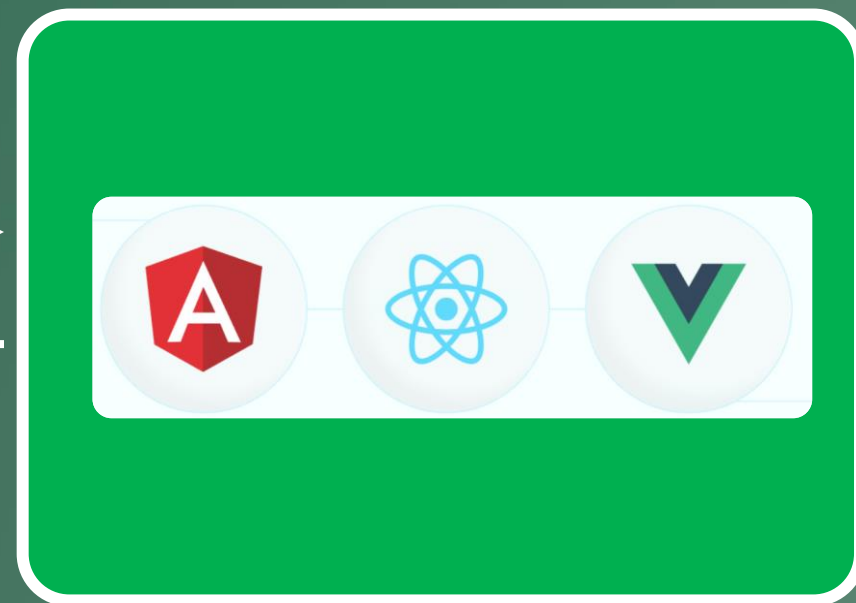


ภาพรวมหลักสูตร

Backend
Django Rest API



Frontend
React/Vue/Angular





Why Framework **django** *Best For Web Development?*

Why Django Framework is Best For Web Development?

- Simplifies Development to a Huge Extent
- Provides Inbuilt and Up-to-Date Security Features
- Suitable For Any Type of Project
- Implements DRY and KISS
- Contains Support For REST APIs
- Massive Community Support

Features of **django**

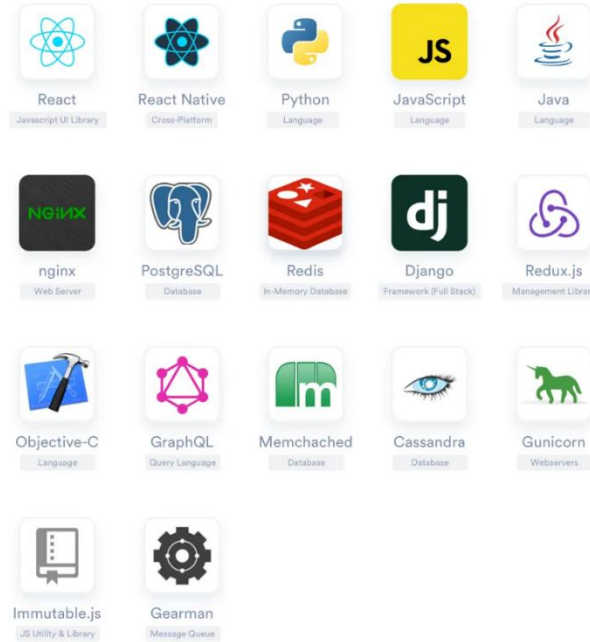


ใครใช้ Django อยู่บ้าง ?

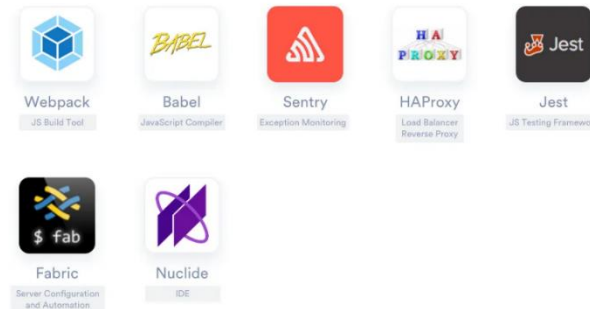




Application and Data



DevOps





NGINX

nginx



Python



React



Java



MySQL



Amazon EC2



Redis



Amazon S3



Django



Amazon
CloudFront



Go



Objective-C



Backbone.js



Memcached

UNDERSCORE.JS

Underscore



Hadoop



HBase



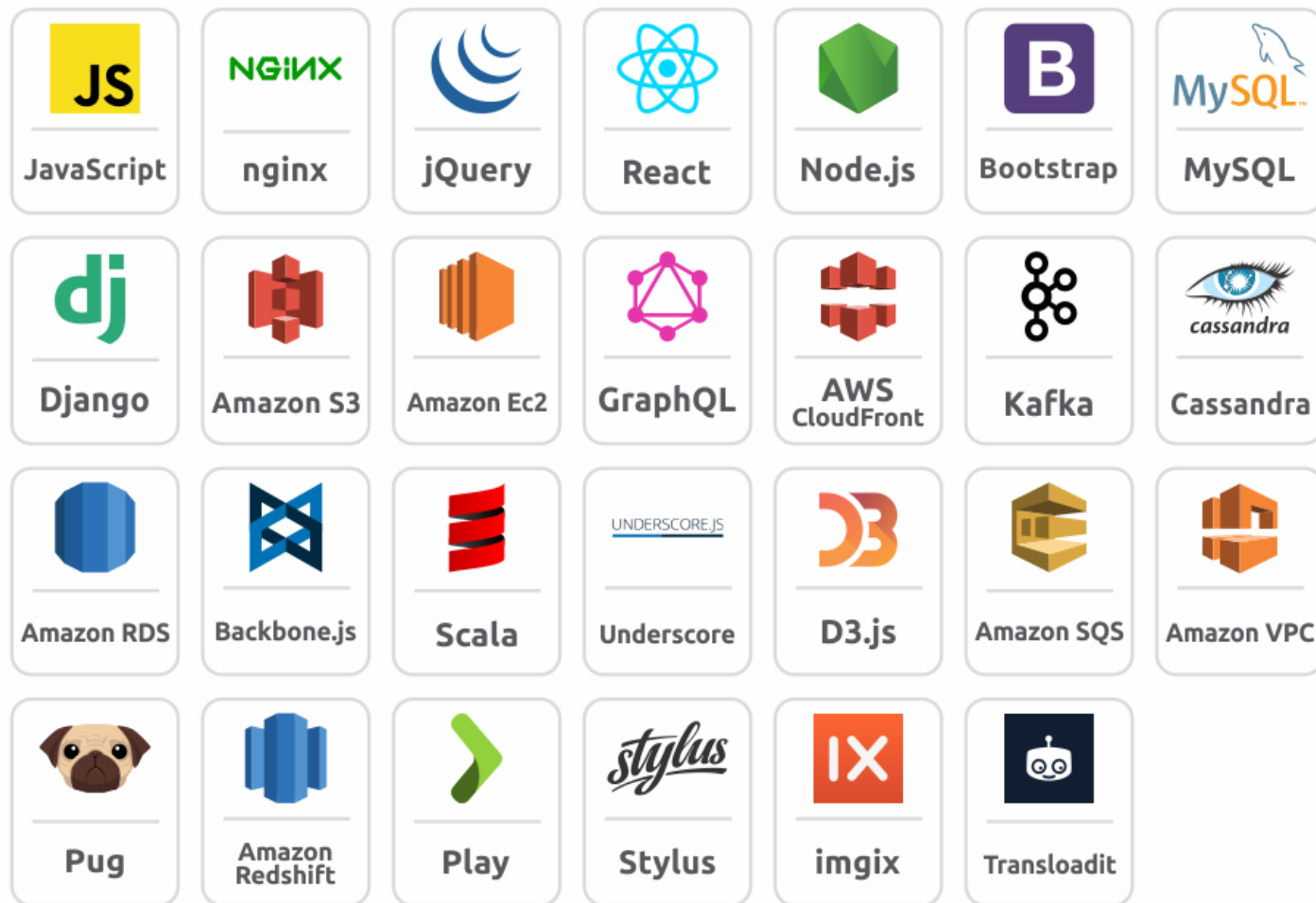
EdgeCast



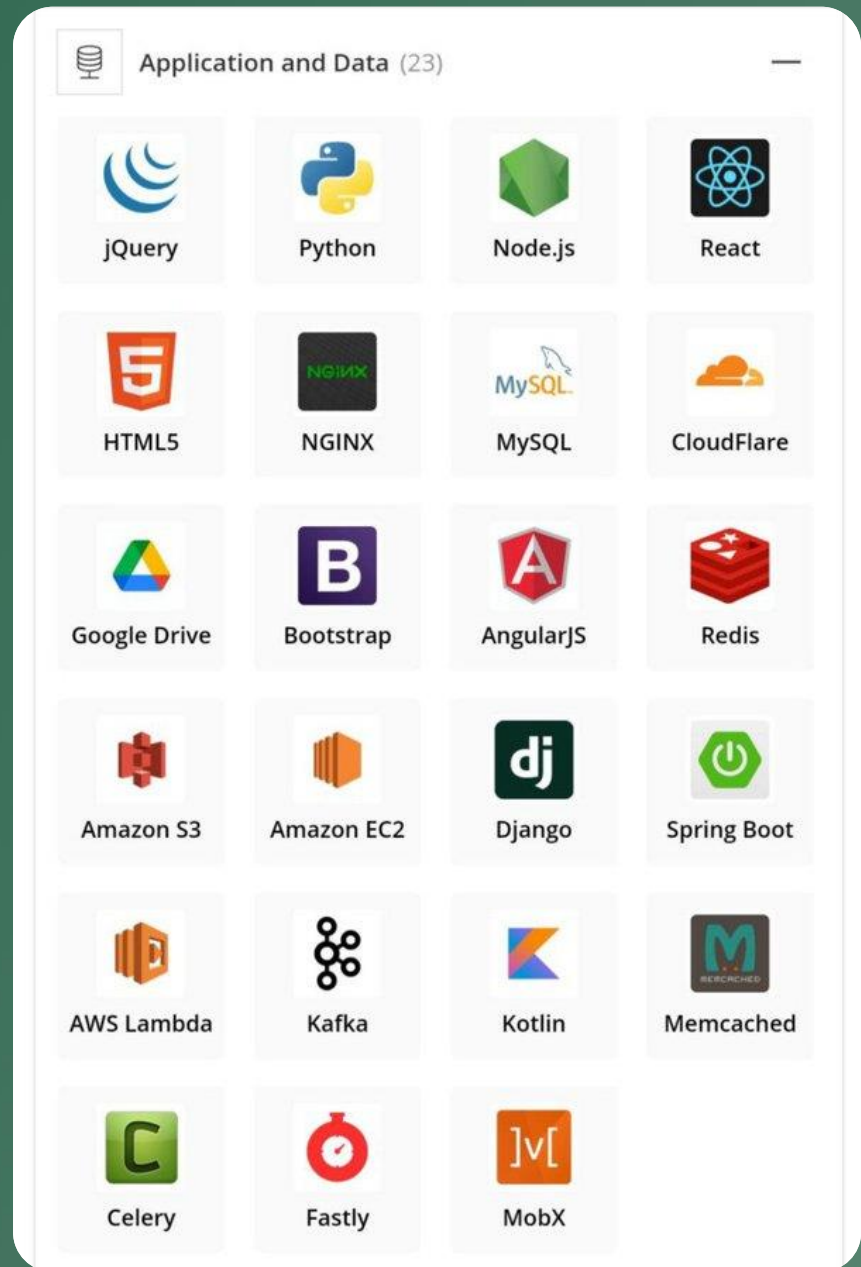
Qubole



mysql_utils



<https://stackshare.io/coursera/coursera>



เตรียมเครื่องมือ



| โปรแกรม (Tool and Editor) ที่ใช้อบรม

1. Python 3.13.x
2. Node JS 22.x
3. Visual Studio Code พร้อมส่วนเสริมที่จำเป็น
4. Git





Python



Python 3.13.x

<https://www.python.org/downloads/>

```
python --version
```

```
pip --version
```



Node JS 22.x

<https://nodejs.org/en/download/package-manager>

```
node -v
```

```
npm -v / npx -v
```



Visual Studio Code

<https://code.visualstudio.com/>

```
code -v
```



Git

<https://git-scm.com/>

```
git --version
```

Tool & Editor

NodeJS



Python 3.13.x

<https://www.python.org/downloads/>

```
python3 --version
```

```
pip3 --version
```



Node JS 22.x

<https://nodejs.org/en/download/package-manager>

```
node -v
```

```
npm -v / npx -v
```



Visual Studio Code

<https://code.visualstudio.com/>

```
code -v
```



Git

<https://git-scm.com/>

```
git --version
```



ติดตั้ง Python 3.13.x บน Windows



https://www.python.org

The screenshot shows the Python.org website with the 'Downloads' menu open. The 'Download for Windows' option is highlighted, and a red box is drawn around it. Below this, a button for 'Python 3.13.0' is visible, with a mouse cursor hovering over it. A note states: 'Note that Python 3.9+ cannot be used on Windows 7 or earlier.' Below this note, it says: 'Not the OS you are looking for? Python can be used on many operating systems and environments. View the full list of downloads.'

Download Python | Python.org

python.org/downloads/

Python PSF Docs PyPI Jobs Community

python™

Donate Search GO Socialize

About Downloads Documentation Community Success Stories News Events

Download

Download Python

Looking for Python

Linux/UNIX, macOS

Want to help test d

Docker images

All releases

Source code

Windows

macOS

Other Platforms

License

Alternative Implementations

Download for Windows

Python 3.13.0

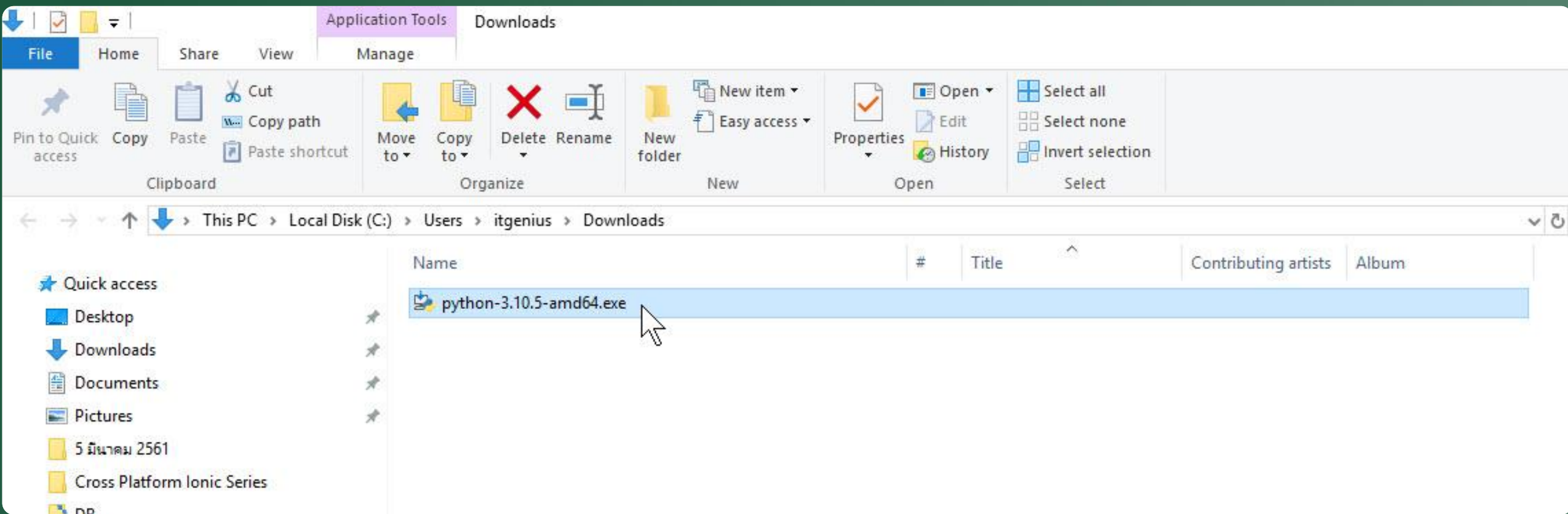
Note that Python 3.9+ cannot be used on Windows 7 or earlier.

Not the OS you are looking for? Python can be used on many operating systems and environments. View the full list of downloads.

⚡🐍⚡ Power Python with a 30% discount on PyCharm- all proceeds go to the Python Software Foundation. Offer ends soon, so grab it today!

GET 30% OFF PYCHARM

<https://www.python.org/ftp/python/3.13.0/python-3.13.0-amd64.exe>



Install Python 3.10.5 (64-bit)

Select Install Now to install Python with default settings, or choose Customize to enable or disable features.



Install Now

C:\Users\itgenius\AppData\Local\Programs\Python\Python310

Includes IDLE, pip and documentation
Creates shortcuts and file associations



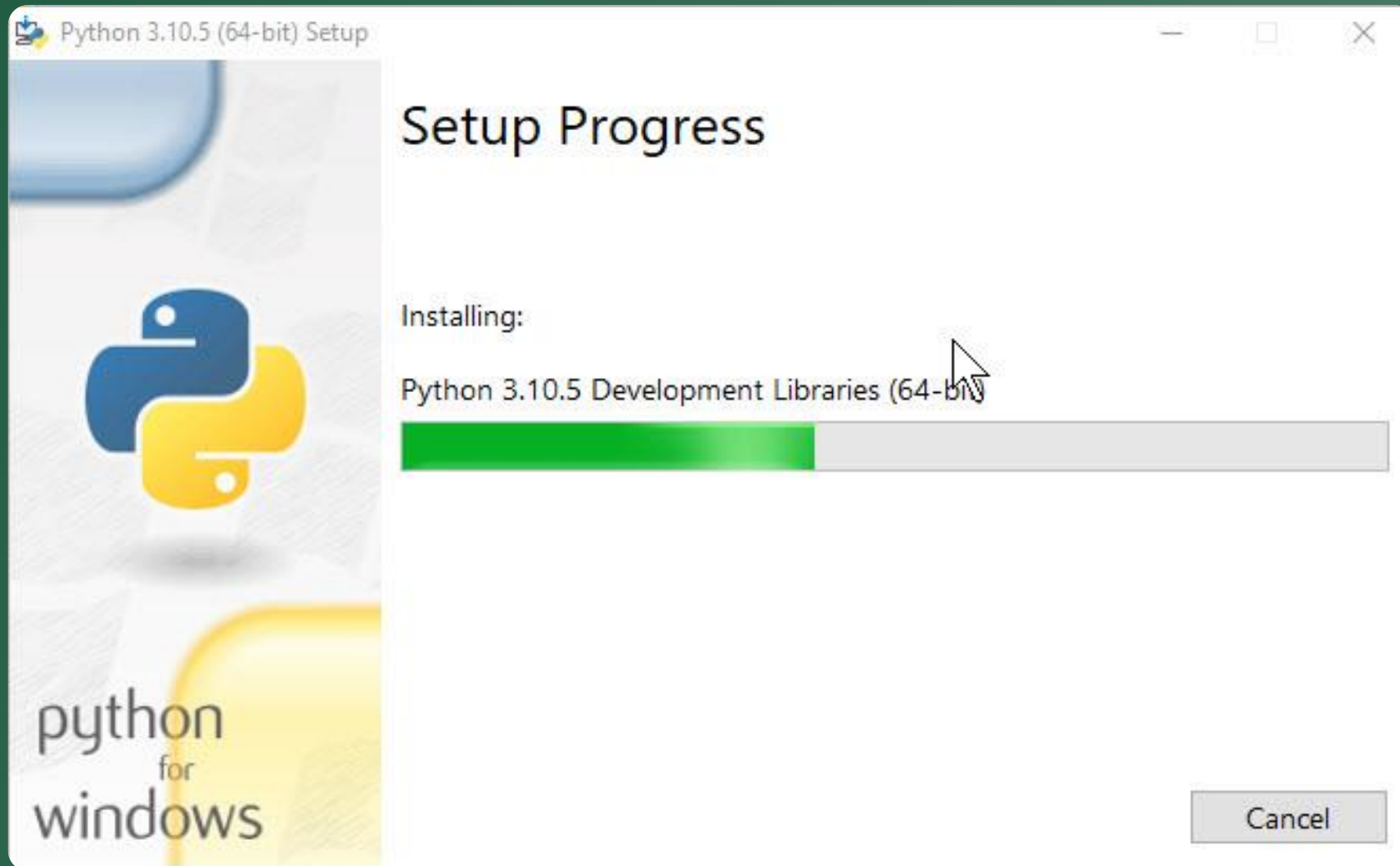
Customize installation

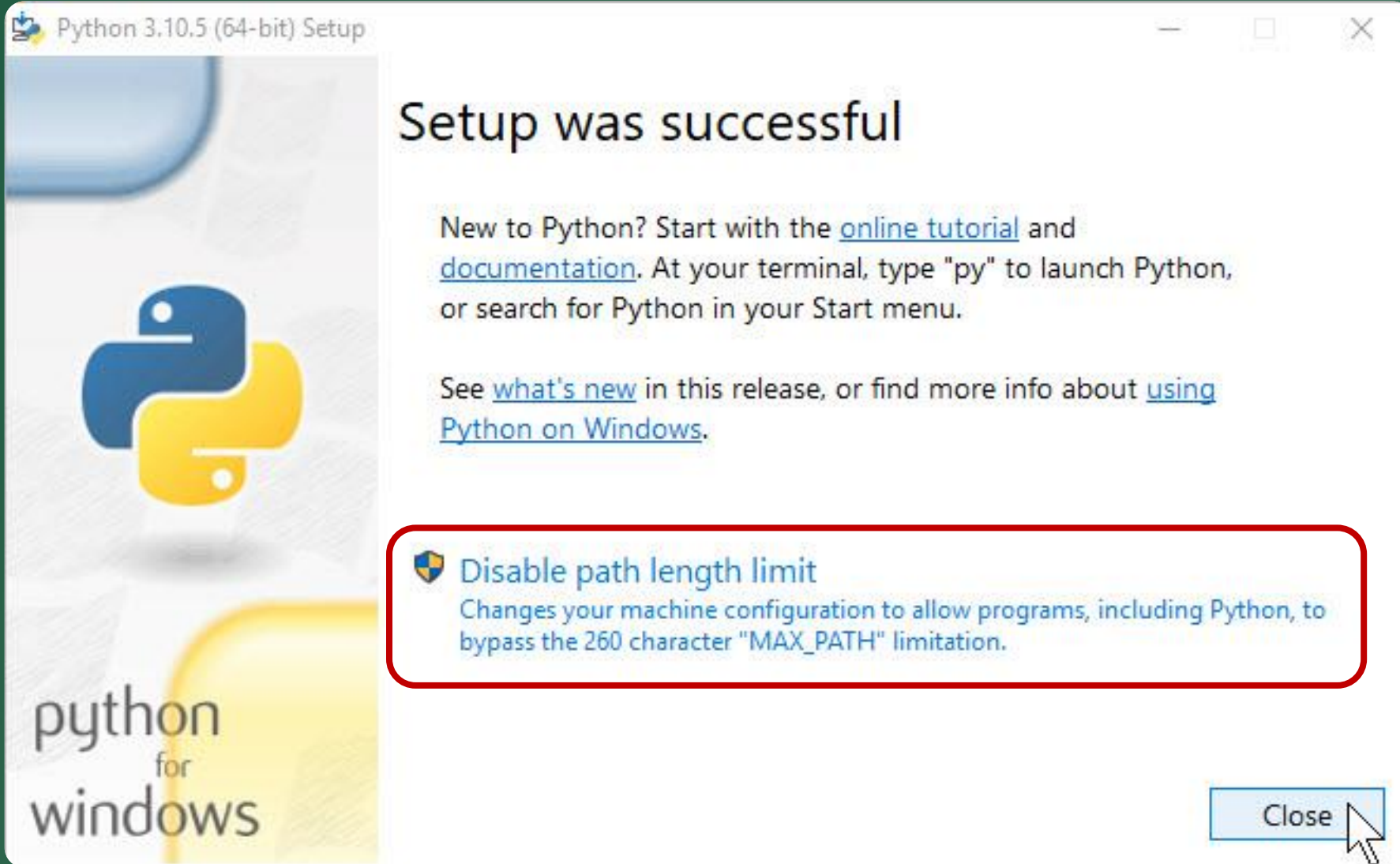
Choose location and features

☒ Install launcher for all users (recommended)

☒ Add Python 3.10 to PATH

Cancel





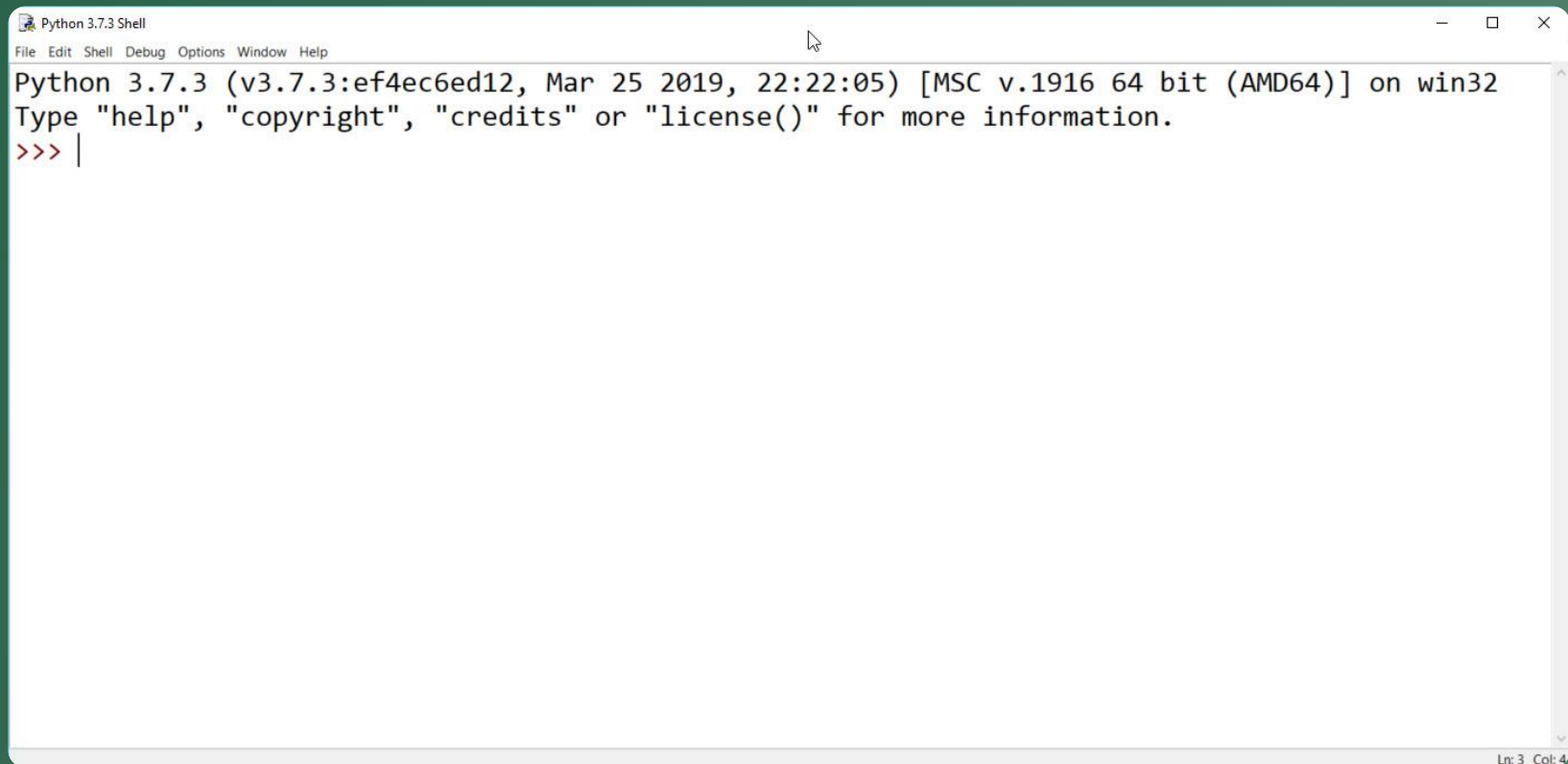
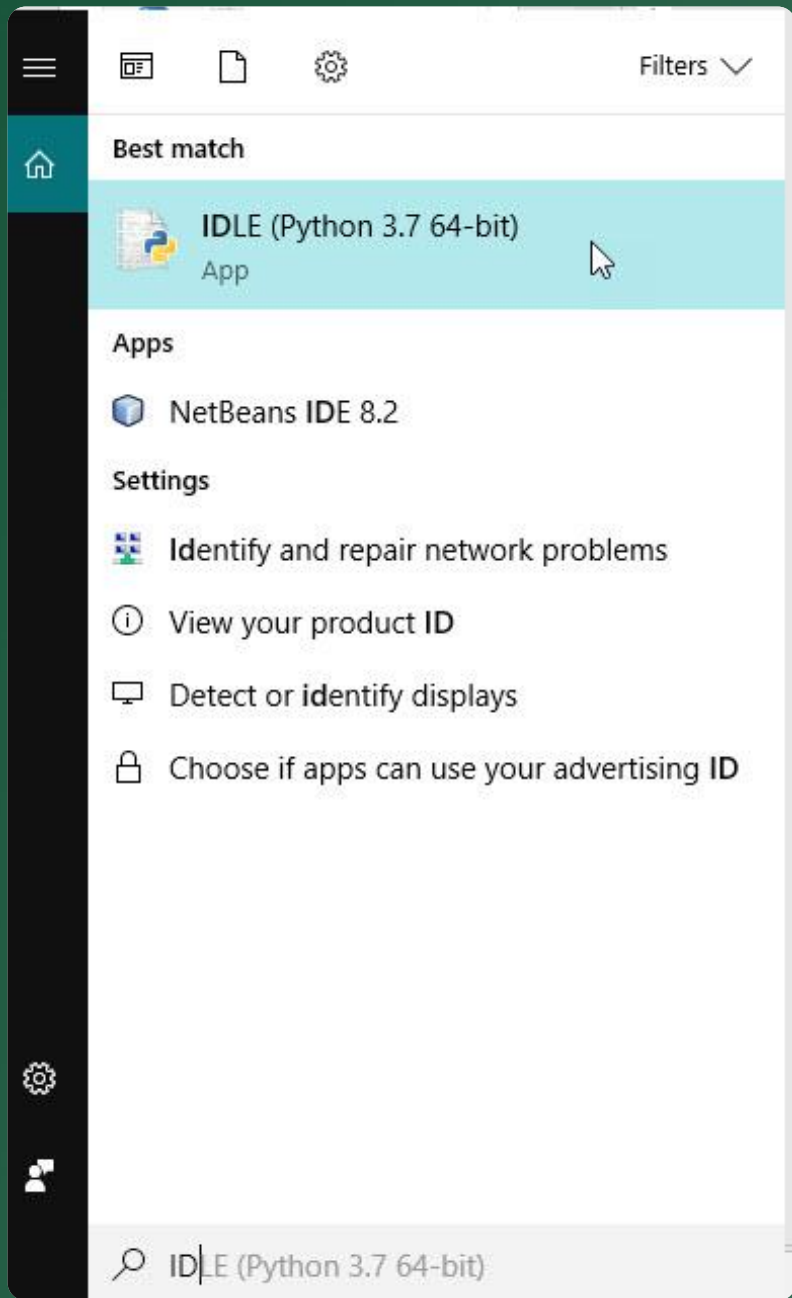


Setup was successful

New to Python? Start with the [online tutorial](#) and [documentation](#). At your terminal, type "py" to launch Python, or search for Python in your Start menu.

See [what's new](#) in this release, or find more info about [using Python on Windows](#).

Close



CA Select Command Prompt

Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\itgenius>python --version
Python 3.10.5

C:\Users\itgenius>■

Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\itgenius>python --version
Python 3.10.5

C:\Users\itgenius>python -V
Python 3.10.5

C:\Users\itgenius>

CA Select Command Prompt

Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\itgenius>python --version
Python 3.10.5

C:\Users\itgenius>python -V
Python 3.10.5

C:\Users\itgenius>where python
C:\Users\itgenius\AppData\Local\Programs\Python\Python310\python.exe

C:\Users\itgenius>■

Microsoft Windows [Version 10.0.17134.1304]

(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\itgenius>python --version
Python 3.10.5

C:\Users\itgenius>python -V
Python 3.10.5

C:\Users\itgenius>where python
C:\Users\itgenius\AppData\Local\Programs\Python\Python310\python.exe

C:\Users\itgenius>pip list

Package	Version
---------	---------

pip	22.0.4
-----	--------

setuptools	58.1.0
------------	--------

WARNING: You are using pip version 22.0.4; however, version 22.1.2 is available.

You should consider upgrading via the 'C:\Users\itgenius\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip' command.

C:\Users\itgenius>

(c) 2018 Microsoft Corporation. All rights reserved.

```
C:\Users\itgenius>python --version  
Python 3.10.5
```

```
C:\Users\itgenius>python -V  
Python 3.10.5
```

```
C:\Users\itgenius>where python  
C:\Users\itgenius\AppData\Local\Programs\Python\Python310\python.exe
```

```
C:\Users\itgenius>pip list  
Package      Version
```

```
-----  
pip          22.0.4  
setuptools   58.1.0
```

WARNING: You are using pip version 22.0.4; however, version 22.1.2 is available.
You should consider upgrading via the 'C:\Users\itgenius\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip' command.

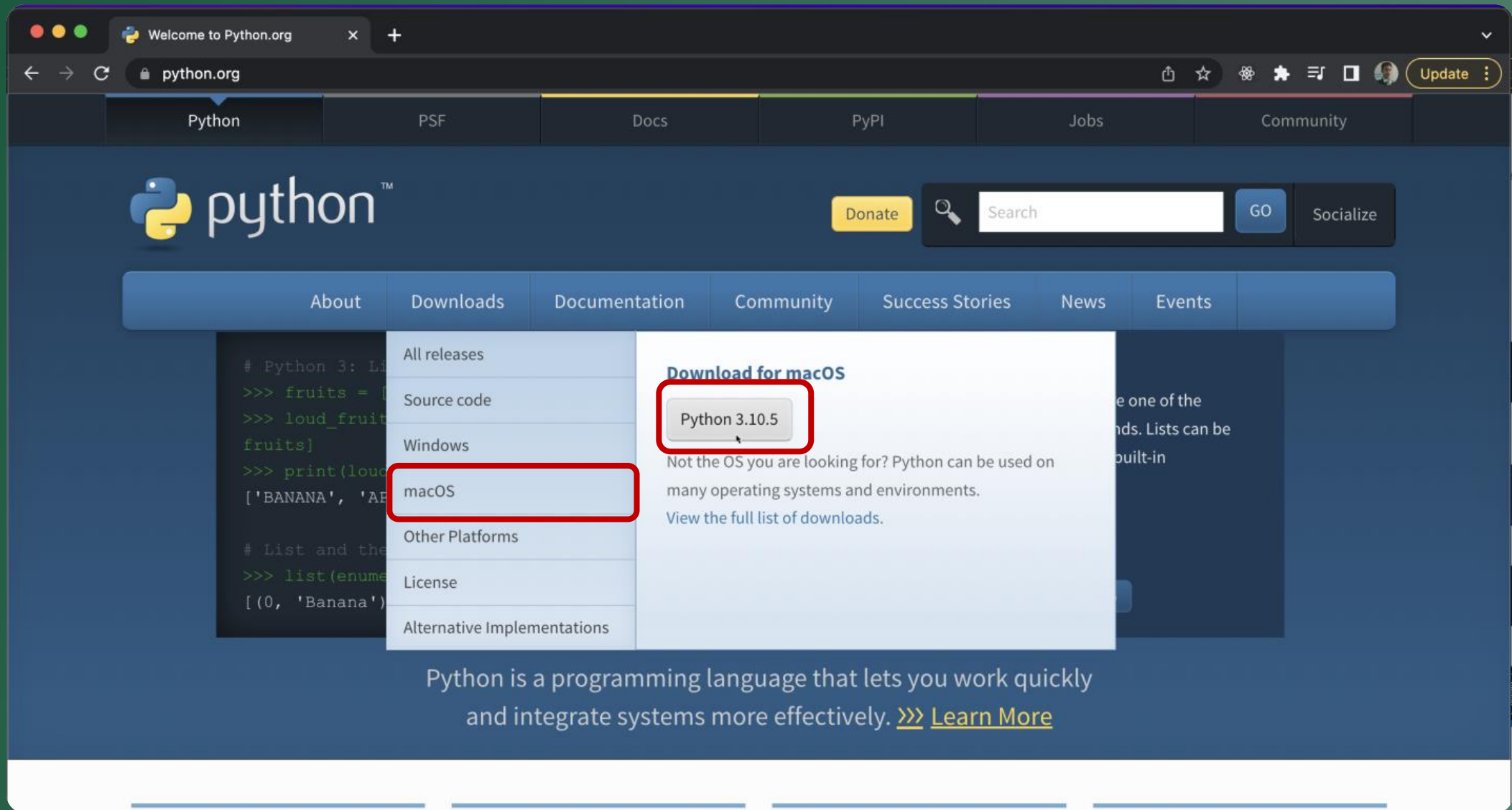
```
C:\Users\itgenius>pip freeze
```

```
C:\Users\itgenius>
```



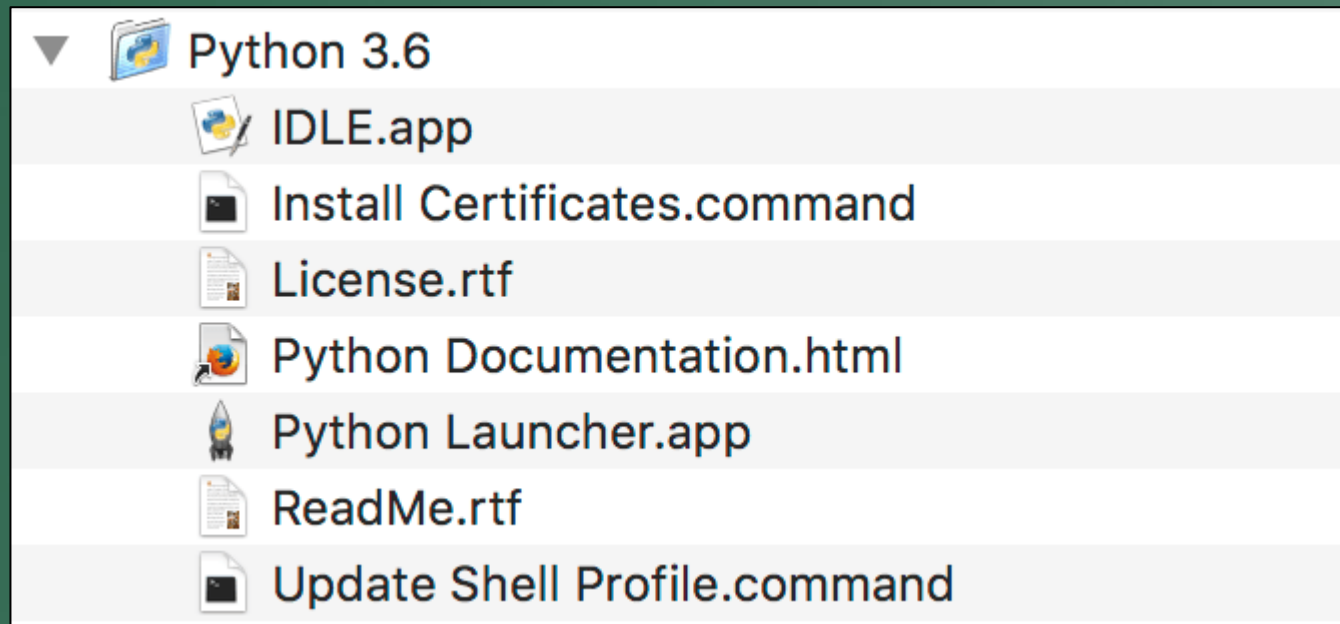
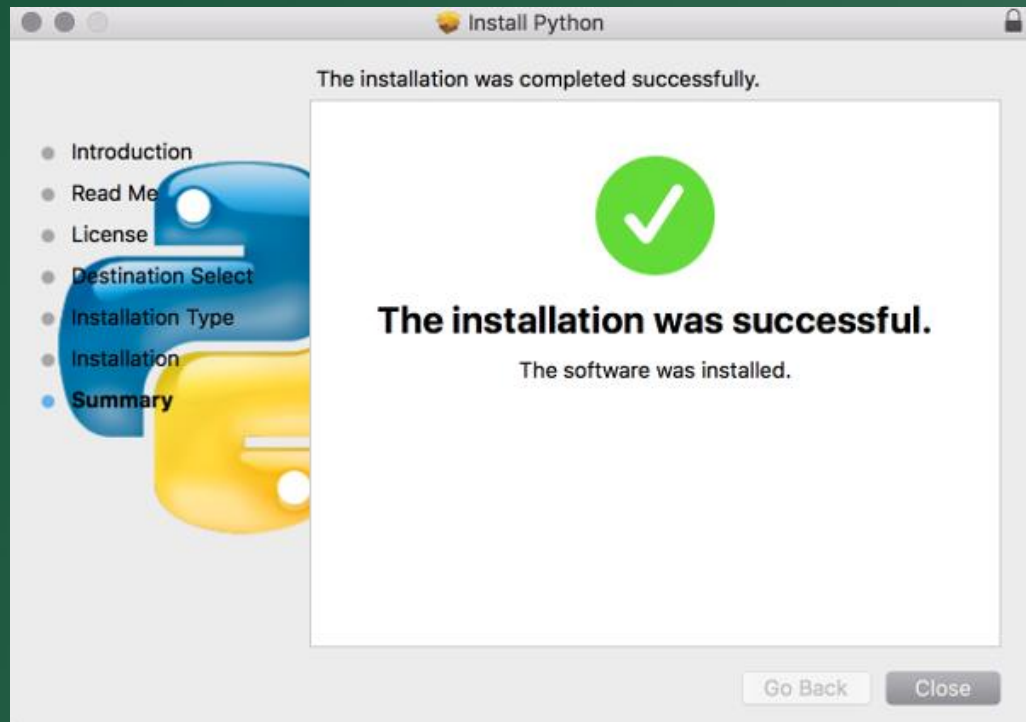
ติดตั้ง Python 3.13.x บน MacOS





<https://www.python.org>





`python3 --version`

`pip3 --version`

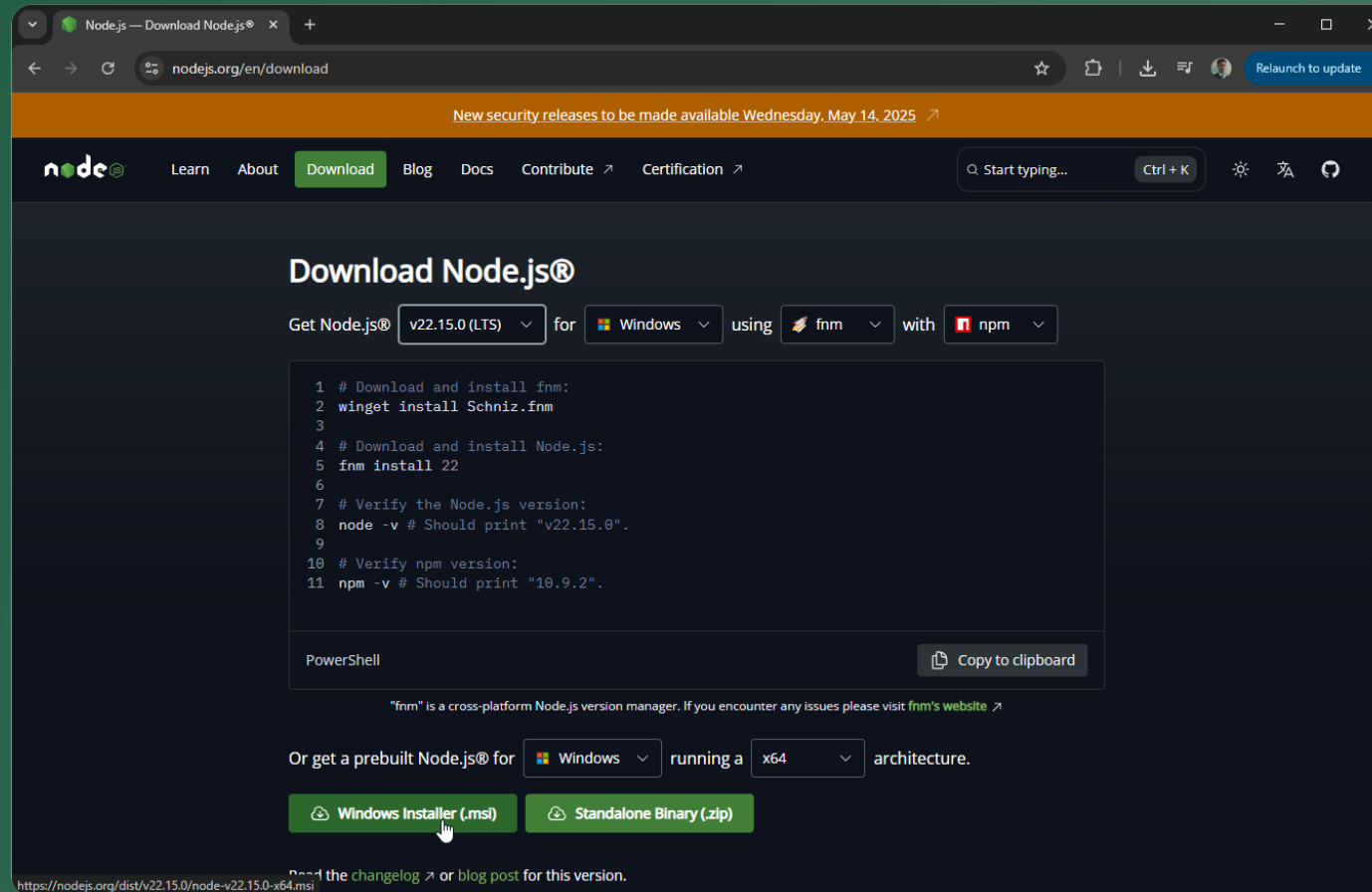


1. ติดตั้ง Node JS



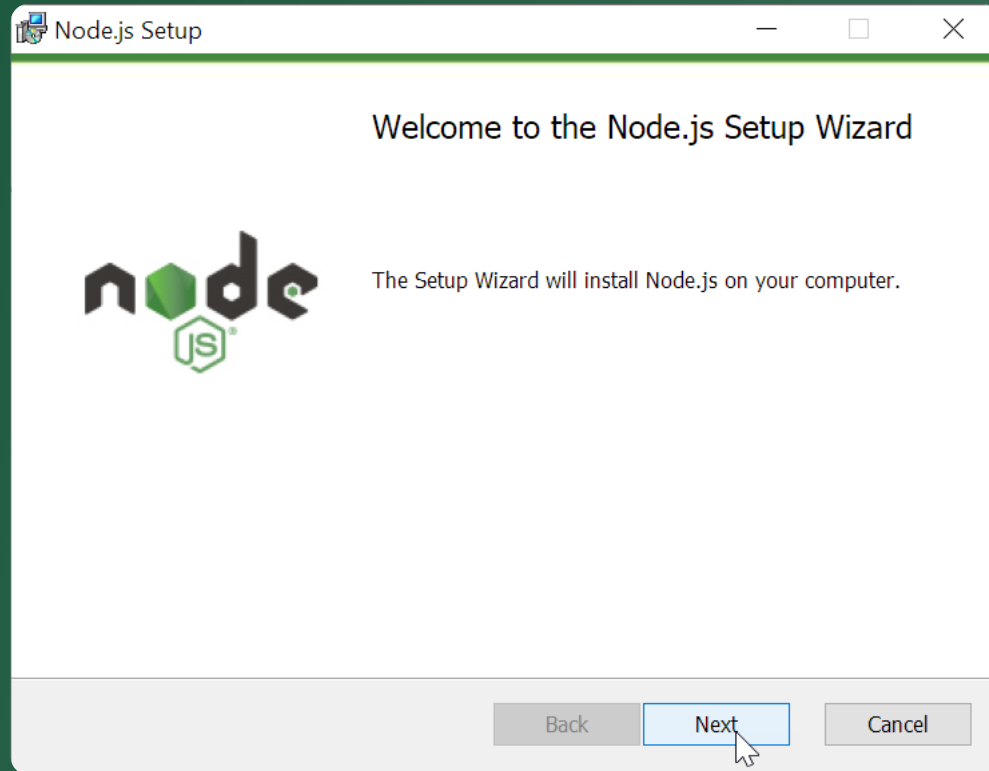
Download Node.JS V.22.x

<https://nodejs.org/en/>

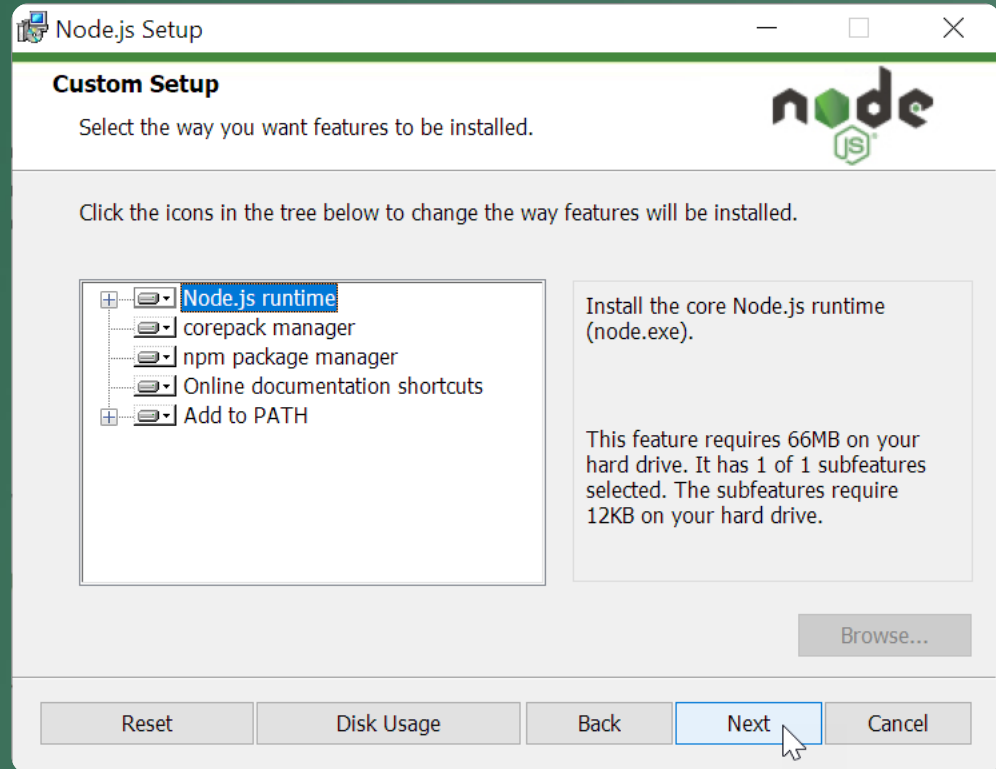
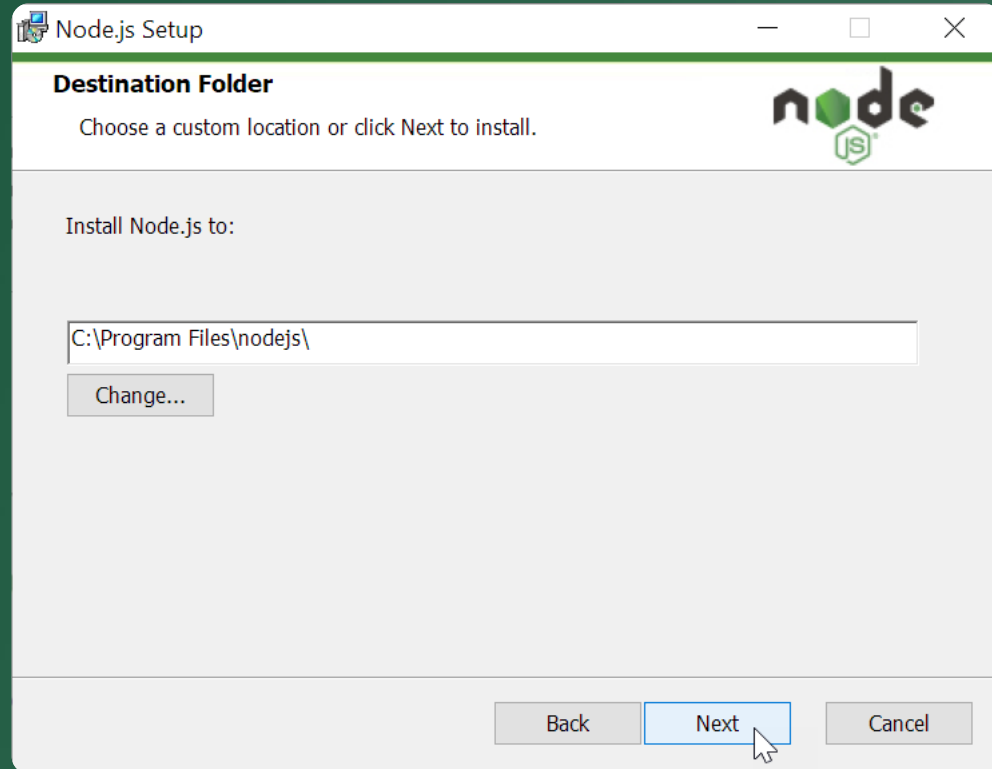


หมายเหตุ การอบรมรองรับตั้งแต่ Node.JS 20 ขึ้นไป สามารถใช้ Node.JS 21 , 22, 23 หรือ 24 ก็ได้

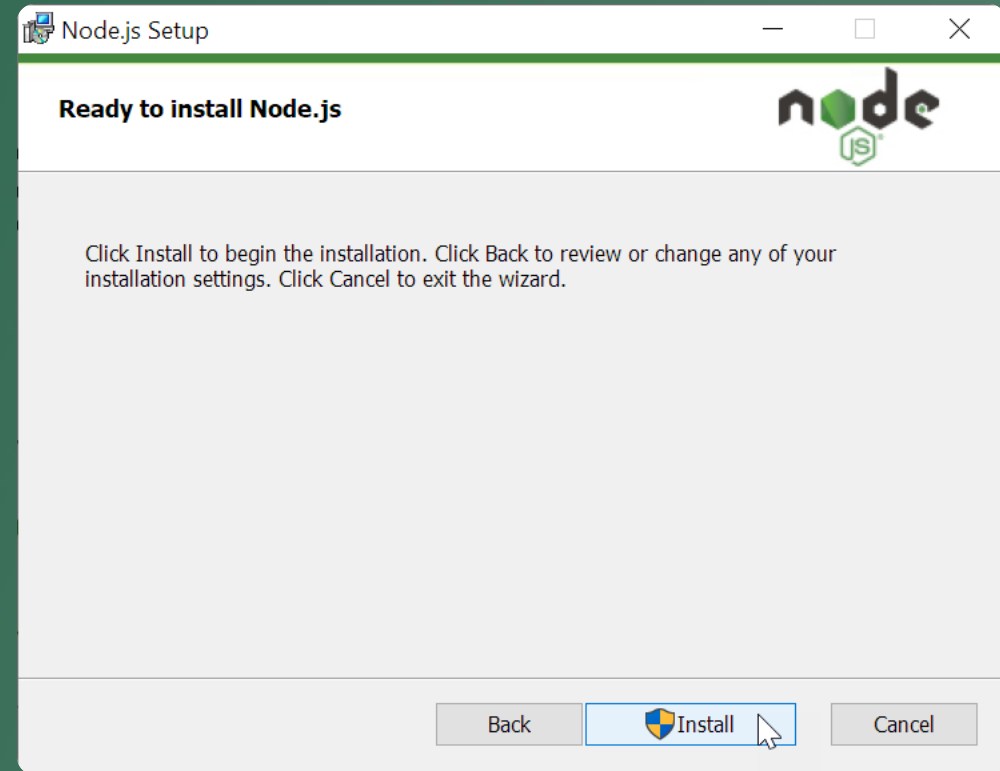
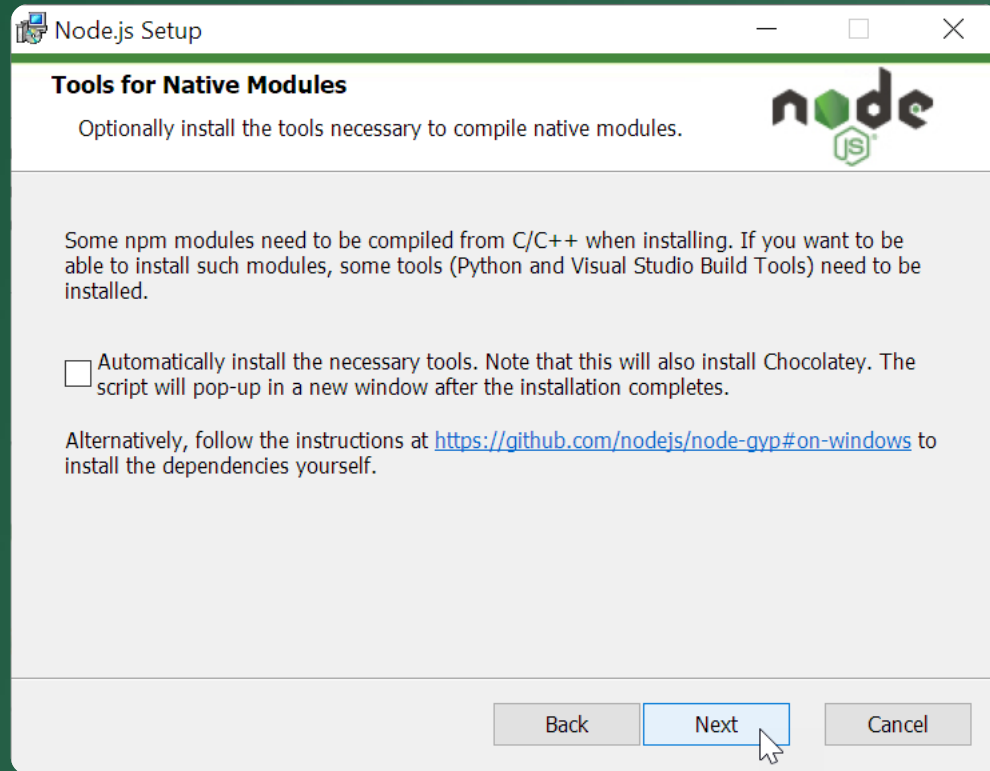
ติดตั้ง Node.JS V.22.x



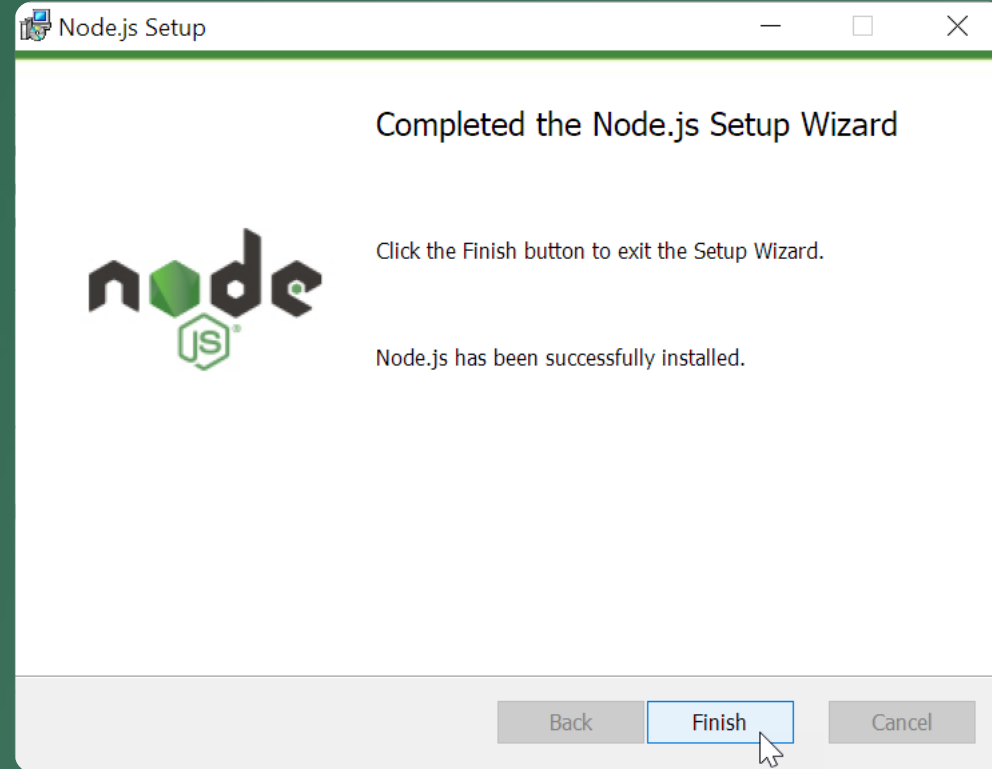
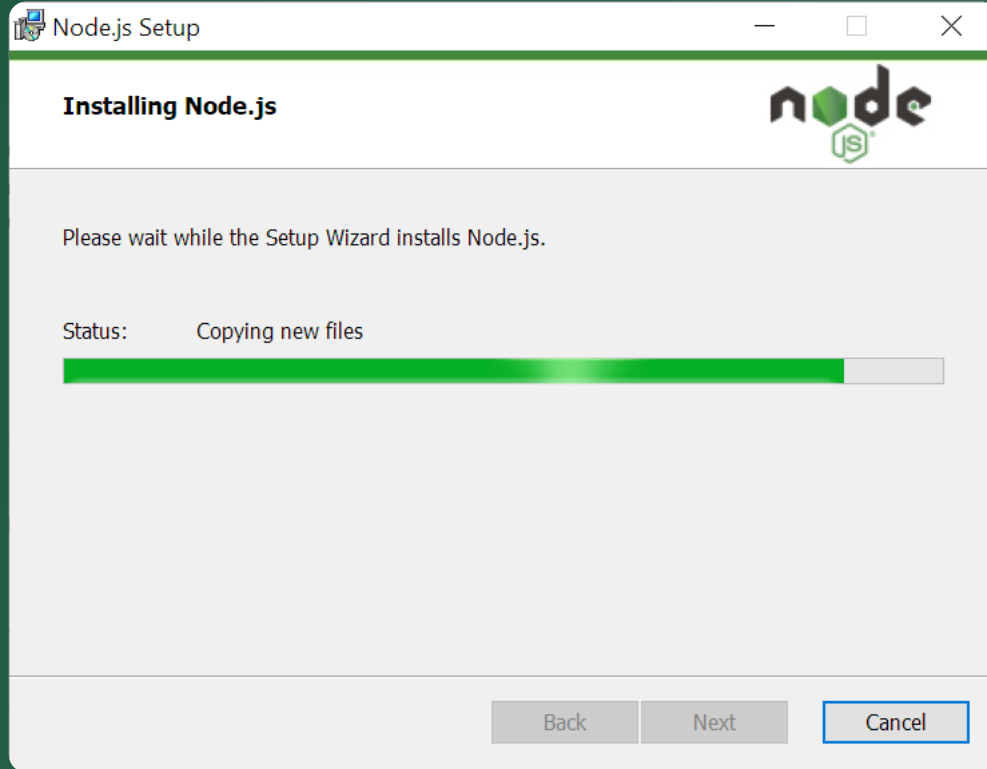
ติดตั้ง Node.JS V.22.x



ติดตั้ง Node.JS V.22.x



ติดตั้ง Node.JS V.22.x



ทดสอบหลังติดตั้งเสร็จ

node -v

```
C:\Users\Samit>node -v  
v22.14.0
```

```
C:\Users\Samit>|
```

npx -v

```
C:\Users\Samit>npx -v  
10.9.2
```

```
C:\Users\Samit>|
```

npm -v

```
C:\Users\Samit>npm -v  
10.9.2
```

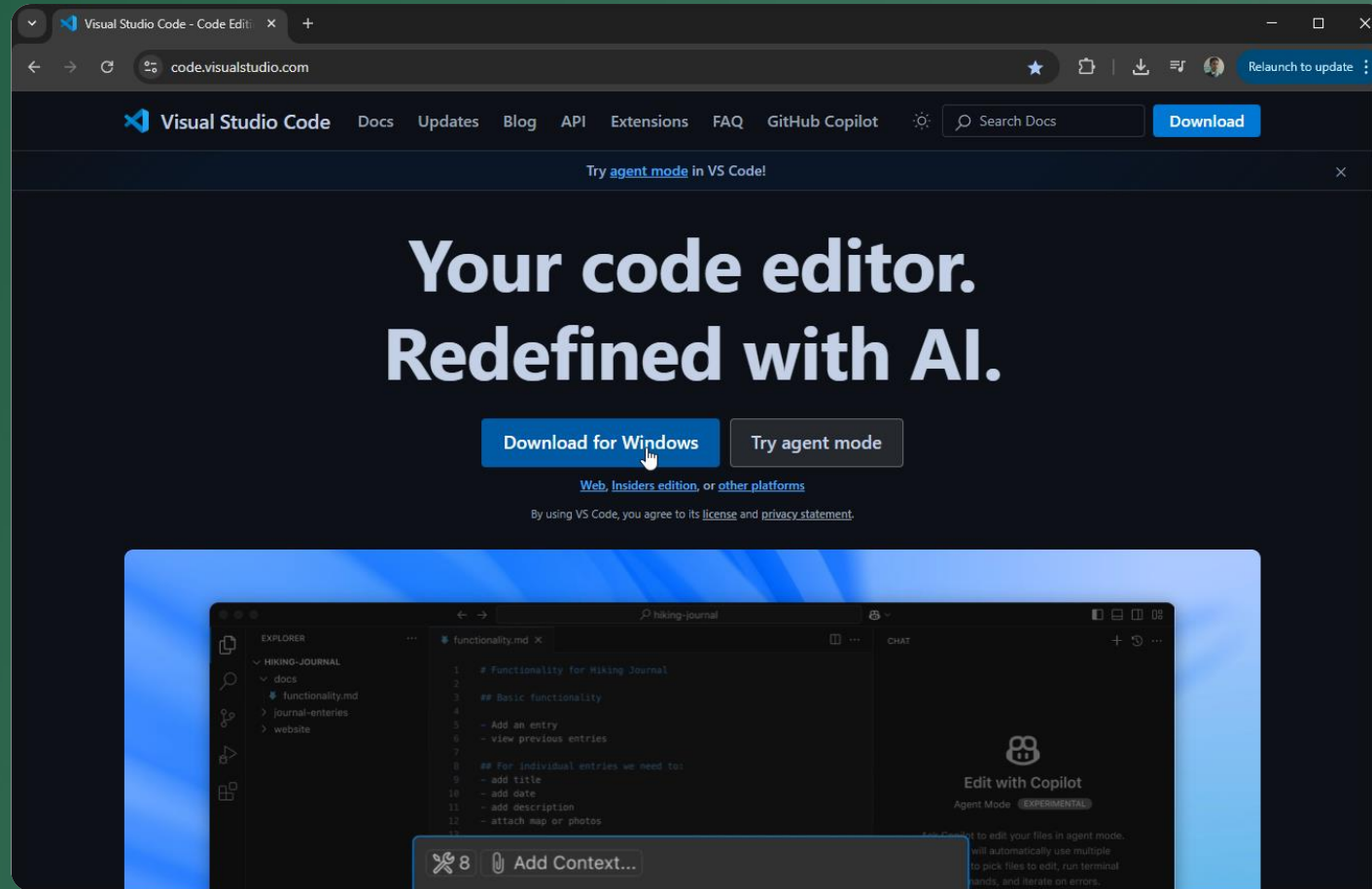
```
C:\Users\Samit>|
```



ติดตั้ง Visual Studio Code

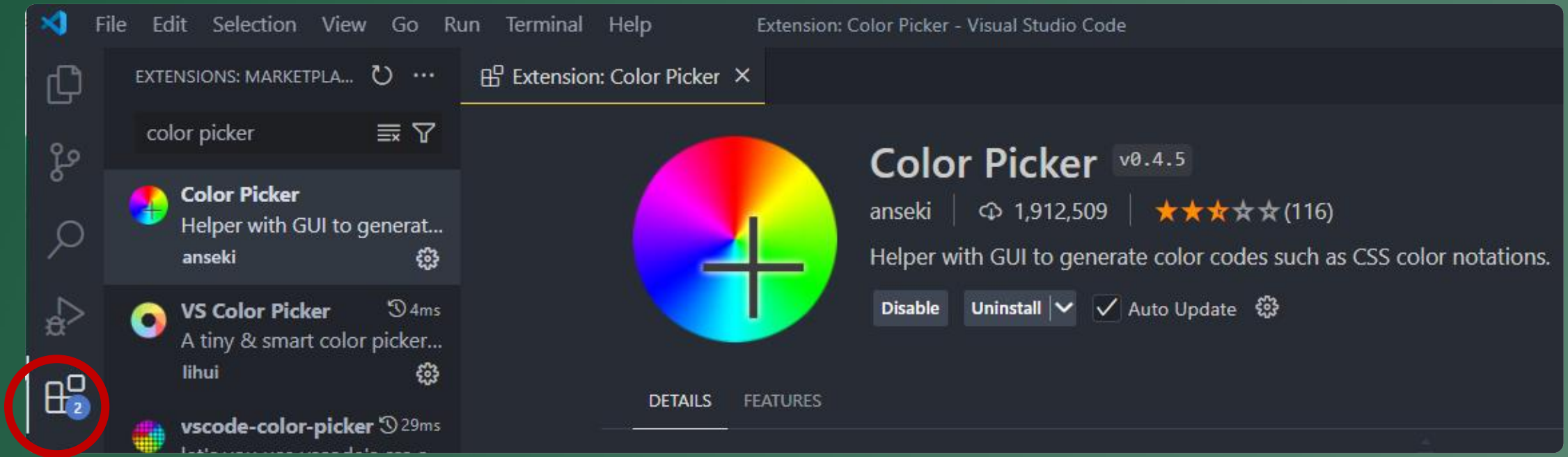


ติดตั้ง Visual Studio Code พร้อมส่วนเสริมที่จำเป็น



เข้าไปดาวน์โหลด Visual Studio Code ได้ที่ <https://code.visualstudio.com>

การติดตั้งส่วนเสริม (Extension) ของ Visual Studio Code



รายการ Extension ที่แนะนำสำหรับ Python Django

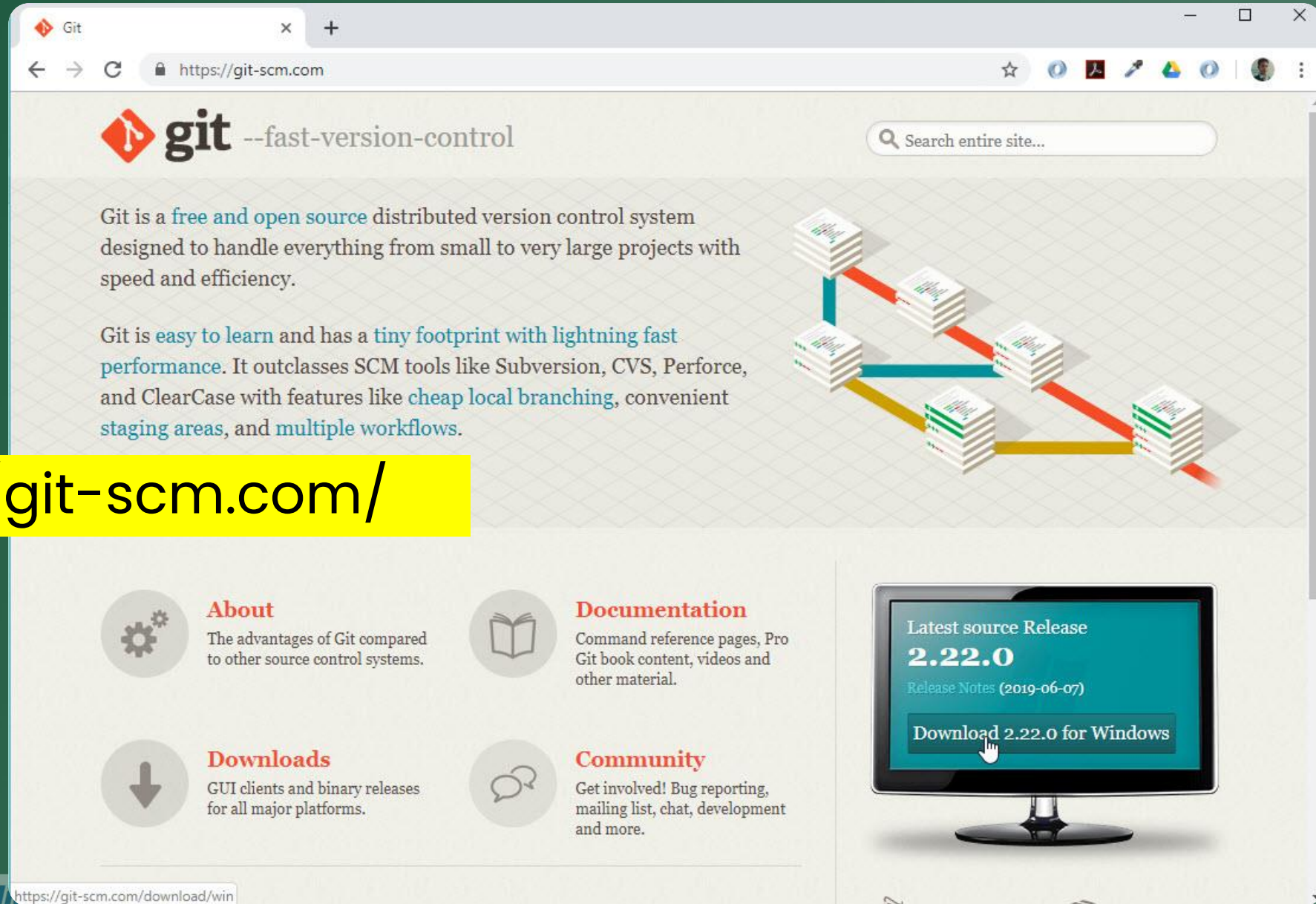
Extension พื้นฐานสำหรับ Python & Django

1. **Color Picker** by anseki
2. **Color Highlight** by Sergii N
3. **AutoFileName** by JerryHong
4. **Python** by Microsoft
5. **Django** by Baptiste Darthenay
6. **Material Icon Theme** by Philipp Kief
7. **Prettier - Code formatter** by Prettier
8. **Thunder Client** by Thunder Client
9. **One Dark Pro** by binaryify

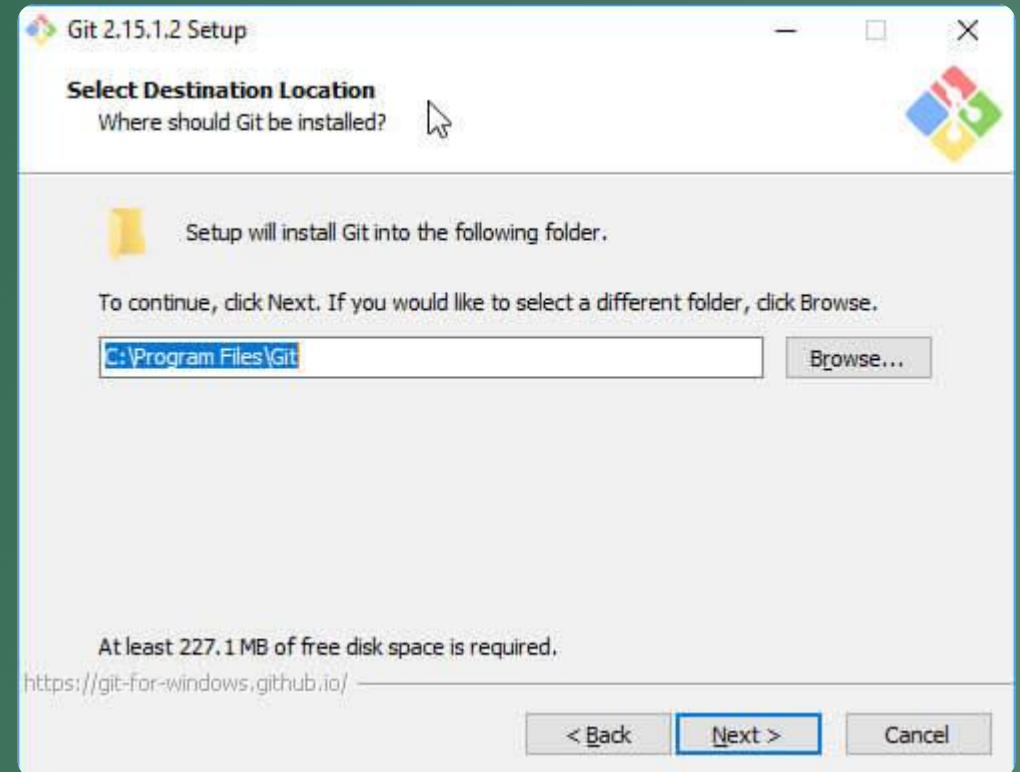


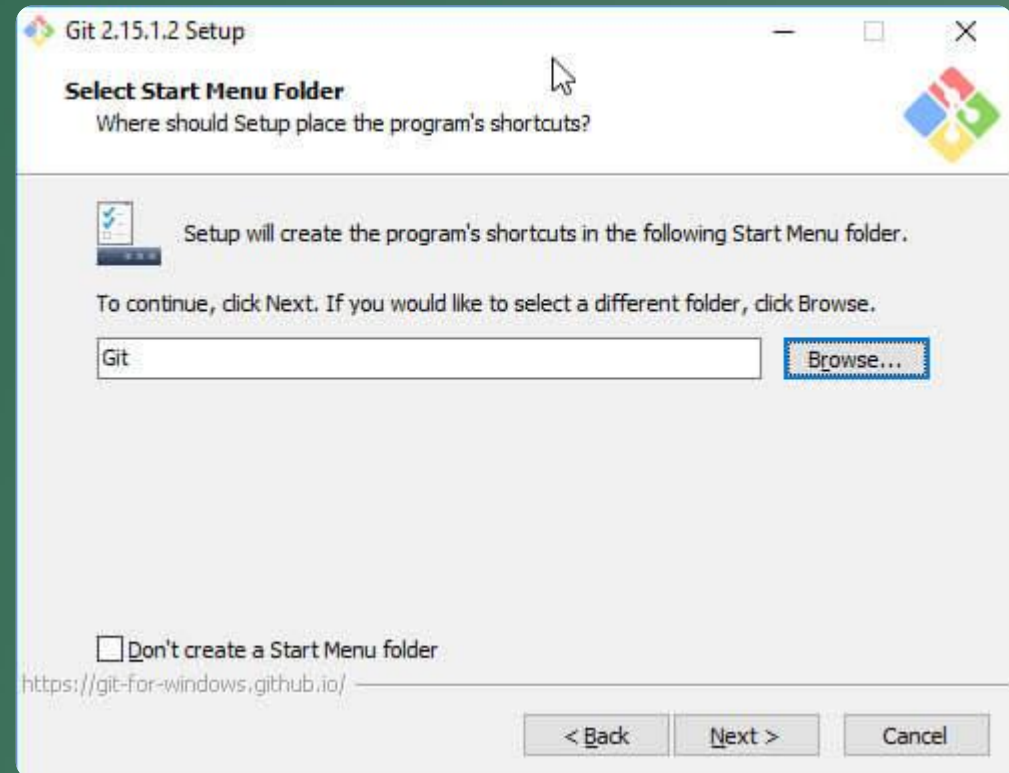
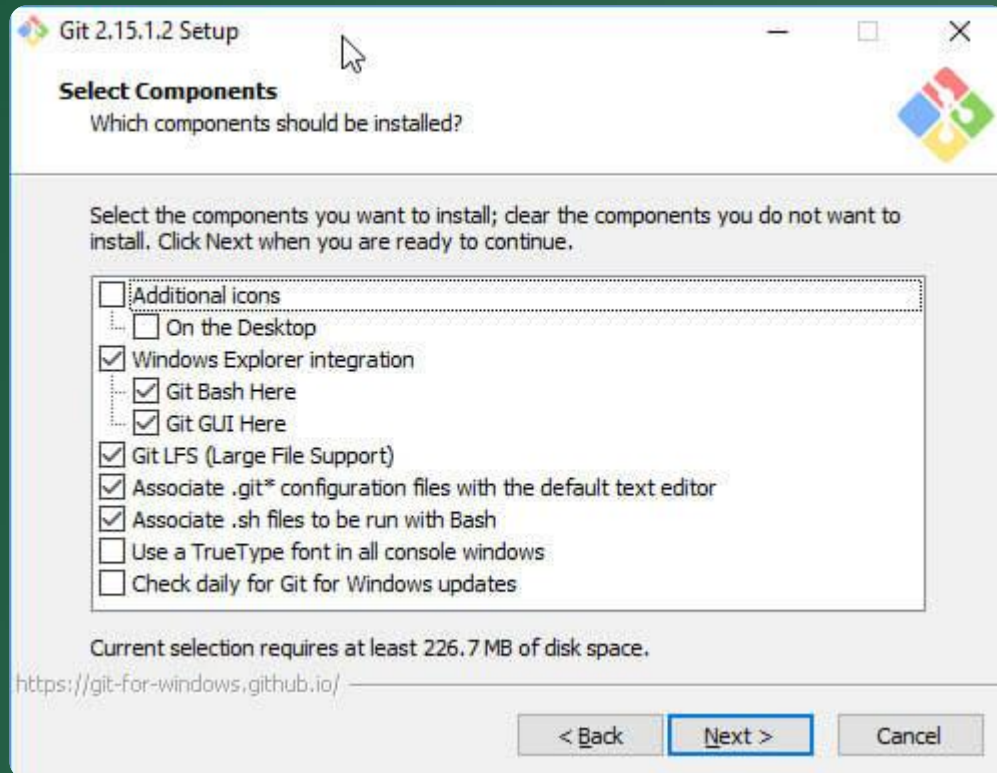
ติดตั้ง git

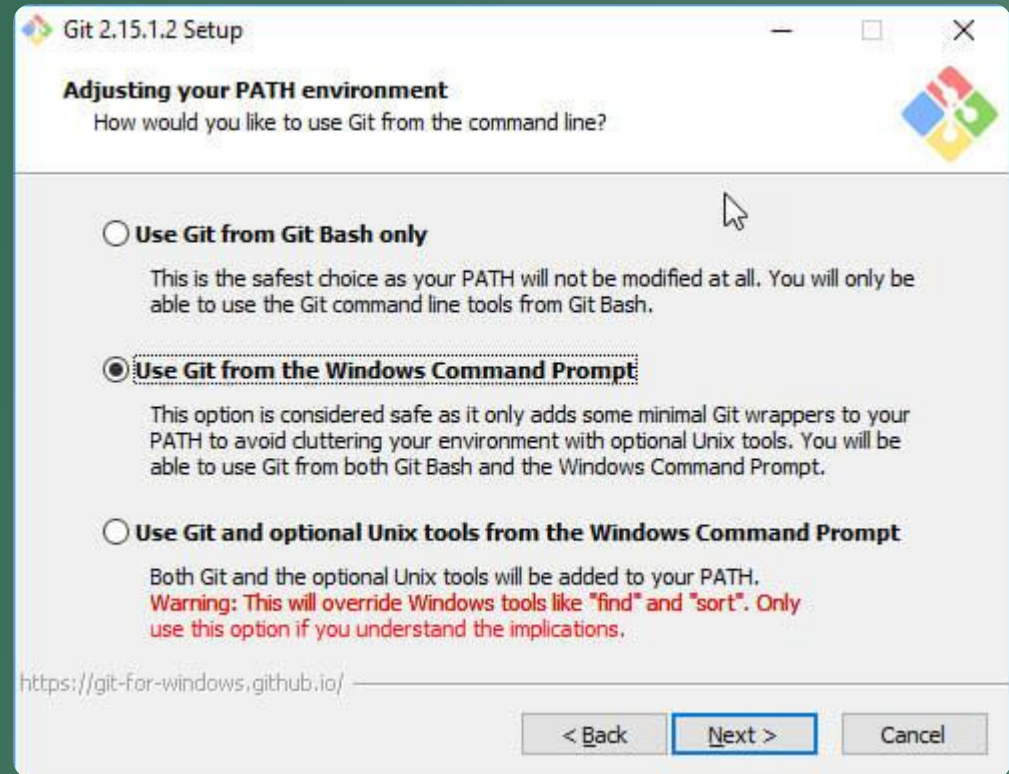
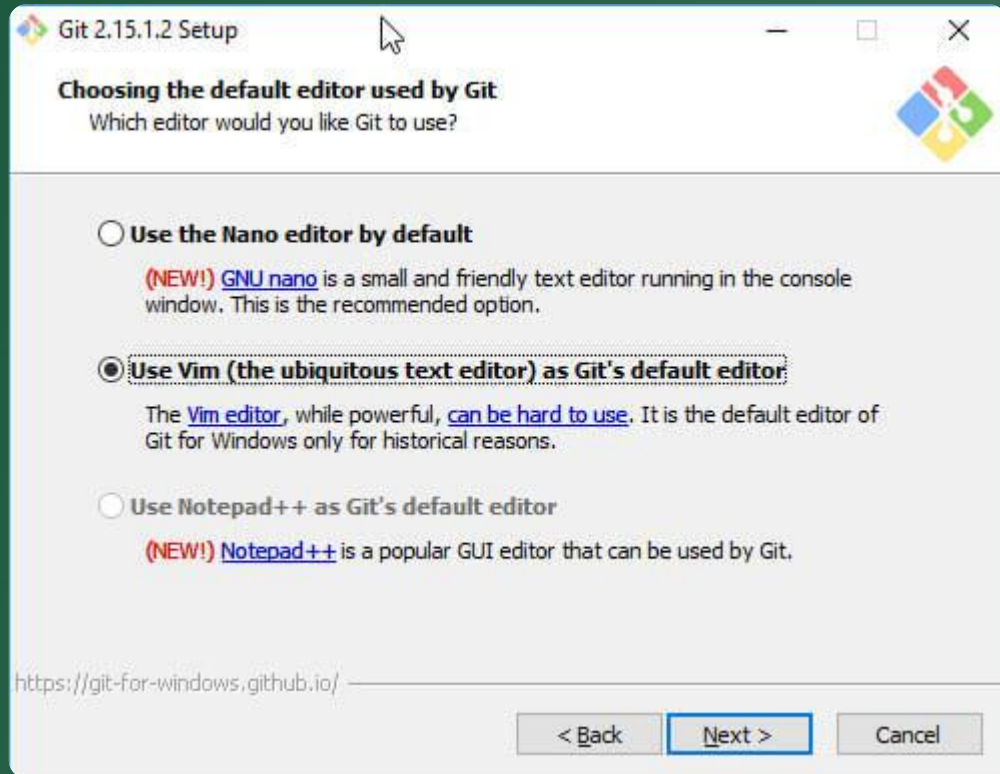


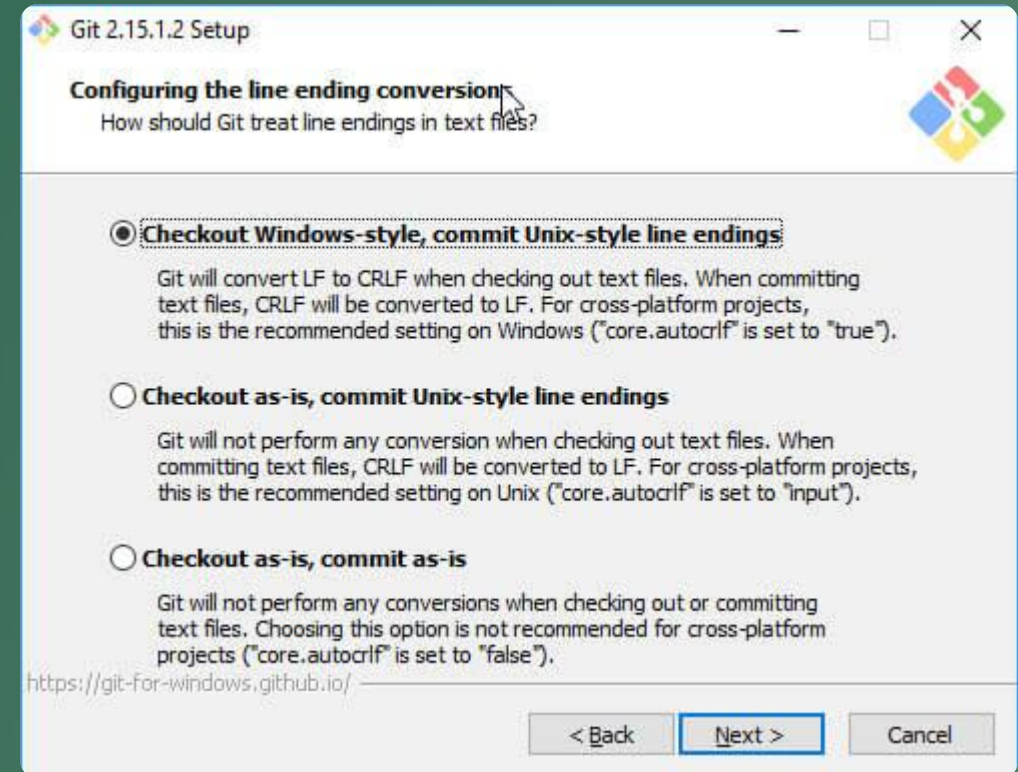
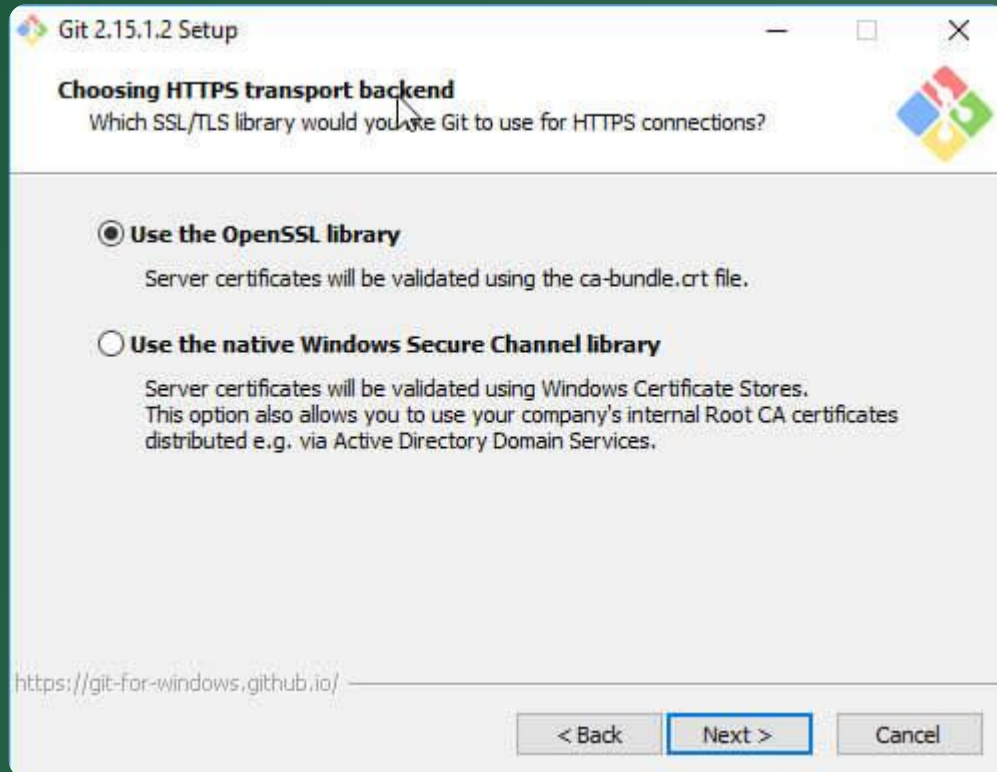


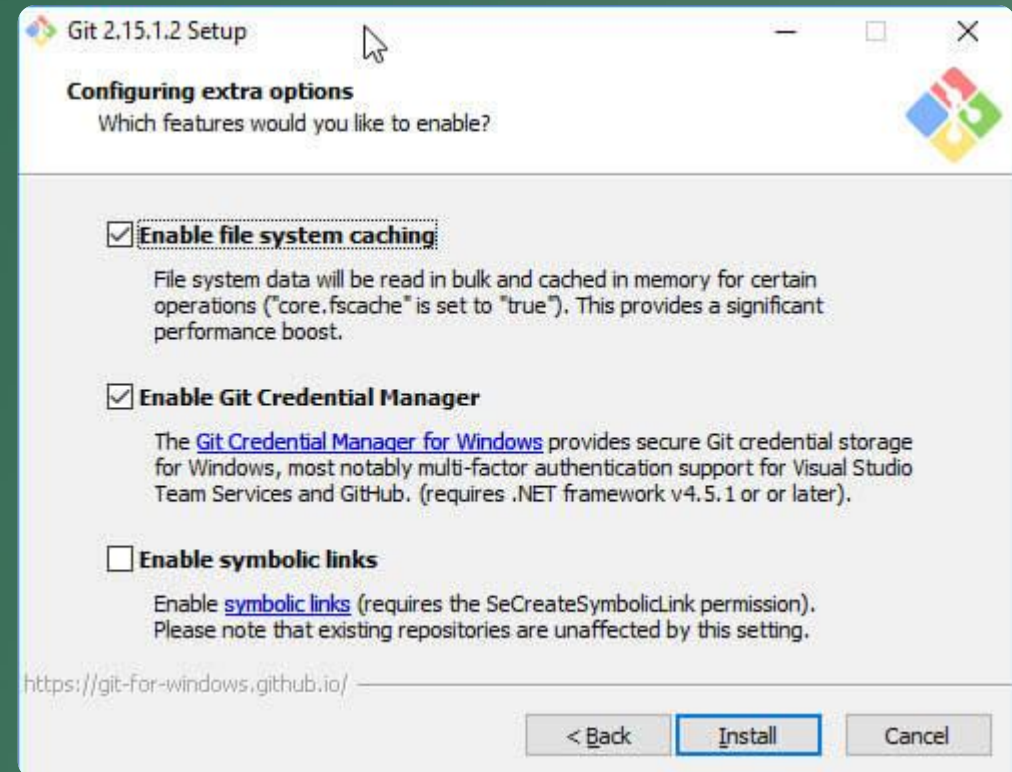
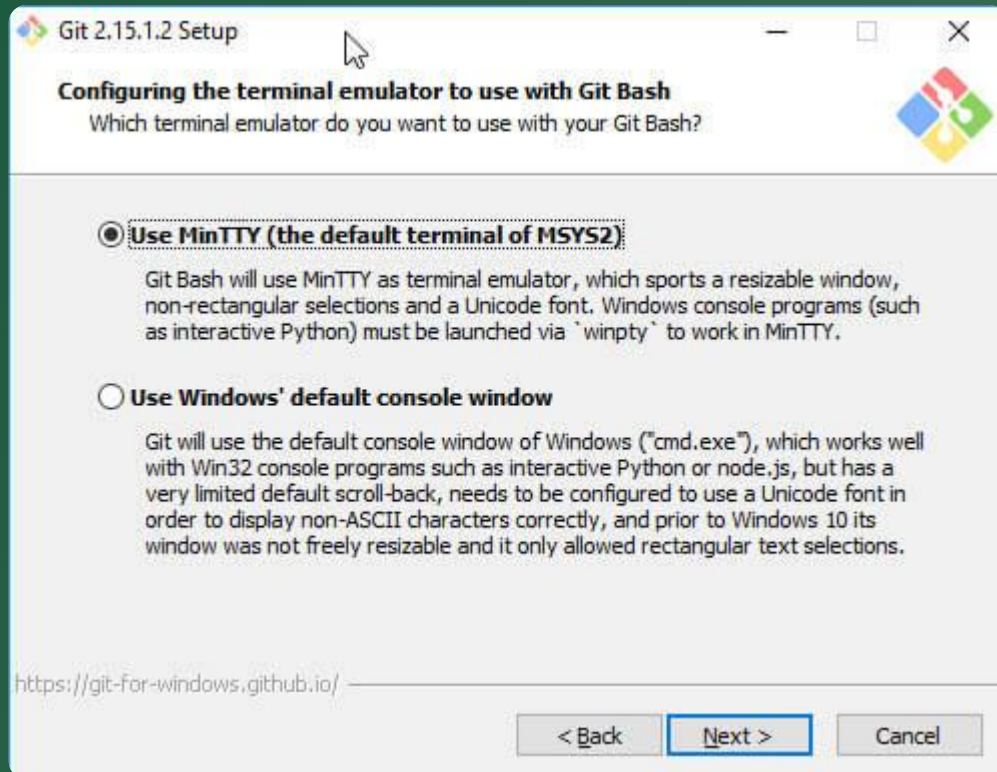
<https://git-scm.com/>

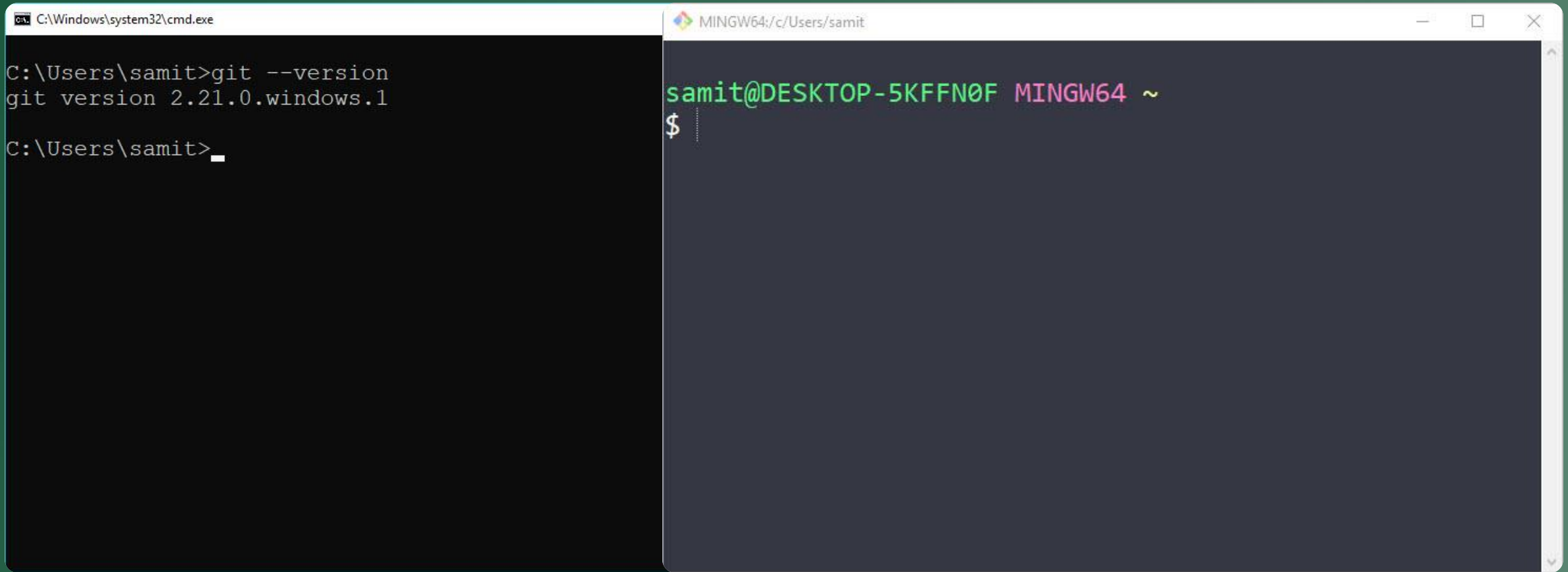












```
C:\Windows\system32\cmd.exe
C:\Users\samit>git --version
git version 2.21.0.windows.1
C:\Users\samit>_

MINGW64:/c/Users/samit
samit@DESKTOP-5KFFNØF MINGW64 ~
$
```

git version

Installing Virtualenv

Install Virtualenv



```
python -m venv name
```

example

```
D:\BasicPythonWisdom>python -m venv env
```

```
\Include  
\Lib  
\Scripts  
pyvenv.cfg
```



```
$ python3 -m venv name
```

example

```
$ python3 -m venv env
```

```
/bin  
/Include  
/Lib  
pyvenv.cfg
```

Activate virtual environment



```
env\Scripts\activate
```

```
Command Prompt
D:\BasicPythonEGAT>env\Scripts\activate
(env) D:\BasicPythonEGAT>
```



```
$ source env/bin/activate
```

```
(env) samit@MacbookM1Pro pythonbeginner % pip list
Package      Version
-----
pip           22.1.2
setuptools    49.2.1
(env) samit@MacbookM1Pro pythonbeginner % python hello.py
Hello Python
(env) samit@MacbookM1Pro pythonbeginner %
```

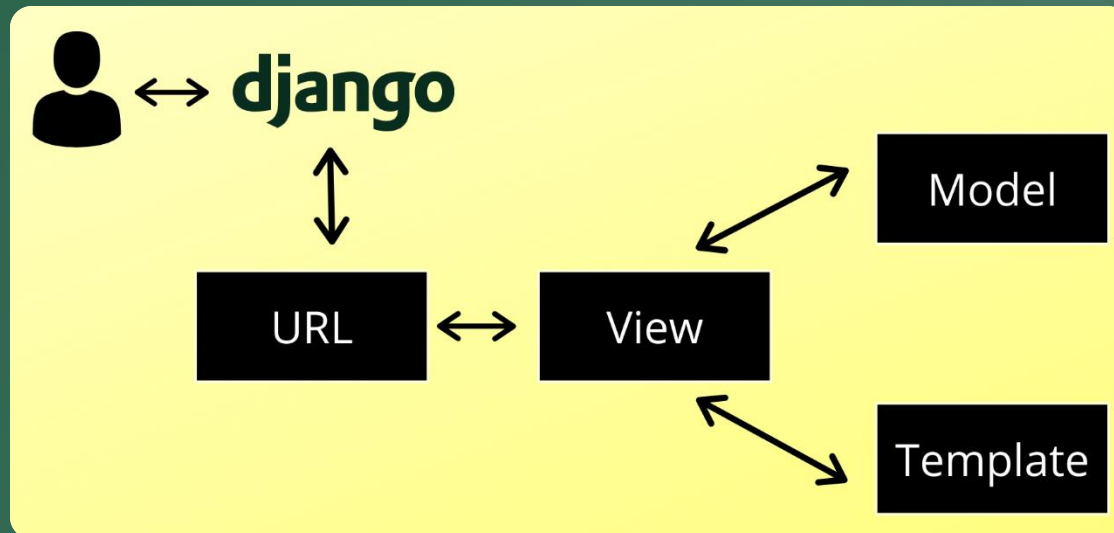


django

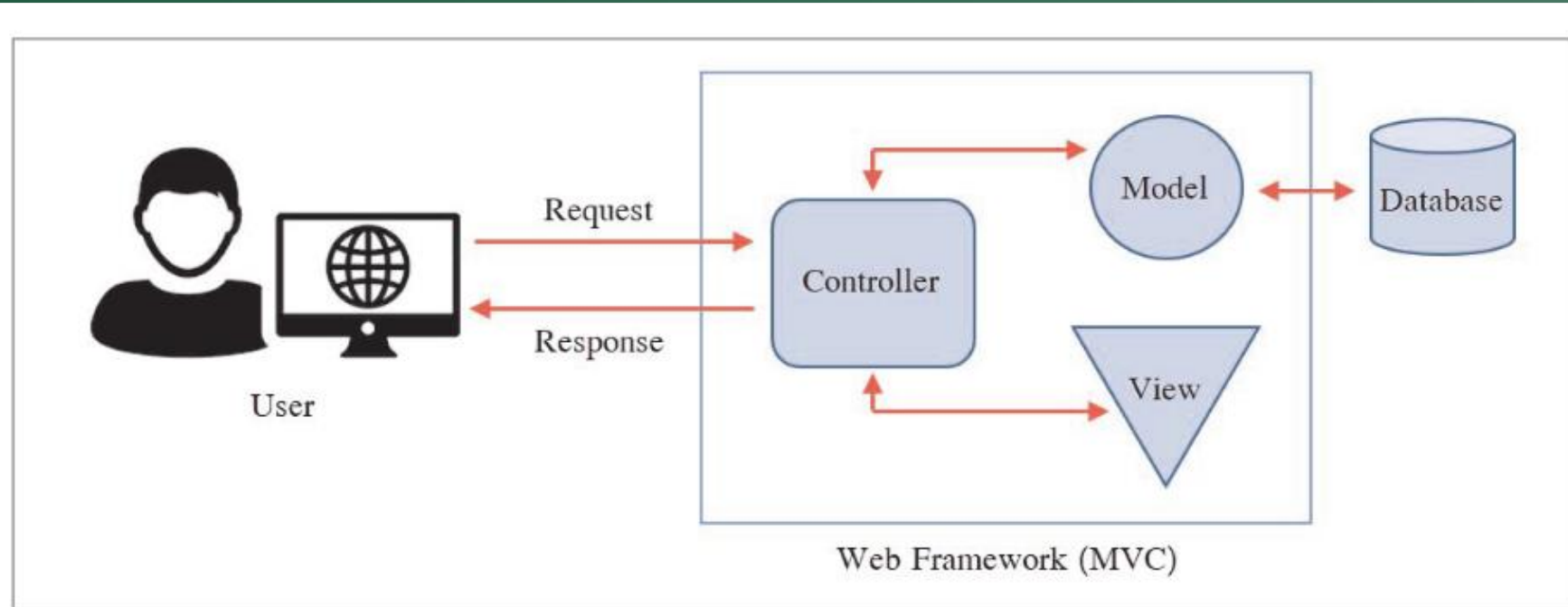
Learn Django framework

What is Django

Django is a high-level Python framework. It is free and open-source, written in Python itself, and follows the **model-view-template (MVT)** architectural pattern. We can use it to develop quality web applications faster and easier

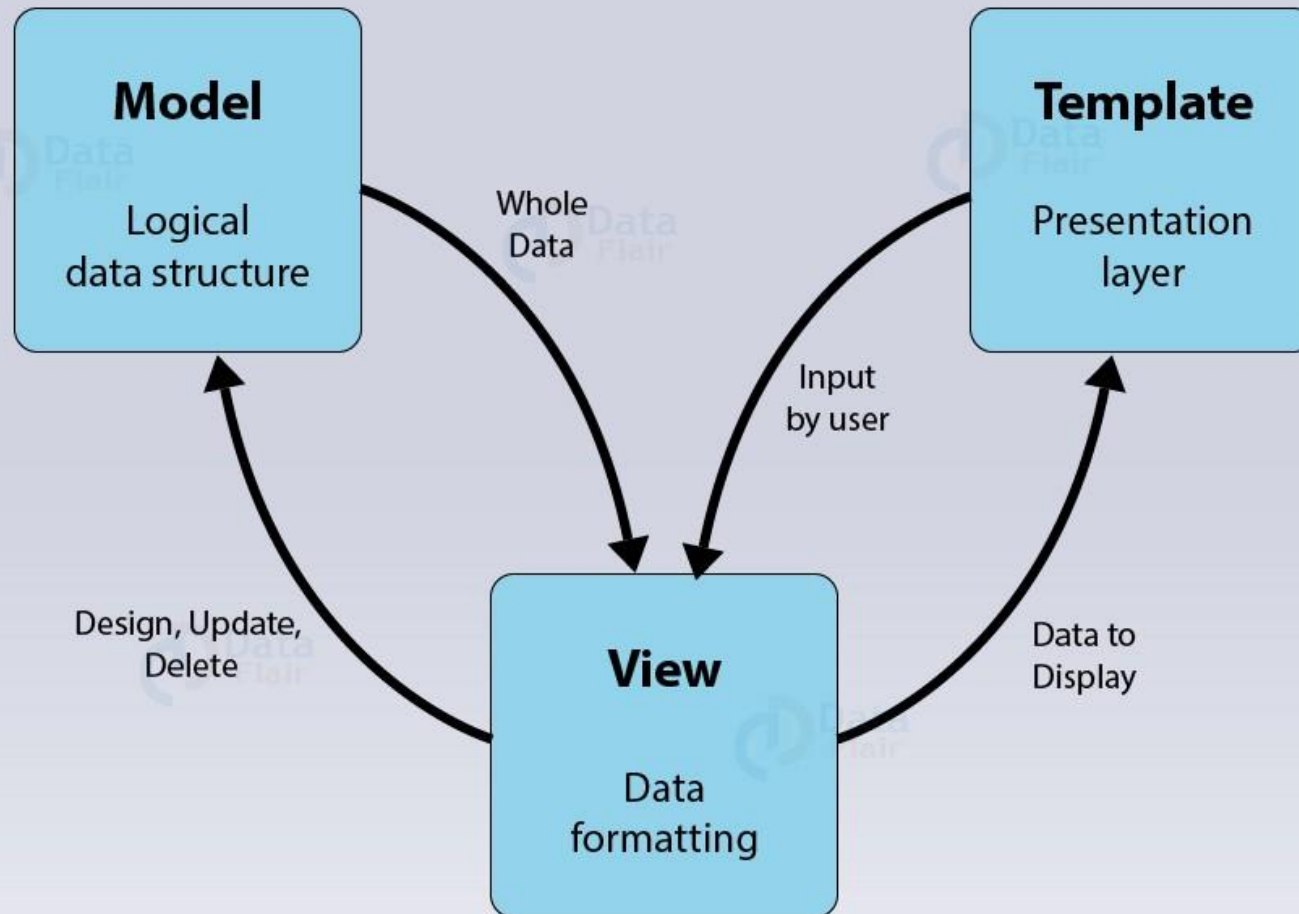


Web Framework (MVC)

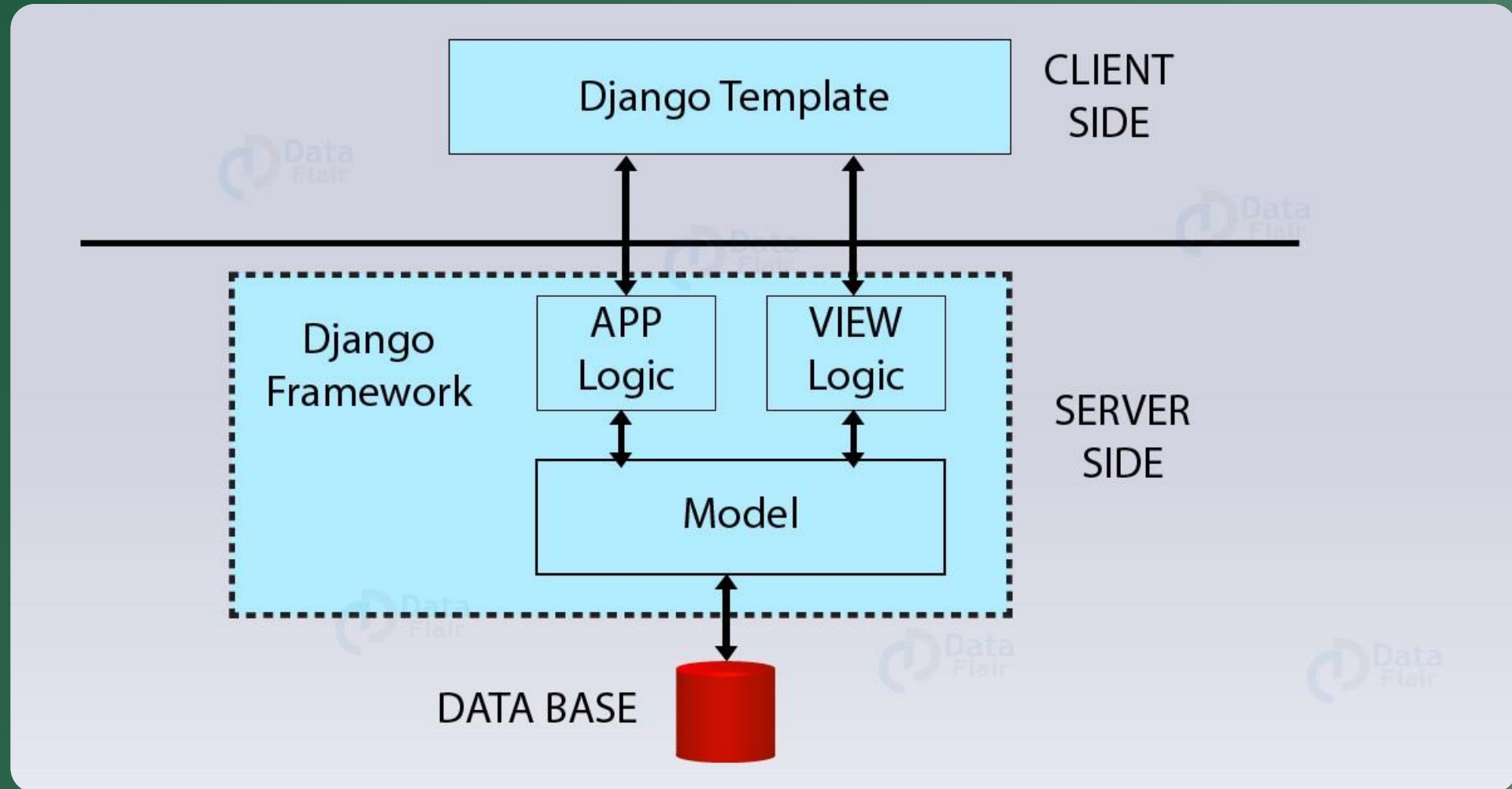


- **Model (M)** สำหรับติดต่อและจัดการกับฐานข้อมูล
- **View (V)** กำหนดโครงร่าง (Layout) และการแสดงผล
- **Controller (C)** กำหนดควบคุมการทำงานและเชื่อมโยงระหว่าง Model และ View

What is Django

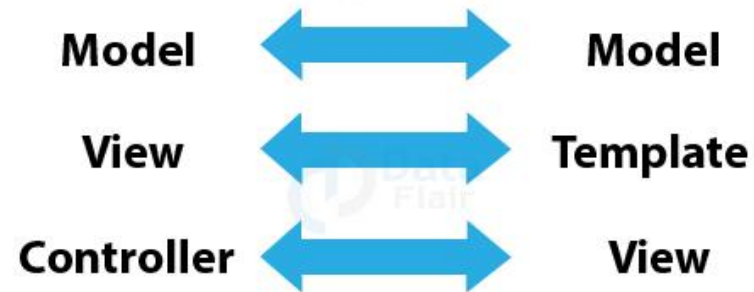


Working of MVT Architecture



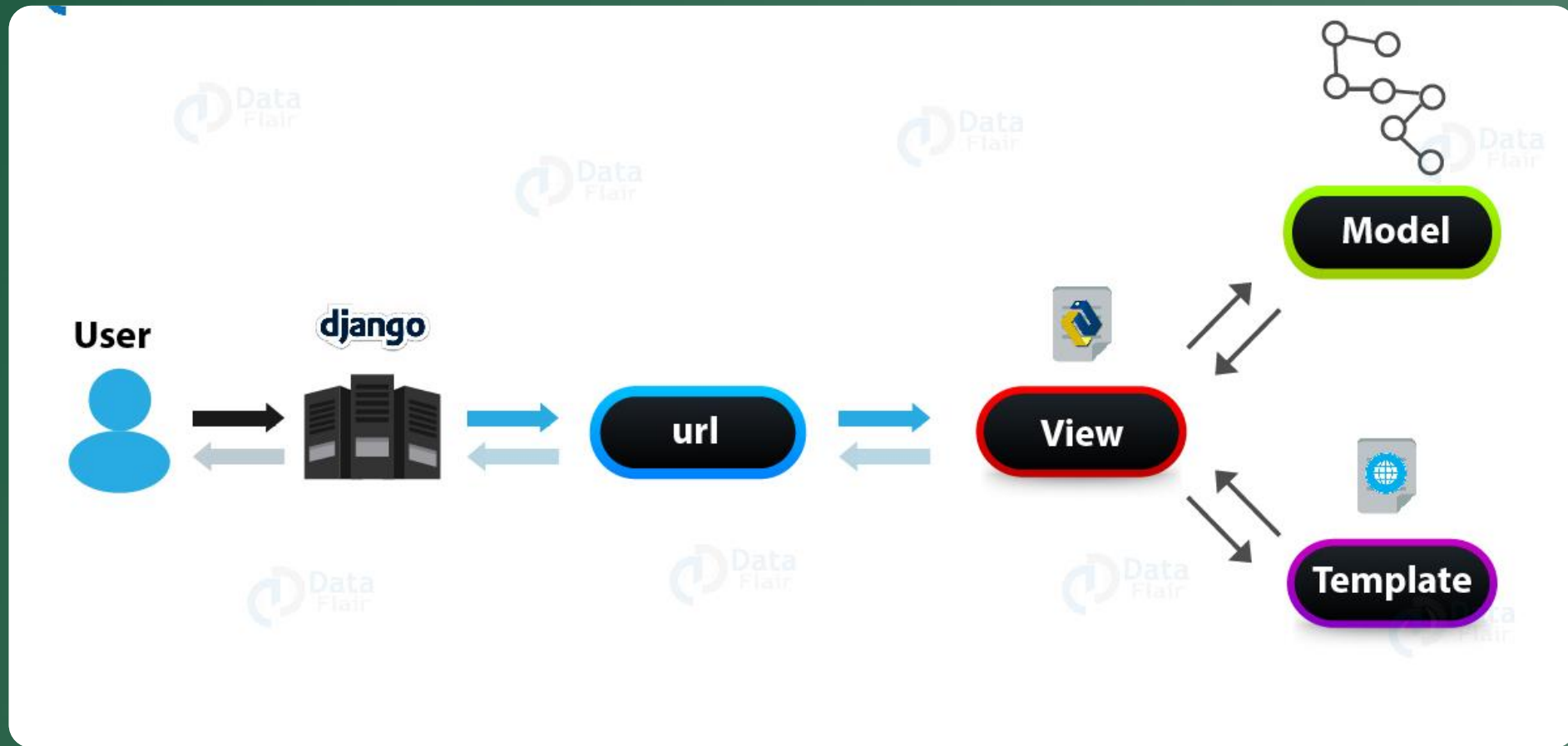
MVT vs MVC in Django

MTV instead of MVC in **django**

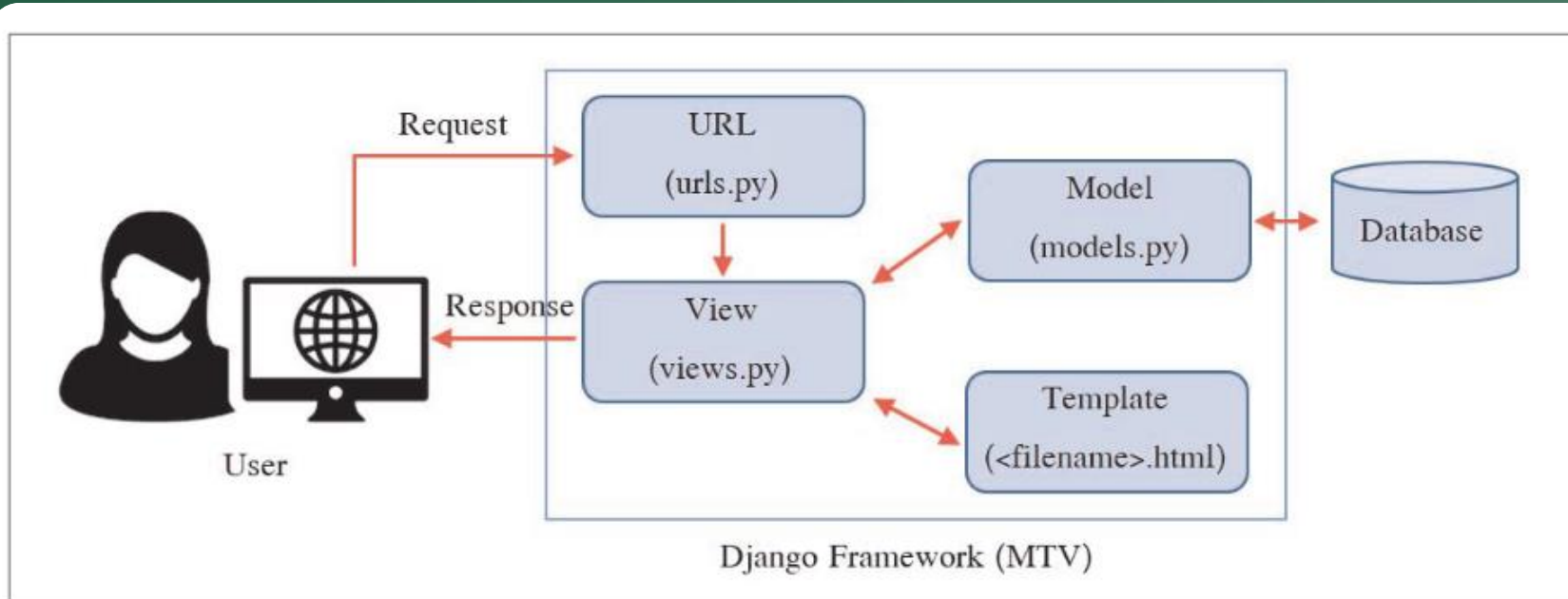


Model	Data description
View	Handles the display of users
Template	Controls the user interaction
Controller	Presentation layer for users

MVT Pattern in Django Structure

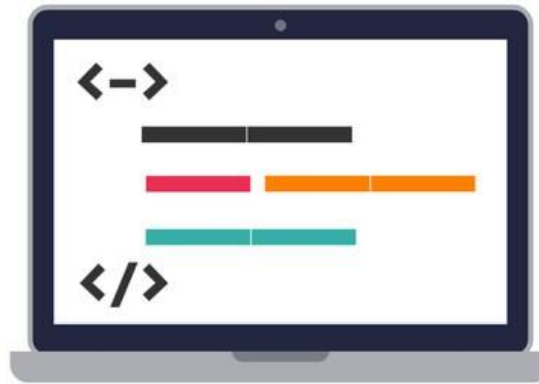


MVT Pattern in Django Structure



- **Model (M)** มีหน้าที่ในการเชื่อมต่อกับฐานข้อมูลเช่นเดียวกับ MVC
- **Template (T)** สำหรับกำหนดโครงร่างของเพจและแสดงผล เช่นเดียวกับ View แบบ MVC
- **View (V)** สำหรับควบคุมกระบวนการทำงานและเชื่อมโยงระหว่าง Model และ Template เช่นเดียวกับ Controller แบบ MVC

django



Installing Django

How to Install Django



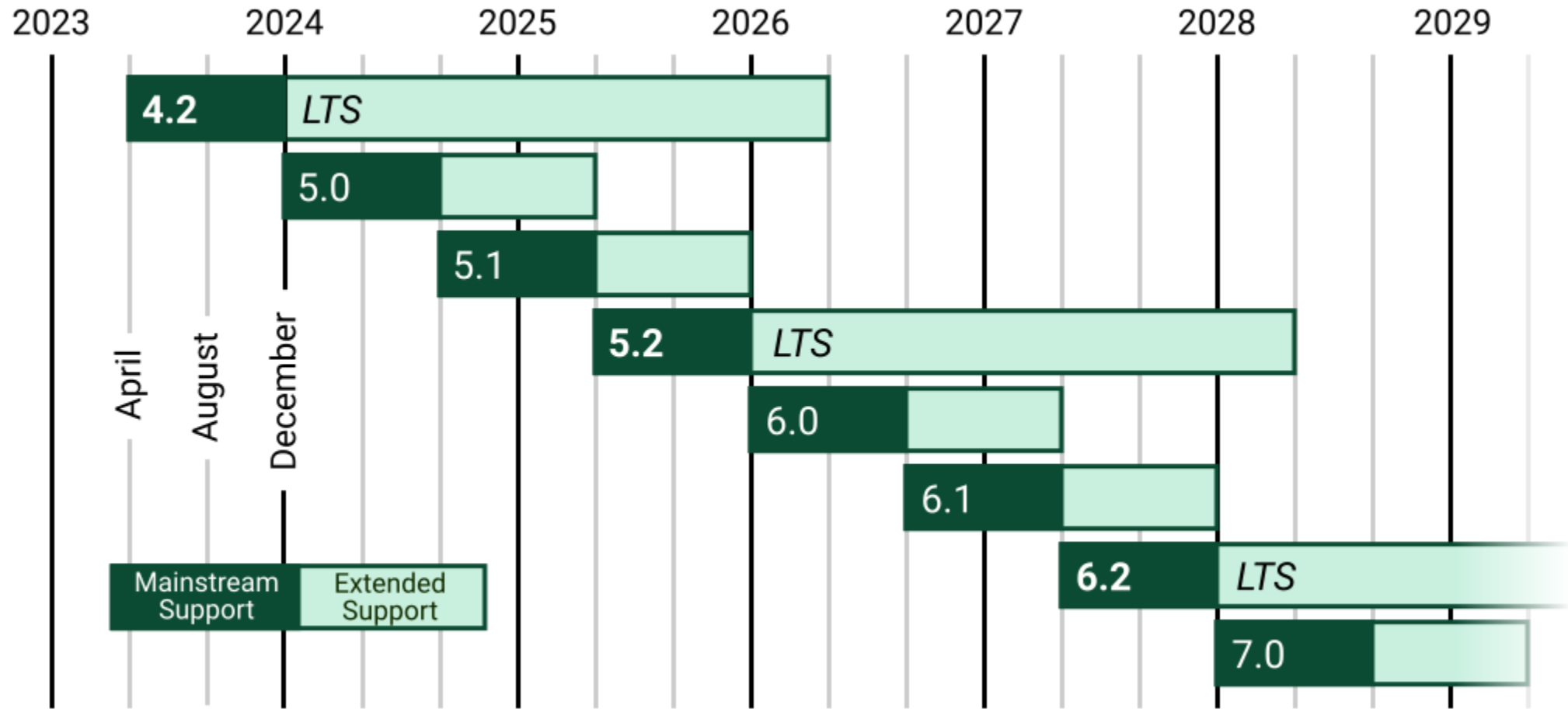
Requirement

- Python 3.13.x
- Python Virtual Environment
- Django 5.1.x

What Python version can I use with Django? 📌

Django version	Python versions
3.2	3.6, 3.7, 3.8, 3.9, 3.10 (added in 3.2.9)
4.0	3.8, 3.9, 3.10
4.1	3.8, 3.9, 3.10, 3.11 (added in 4.1.3)
4.2	3.8, 3.9, 3.10, 3.11, 3.12 (added in 4.2.8)
5.0	3.10, 3.11, 3.12
5.1	3.10, 3.11, 3.12, 3.13 (added in 5.1.3)

Supported Versions



<https://www.djangoproject.com/download/>

Release Series	Latest Release	End of mainstream support ¹	End of extended support ²
5.2 LTS	5.2.5	December 2025	April 2028
5.1	5.1.11	April 2, 2025	December 2025
4.2 LTS	4.2.23	December 4, 2023	April 2026

Future Roadmap

Release Series	Release Date	End of mainstream support ¹	End of extended support ²
7.0	December 2027	August 2028	April 2029
6.2 LTS	April 2027	December 2027	April 2030
6.1	August 2026	April 2027	December 2027
6.0	December 2025	August 2026	April 2027

<https://www.djangoproject.com/download/>

Unsupported previous releases

These release series no longer receive security updates or bug fixes.

Release Series	Latest Release	End of mainstream support ¹	End of extended support ²
5.0	5.0.14	August 7, 2024	April 2, 2025
4.1	4.1.13	April 5, 2023	December 1, 2023
4.0	4.0.10	August 3, 2022	April 1, 2023
3.2 LTS	3.2.25	December 7, 2021	April 1, 2024
3.1	3.1.14	April 6, 2021	December 7, 2021
3.0	3.0.14	August 3, 2020	April 6, 2021
2.2 LTS	2.2.28	December 2, 2019	April 11, 2022
2.1	2.1.15	April 1, 2019	December 2, 2019
2.0	2.0.13	August 1, 2018	April 1, 2019
1.11 LTS ³	1.11.29	December 2, 2017	April 1, 2020
1.10	1.10.8	April 4, 2017	December 2, 2017
1.9	1.9.13	August 1, 2016	April 4, 2017
1.8 LTS	1.8.19	December 1, 2015	April 1, 2018
1.7	1.7.11	April 1, 2015	December 1, 2015
1.6	1.6.11	September 2, 2014	April 1, 2015
1.5	1.5.12	November 6, 2013	September 2, 2014
1.4 LTS	1.4.22	February 26, 2013	October 1, 2015
1.3	1.3.7	March 23, 2012	February 26, 2013



Installing Django

Installing Django 5.2.x

django



`pip install Django==5.2.5`



`pip3 install Django==5.2.5`

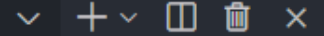
PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

1: cmd



```
(env) G:\TrainingWorkshop\PythonDjangoCoreWisdomClass\BasicDjango\demodjango>pip install Django==4.0.5
```

```
Collecting Django==4.0.5
```

```
Using cached Django-4.0.5-py3-none-any.whl (8.0 MB)
```

```
Collecting sqlparse>=0.2.2
```

```
Using cached sqlparse-0.4.2-py3-none-any.whl (42 kB)
```

```
Collecting tzdata
```

```
Using cached tzdata-2022.1-py2.py3-none-any.whl (339 kB)
```

```
Collecting asgiref<4,>=3.4.1
```

```
Using cached asgiref-3.5.2-py3-none-any.whl (22 kB)
```

```
Installing collected packages: tzdata, sqlparse, asgiref, Django
```

```
Successfully installed Django-4.0.5 asgiref-3.5.2 sqlparse-0.4.2 tzdata-2022.1
```

Name	Date modified	Type	Size
_distutils_hack	6/13/2022 11:46 PM	File folder	
asgiref	6/13/2022 11:50 PM	File folder	
asgiref-3.5.2.dist-info	6/13/2022 11:50 PM	File folder	
django	6/13/2022 11:50 PM	File folder	
Django-4.0.5.dist-info	6/13/2022 11:50 PM	File folder	
pip	6/13/2022 11:51 PM	File folder	
pip-22.1.2.dist-info	6/13/2022 11:51 PM	File folder	
pkg_resources	6/13/2022 11:46 PM	File folder	
setuptools	6/13/2022 11:46 PM	File folder	
setuptools-57.4.0.dist-info	6/13/2022 11:46 PM	File folder	
sqlparse	6/13/2022 11:50 PM	File folder	
sqlparse-0.4.2.dist-info	6/13/2022 11:50 PM	File folder	
tzdata	6/13/2022 11:50 PM	File folder	
tzdata-2022.1.dist-info	6/13/2022 11:50 PM	File folder	
distutils-precedence.pth	6/13/2022 11:46 PM	PTH File	1 KB

Test Django

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
(env) G:\TrainingWorkshop\PythonDjangoCoreWisdomClass\BasicDjango\demodjango>python
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import django
>>> print(django.get_version())
4.0.5
>>> |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
(env) G:\TrainingWorkshop\PythonDjangoCoreWisdomClass\BasicDjango\demodjango>python -m django --version
4.0.5
```

1: cn

Create Project in Django

The Django logo, which consists of the word "django" in a white, lowercase, sans-serif font, centered within a dark green rounded rectangle with a white border.

Create project

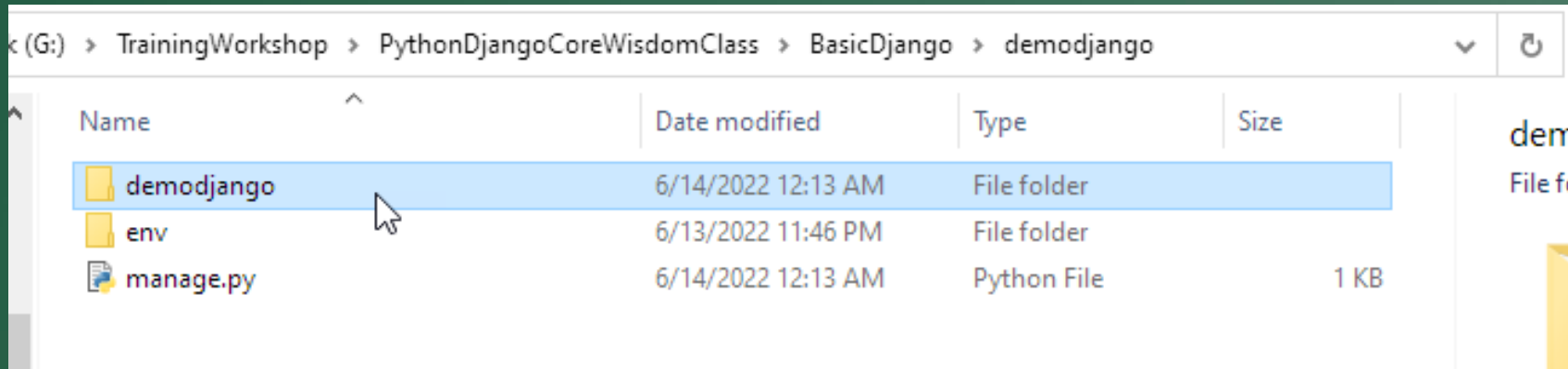
Name	Date modified	Type	Size
 activate	6/13/2022 11:46 PM	File	3 KB
 activate.bat	6/13/2022 11:46 PM	Windows Batch File	2 KB
 Activate.ps1	6/13/2022 11:46 PM	Windows PowerS...	20 KB
 deactivate.bat	6/13/2022 11:46 PM	Windows Batch File	1 KB
 django-admin.exe	6/13/2022 11:50 PM	Application	104 KB
 pip.exe	6/13/2022 11:51 PM	Application	104 KB
 pip3.10.exe	6/13/2022 11:51 PM	Application	104 KB
 pip3.exe	6/13/2022 11:51 PM	Application	104 KB
 python.exe	6/13/2022 11:46 PM	Application	238 KB
 pythonw.exe	6/13/2022 11:46 PM	Application	228 KB
 sqlformat.exe	6/13/2022 11:50 PM	Application	104 KB

Create new project

```
django-admin startproject <sitename>
```

Example

```
django-admin startproject firstdjango .
```



Create new project

ningWorkshop > PythonDjangoCoreWisdomClass > BasicDjango > demodjango > demodjango

Name	Date modified	Type	Size	
 <code>__init__.py</code>	6/14/2022 12:13 AM	Python File	0 KB	
 <code>asgi.py</code>	6/14/2022 12:13 AM	Python File	1 KB	
 <code>settings.py</code>	6/14/2022 12:13 AM	Python File	4 KB	
 <code>urls.py</code>	6/14/2022 12:13 AM	Python File	1 KB	
 <code>wsgi.py</code>	6/14/2022 12:13 AM	Python File	1 KB	

5 items

Run Project

Run Django Project

```
python manage.py runserver
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

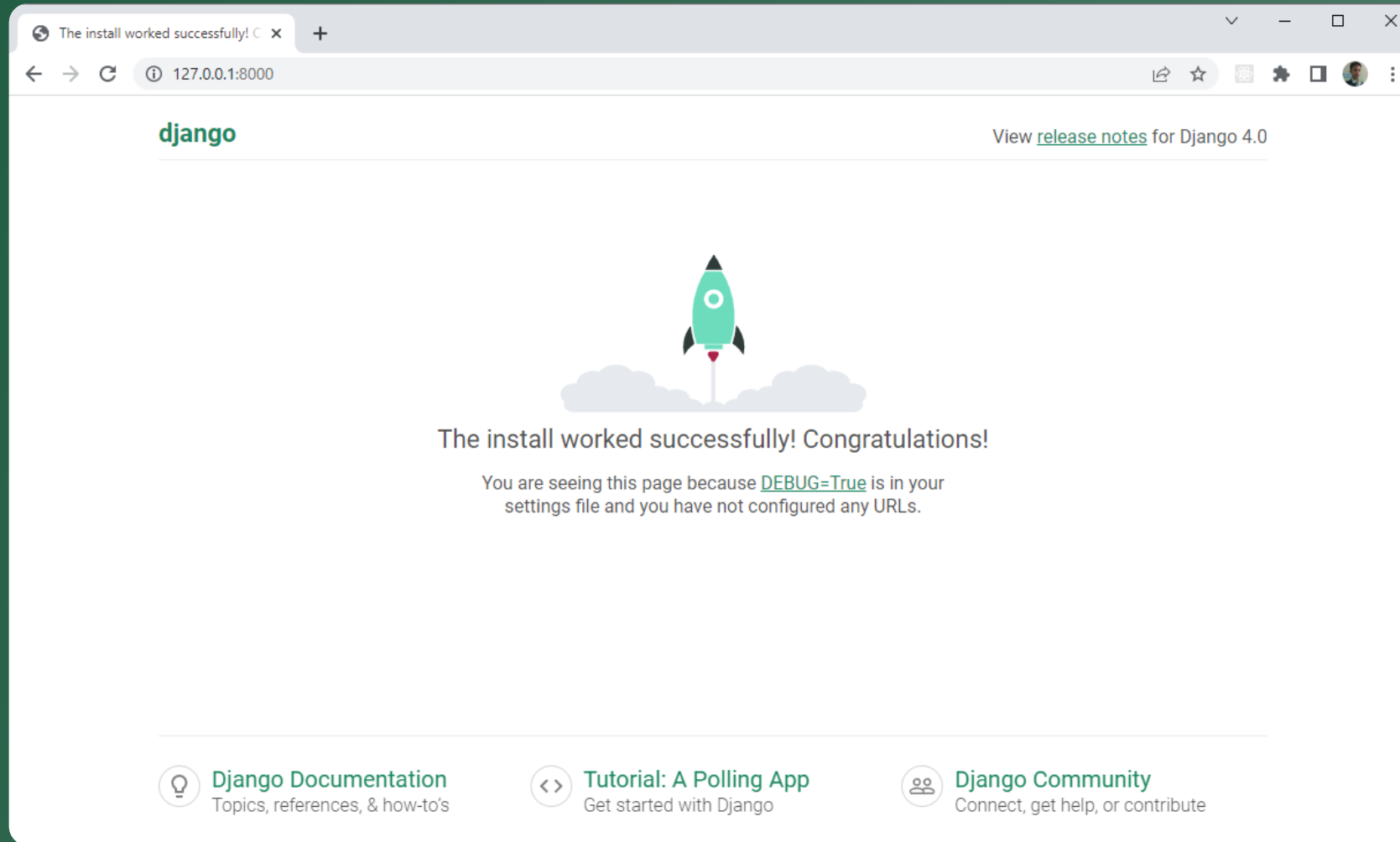
(env) G:\TrainingWorkshop\PythonDjangoCoreWisdomClass\BasicDjango\demodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for
sessions.
Run 'python manage.py migrate' to apply them.
June 14, 2022 - 00:16:58
Django version 4.0.5, using settings 'demodjango.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
|
```

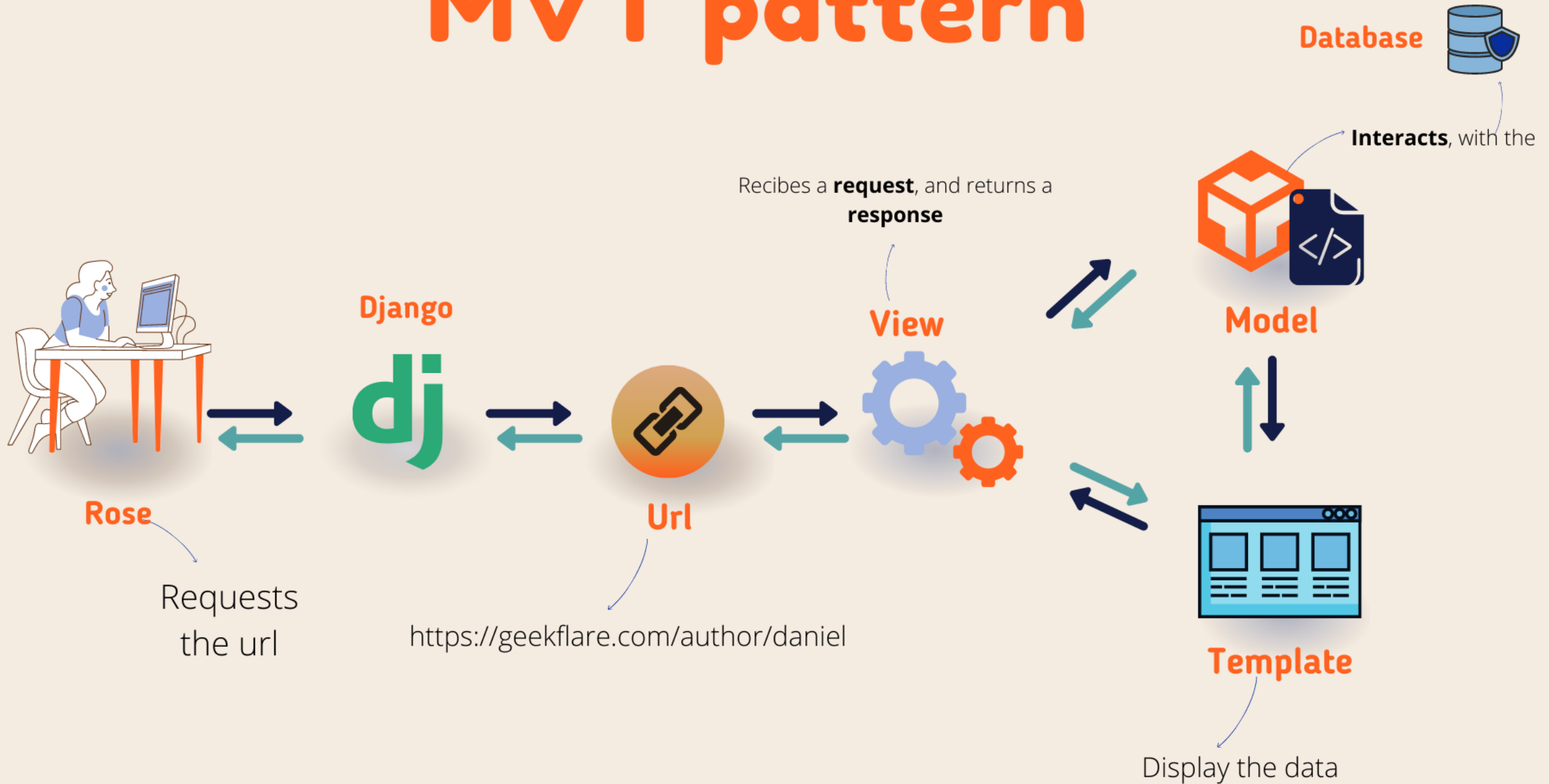
<http://127.0.0.1:8000/>

Run Django Project at http://127.0.0.1:8000



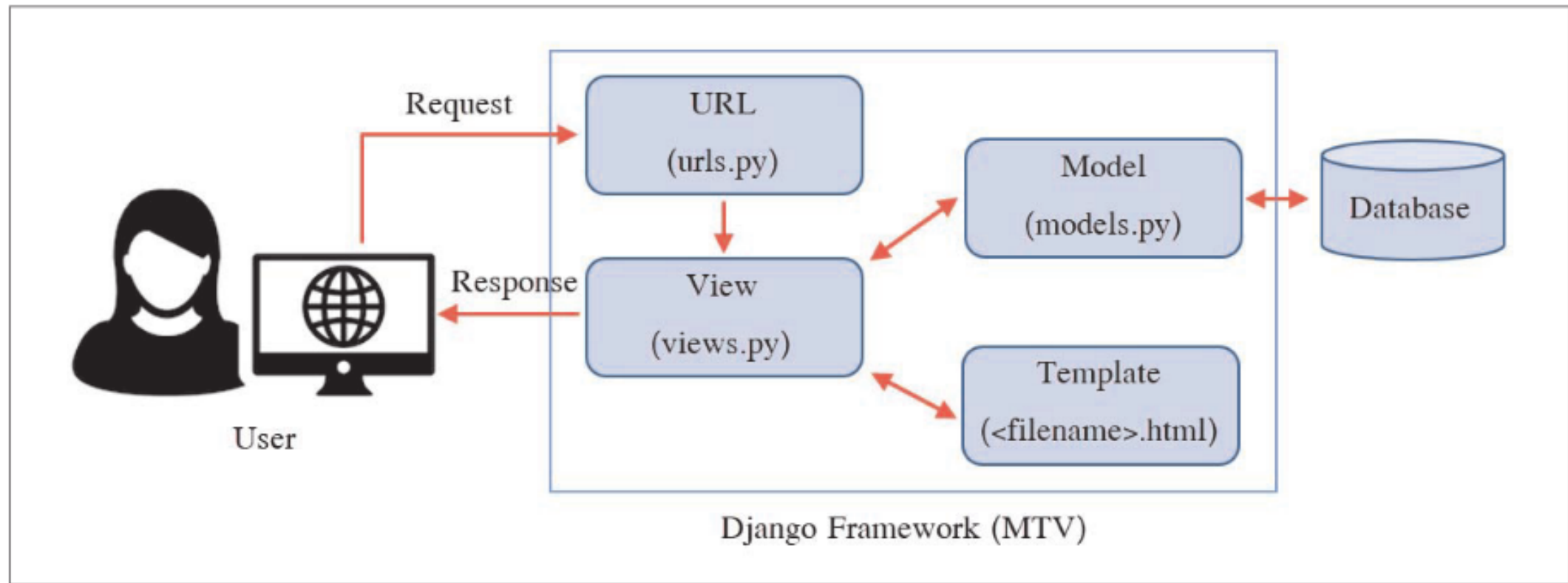
Django URL and View

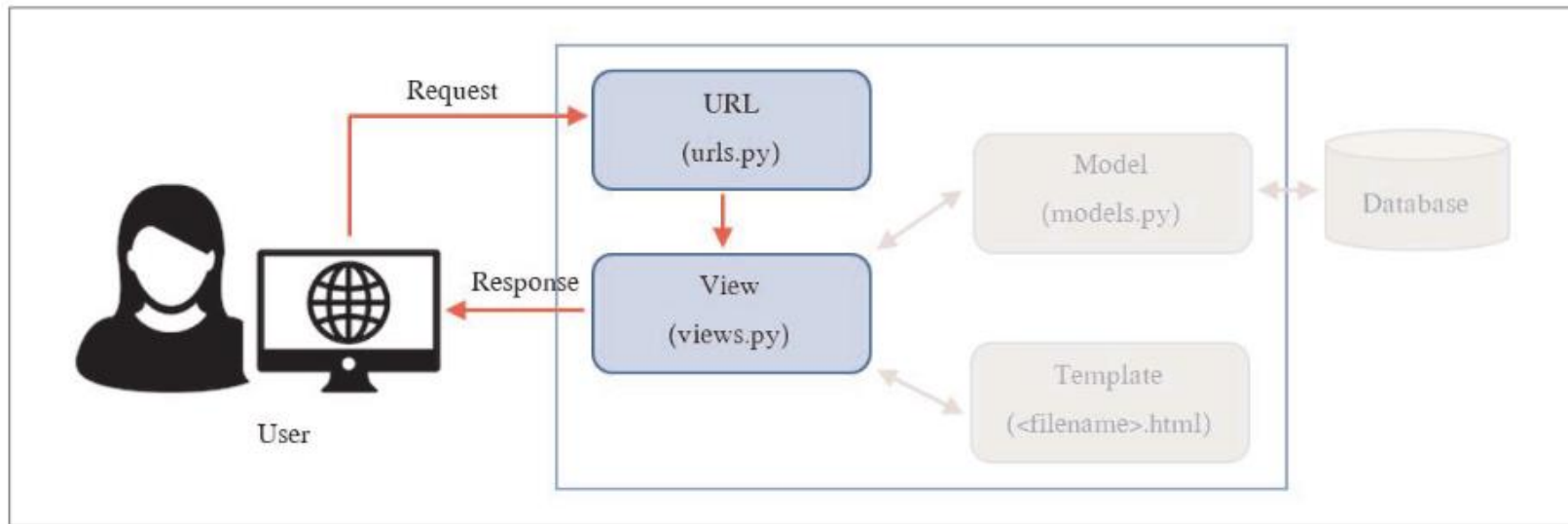
MVT pattern



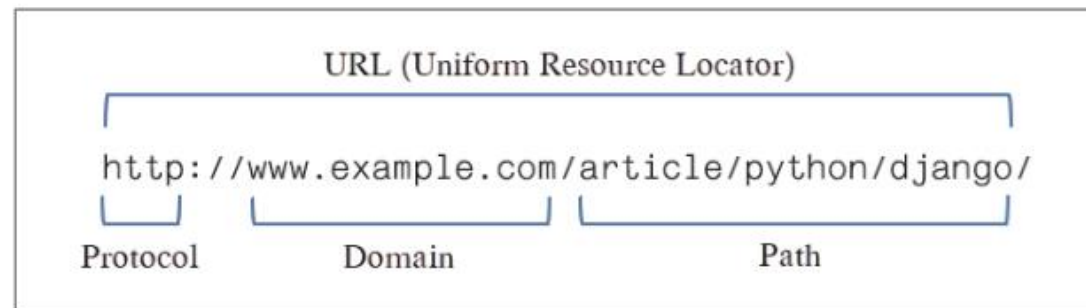
หลักการทำงานในเบื้องต้นของ URL และ View

ในบทที่ผ่านมา เราได้ทราบไปแล้วว่า ลักษณะโครงสร้างของ Django Framework จะเป็นแบบ Model Template View (MTV) โดยมีองค์ประกอบส่วนต่างๆ ดังภาพ





URL ที่กล่าวมานั้น เป็นส่วนของ Django ทางฝั่งเซิร์ฟเวอร์ แต่ในฝั่งของเบราว์เซอร์ ตำแหน่งของเว็บเพจที่เรากำหนดหรือที่ปรากฏบนเว็บเบราว์เซอร์ ก็จะเรียกว่า URL เช่นกัน ซึ่งเราสามารถแบ่งองค์ประกอบแยกย่อยเป็นดังภาพ



Web Browser

`http://127.0.0.0:8000/hello/`

Hello World

1

URL (urls.py)

`path('hello/', views.say_hello)`

2

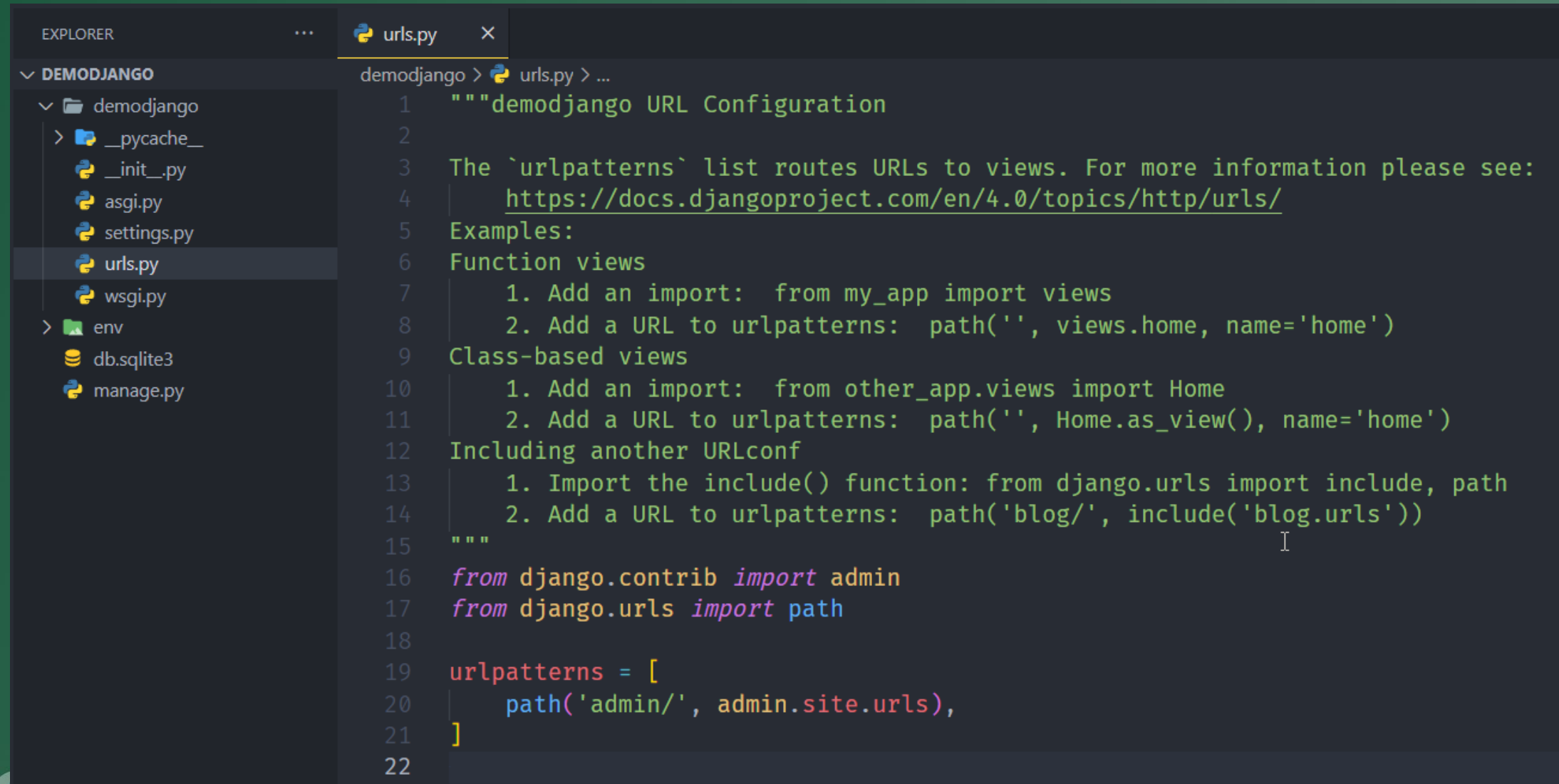
View (views.py)

`def say_hello(request):`

`return HttpResponse('Hello World')`

3

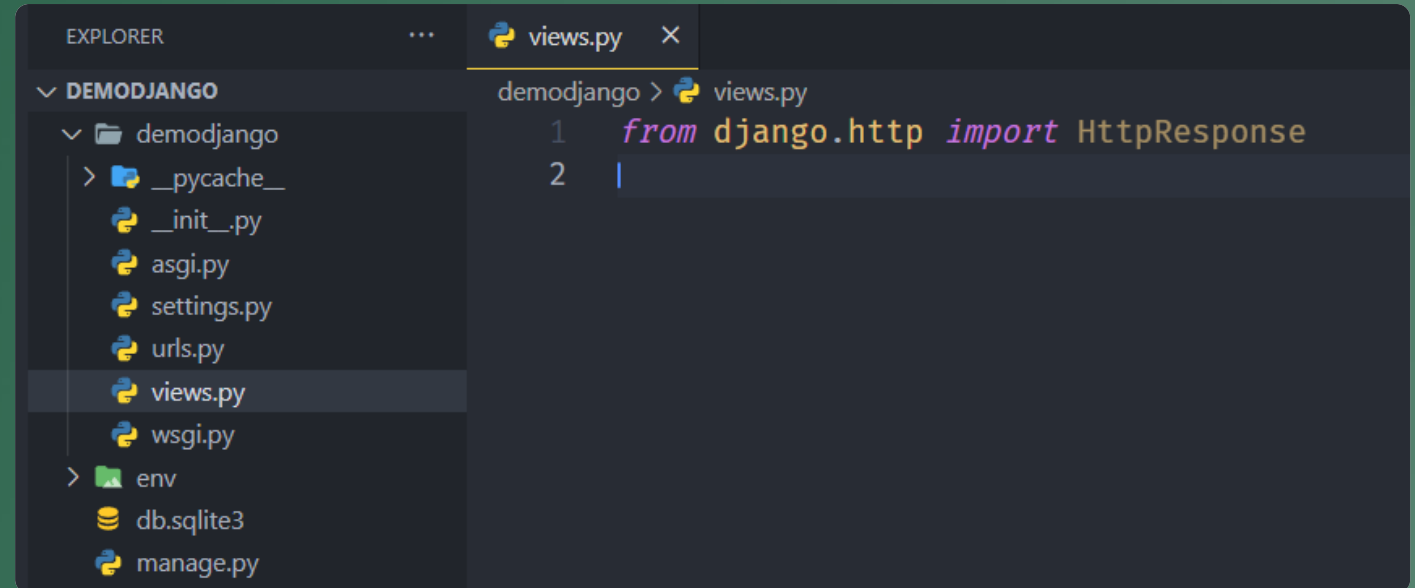
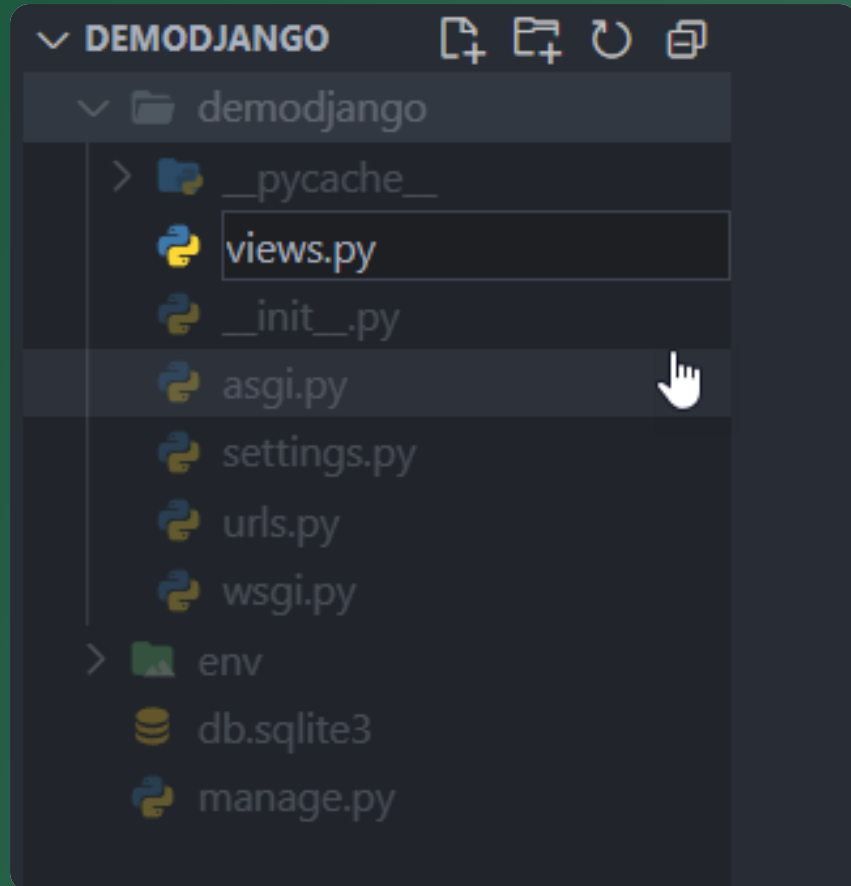
ลักษณะของไฟล์ urls.py



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'DEMODJANGO' with a subdirectory 'demodjango' containing files like '__pycache__', '__init__.py', 'asgi.py', 'settings.py', 'urls.py', and 'wsgi.py'. The 'urls.py' file is selected. The code editor shows the content of 'urls.py' with line numbers 1 through 22. The code is a Django URL configuration file.

```
1  """demodjango URL Configuration
2
3  The `urlpatterns` list routes URLs to views. For more information please see:
4  |   https://docs.djangoproject.com/en/4.0/topics/http/urls/
5  |   Examples:
6  |   Function views
7  |       1. Add an import:  from my_app import views
8  |       2. Add a URL to urlpatterns:  path('', views.home, name='home')
9  |   Class-based views
10 |       1. Add an import:  from other_app.views import Home
11 |       2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
12 |   Including another URLconf
13 |       1. Import the include() function: from django.urls import include, path
14 |       2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
15 |   """
16  from django.contrib import admin
17  from django.urls import path
18
19  urlpatterns = [
20      path('admin/', admin.site.urls),
21  ]
22
```

ลักษณะของไฟล์ views.py



แนวทางการใช้ urls.py ร่วมกับ views.py

The image shows a code editor with two files: `urls.py` and `views.py`. In `urls.py`, the `views` module is imported, and the `index` function from `views` is mapped to the root path. In `views.py`, the `index` function returns a `HttpResponse` with the text "Welcome to my home page". A web browser at `127.0.0.1:8000` displays this text. Yellow arrows show the import and function call, while red arrows show the final output in the browser.

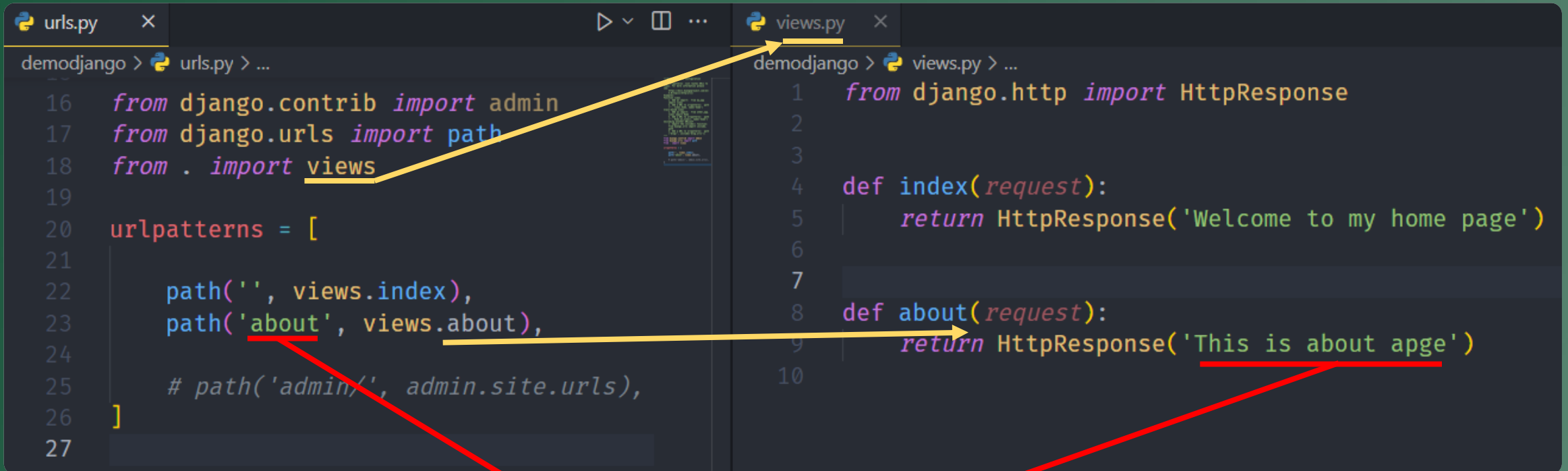
```
demodjango > urls.py > ...
16 from django.contrib import admin
17 from django.urls import path
18 from . import views
19
20 urlpatterns = [
21     path('', views.index),
22     # path('admin/', admin.site.urls),
23 ]
24
25
26
```

```
demodjango > views.py > ...
1 from django.http import HttpResponse
2
3
4 def index(request):
5     return HttpResponse('Welcome to my home page')
6
```

127.0.0.1:8000

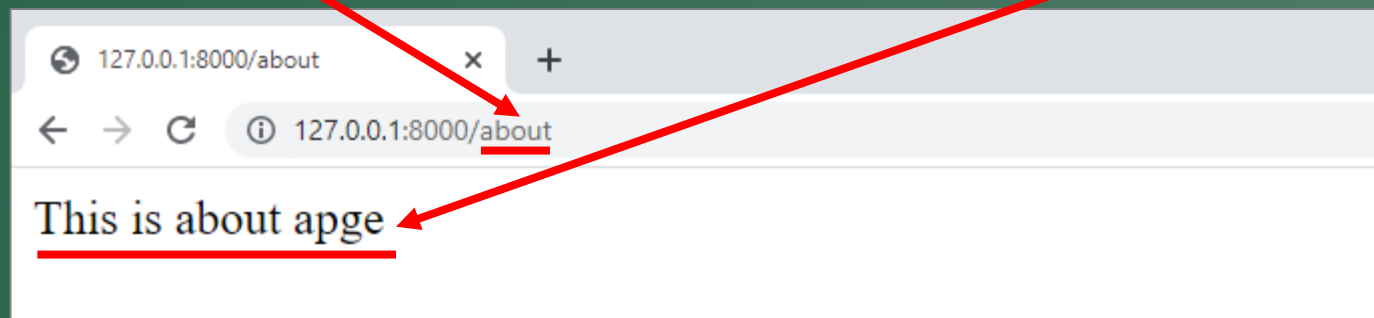
Welcome to my home page

แนวทางการใช้ urls.py ร่วมกับ views.py



```
demodjango > urls.py > ...
16 from django.contrib import admin
17 from django.urls import path
18 from . import views
19
20 urlpatterns = [
21
22     path('', views.index),
23     path('about', views.about),
24
25     # path('admin/', admin.site.urls),
26 ]
27
```

```
demodjango > views.py > ...
1 from django.http import HttpResponse
2
3
4 def index(request):
5     return HttpResponse('Welcome to my home page')
6
7
8 def about(request):
9     return HttpResponse('This is about apge')
10
```



การกำหนด Path แบบมี Parameter

<converter:parameter_name>

int	ข้อมูลแบบเลขจำนวนเต็มบวกเท่านั้น (ติดลบไม่ได้)
str	ข้อมูลแบบสตริง ที่ไม่มีเครื่องหมาย / รวมอยู่ด้วย หากสตริงนั้นมีช่องว่าง จะถูกแปลงเป็น %20 เช่น django framework => django%20framework
path	ข้อมูลแบบสตริง ที่อาจมีเครื่องหมาย / รวมอยู่ด้วย ส่วนใหญ่จะใช้กับการกำหนดค่าที่เป็น พาร หรือ URL
slug	ข้อมูลแบบสตริงที่คั่นระหว่างคำด้วย hyphen (-) และ/หรือ underscore (_) เช่น slugs-are-text-string-with-hyphens-and-underscors developing_web_application_with_django_framework model-template-view_or_mtv

ตัวอย่างการกำหนด Path แบบมี Parameter

```
#urls.py  
path('course/django/chapter/<int:chap>/', views.dj_chap_x)
```

#ตัวอย่าง URL ซึ่งตรงกับพารามิเตอร์ (กำหนดให้แก่เบราว์เซอร์)

<http://www.example.com/course/django/chapter/1/>
<http://www.example.com/course/django/chapter/5/>

```
#urls.py  
path('search/<str:keyword>/<int:page>/', views.search)
```

#ตัวอย่าง URL ซึ่งตรงกับพารามิเตอร์

<http://www.example.com/search/django/1/>
<http://www.example.com/search/python data science/10/>

ตัวอย่างการกำหนด Path แบบมี Parameter

```
#urls.py  
path(news/id=<int:id>/lang=<str:lang>/', views.news)
```

#ตัวอย่าง URL ซึ่งตรงกับพารามิเตอร์
<http://www.example.com/news/id=1234/lang=th/>

```
#urls.py  
path('redirect/<path:target>/', views.redirect)
```

#ตัวอย่าง URL ซึ่งตรงกับพารามิเตอร์
<http://www.example.com/redirect/http://www.developerthai.com/>

```
#urls.py  
path('article/<int:id>/<slug:title>/', views.show_article)
```

#ตัวอย่าง URL ซึ่งตรงกับพารามิเตอร์
<http://www.example.com/article/1234/python-how-to-program/>
http://www.example.com/article/555/django-4-for_beginner_part-2/

การอ่านค่าจาก Parameter

urls.py

```
path('search/<str:keyword>/<int:page>/', views.search)
```

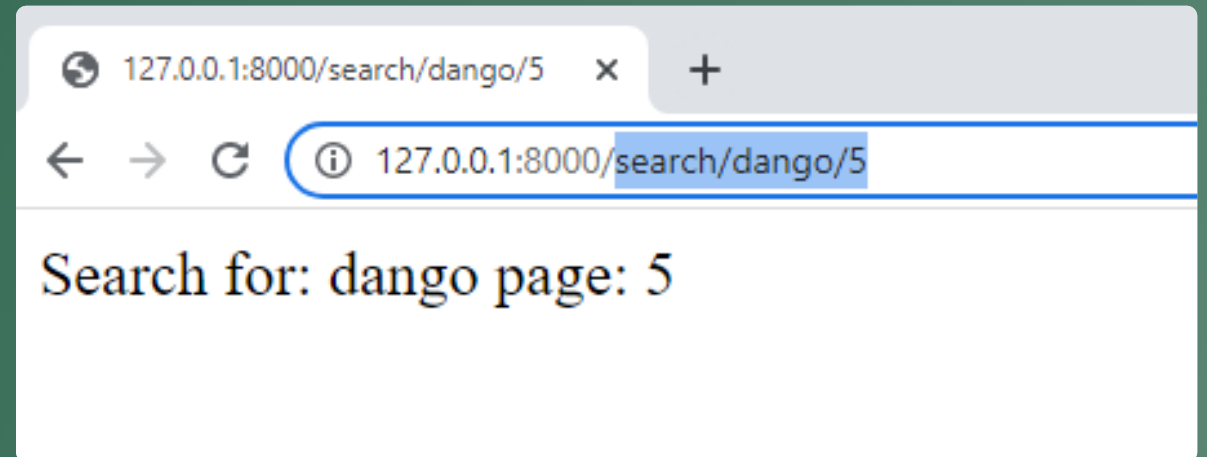
views.py

```
def search(request, keyword, page):  
    return HttpResponse(f'Search for: {keyword} page: {page}')
```

ตัวอย่างการอ่านค่าจาก Parameter

```
urls.py
demodjango > urls.py > ...
16 from django.contrib import admin
17 from django.urls import path
18 from . import views
19
20 urlpatterns = [
21
22     path('', views.index),
23     path('about', views.about),
24
25     path('search/<str:keyword>/<int:page>', views.search)
26
27     # path('admin/', admin.site.urls),
28 ]
29
```

```
views.py
demodjango > views.py > ...
8 def about(request):
9     return HttpResponse('This is about apge')
10
11
12 def search(request, keyword, page):
13     return HttpResponse(f'Search for: {keyword} page: {page}')
14
```



การกำหนด Path และ Query String

<http://www.example.com/search/?q=django&page=3>

parameter_name1=value1¶meter_name2=value2&...

<http://www.example.com/maps/?type=satellite&lat=13.141516&lon=100.123456&zoom=17>

พารามิเตอร์ type มีค่าเป็น satellite

พารามิเตอร์ lat มีค่าเป็น 13.141516

พารามิเตอร์ lon มีค่าเป็น 100.123456

พารามิเตอร์ zoom มีค่าเป็น 17

การอ่าน Path แบบ Query String

request.GET.get('ชื่อพารามิเตอร์', ค่าดีฟอลต์)

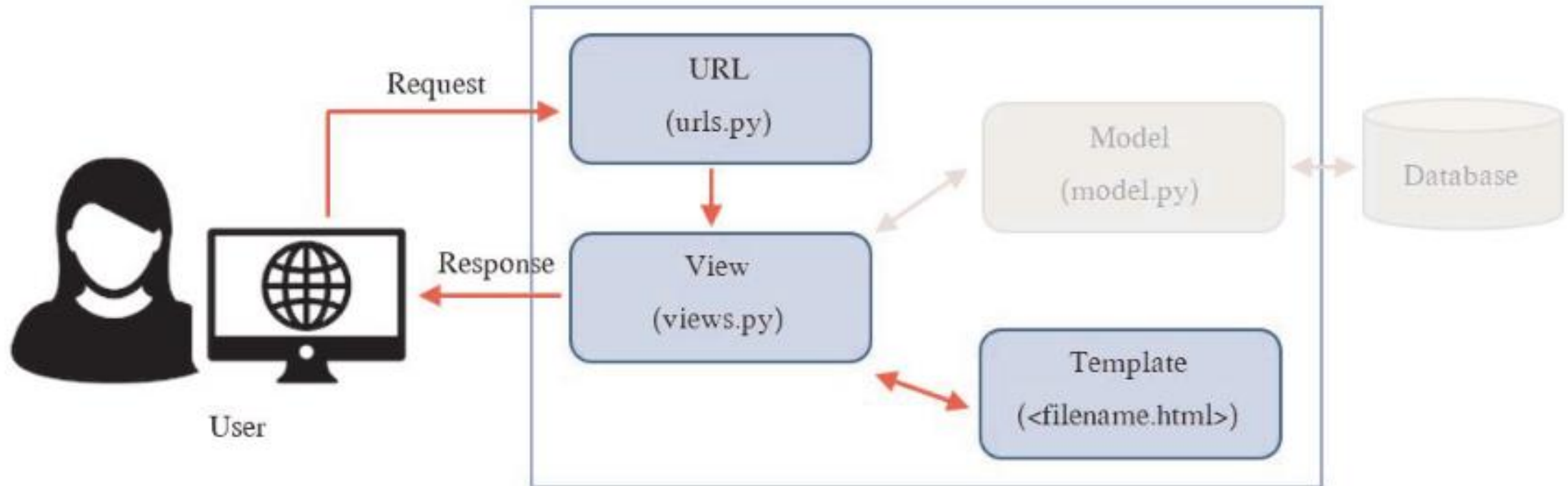
```
urls.py
demodjango > urls.py > ...
15
16 from django.contrib import admin
17 from django.urls import path
18 from . import views
19
20 urlpatterns = [
21     # การกำหนด path
22     path('', views.index),
23     path('about', views.about),
24
25     # path แบบมี parameter
26     path('search/<str:keyword>/<int:page>', views.search),
27
28     # path แบบมี query string
29     path('maps', views.maps)
30
31     # path('admin/', admin.site.urls),
32
33 ]
```

```
views.py
demodjango > views.py > ...
15
16 def maps(request):
17     type = request.GET.get('type', 'hybrid')
18     lat = request.GET.get('lat', '13.7245601')
19     lon = request.GET.get('lon', '100.4930241')
20     zoom = request.GET.get('z', 11)
21
22     return HttpResponse(f"Map type: {type} <br> \
23         Location: {lat},{lon}<br> \
24         Zoom: {zoom}")
25
```

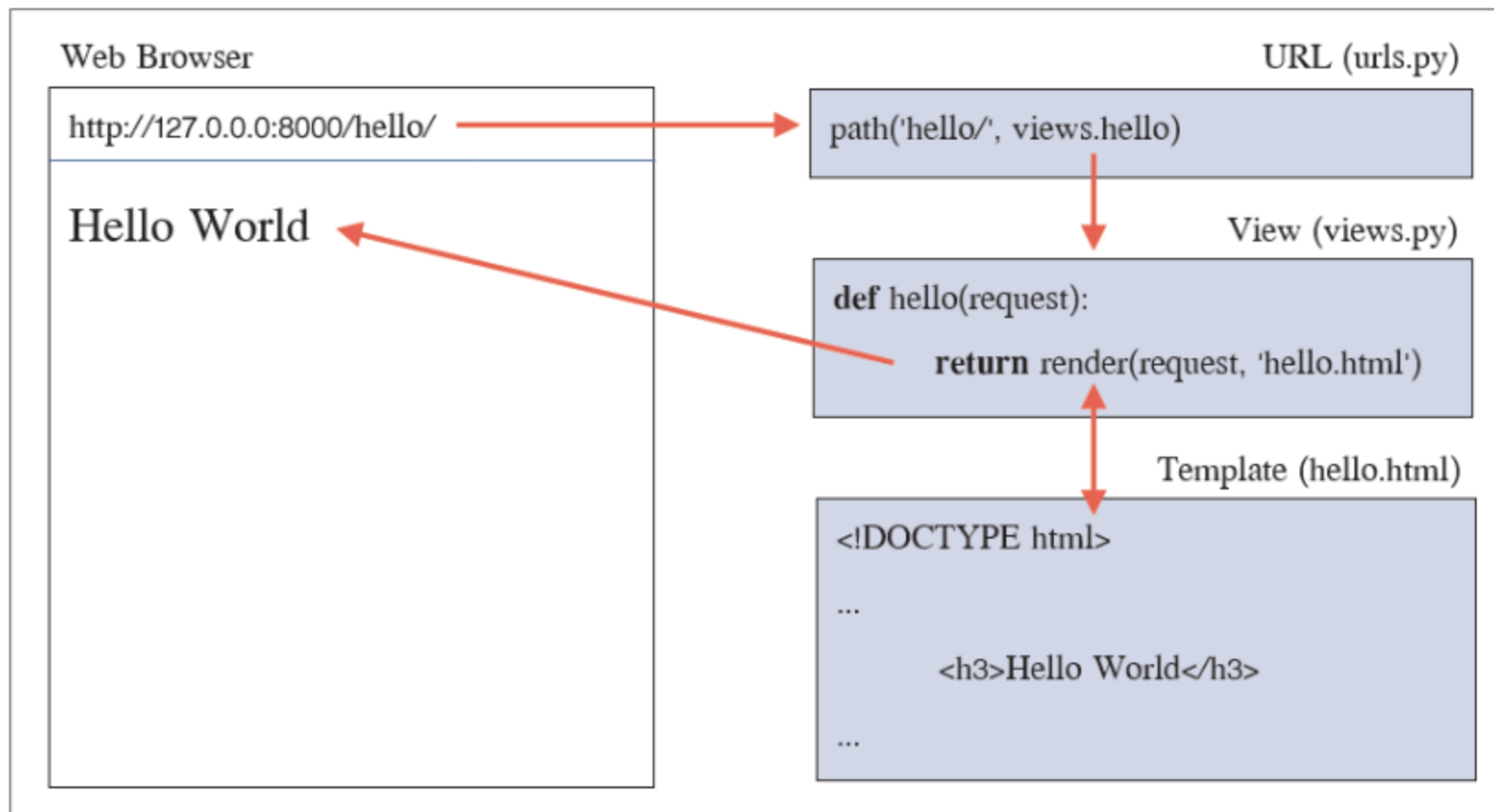
127.0.0.1:8000/maps?type=roadmap & +
127.0.0.1:8000/maps?type=roadmap&lat=13.333333&lon=100.00001&z=15

Map type: roadmap
Location: 13.333333,100.00001
Zoom: 15

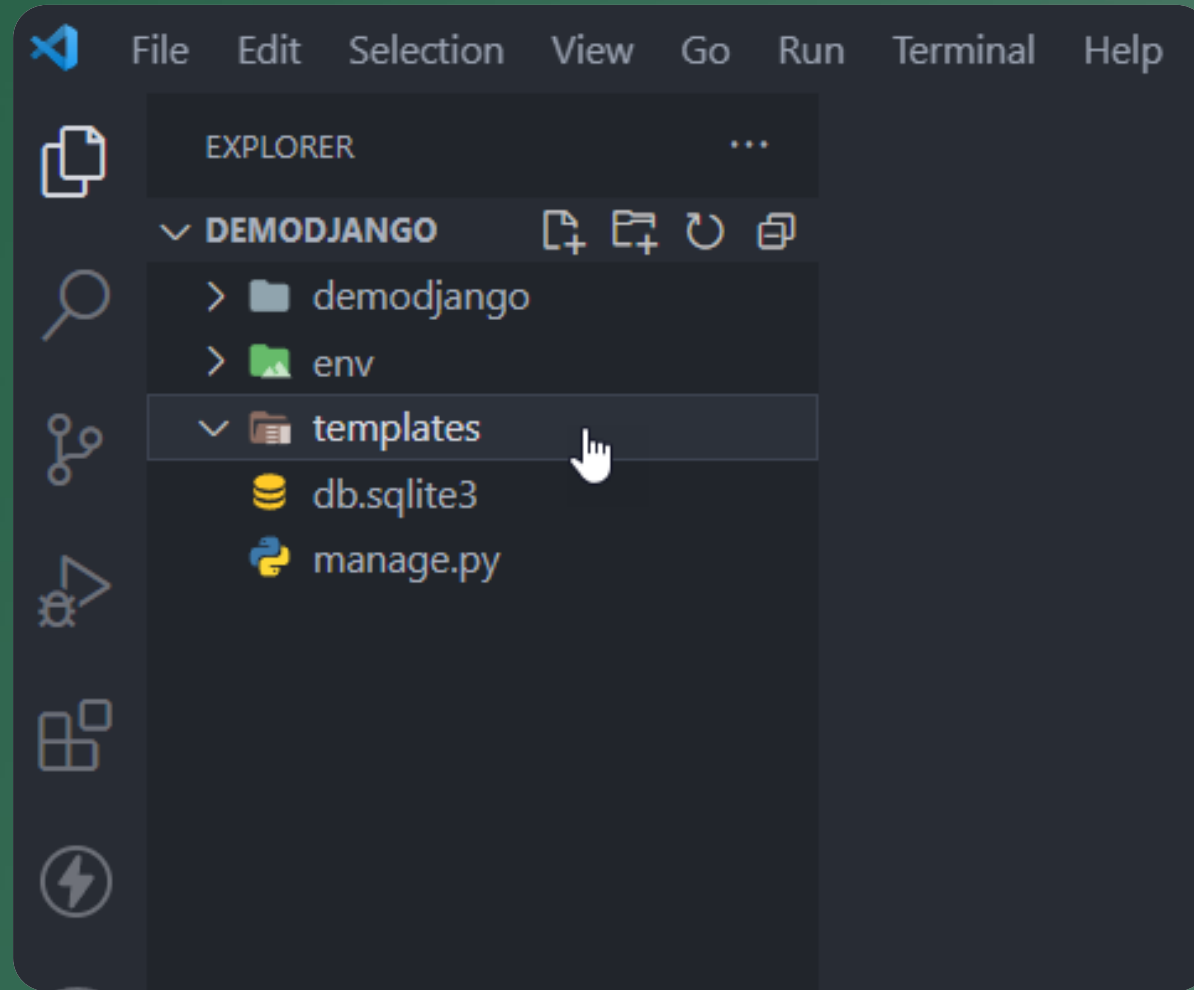
Django Template



อาจกล่าวได้ว่า Template ก็คือไฟล์เว็บเพจ (.html) ที่เราจะส่งกลับไปแสดงผลยังเว็บเบราว์เซอร์นั่นเอง ซึ่งสามารถสรุปขั้นตอนการทำงานโดยสังเขปได้ดังภาพต่อไปนี้



การติดตั้งไฟล์เดอร์ Template



การตั้งค่า path ของ Template ในไฟล์ settings.py

settings.py - demodjango - Visual Studio Code

EXPLORER

demodjango

- demodjango
 - __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py
 - views.py
 - wsgi.py
 - env
 - templates
 - db.sqlite3
 - manage.py

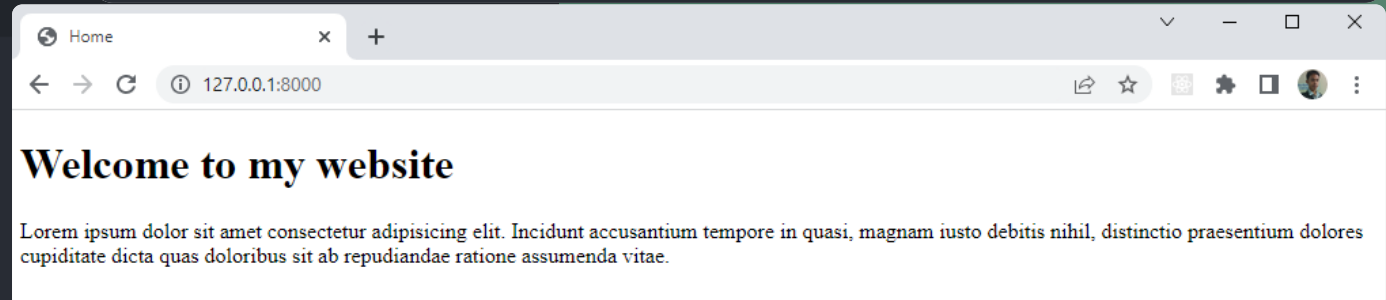
```
53
54 TEMPLATES = [
55     {
56         'BACKEND': 'django.template.backends.django.DjangoTemplates',
57         'DIRS': [Path.joinpath(BASE_DIR, 'templates')],
58         'APP_DIRS': True,
59         'OPTIONS': {
60             'context_processors': [
61                 'django.template.context_processors.debug',
62                 'django.template.context_processors.request',
63                 'django.contrib.auth.context_processors.auth',
64                 'django.contrib.messages.context_processors.messages',
65             ],
66         },
67     },
68 ]
69
```

การสร้างและแสดงผลจากไฟล์ Template

```
urls.py
demodjango > urls.py > ...
15 """
16 from django.contrib import admin
17 from django.urls import path
18 from . import views
19
20 urlpatterns = [
21     # การกำหนด path
22     path('', views.index, name='index'),
23     path('about', views.about),
24
25     # path แบบมี parameter
26     path('search/<str:keyword>/<int:page>', views.search),
27 ]
```

```
index.html
templates > index.html > html > head
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta http-equiv="X-UA-Compatible" content="IE=edge">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>Home</title>
8 </head>
9 <body>
10     <h1>Welcome to my website</h1>
11     <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Incidunt accu-
12     debitis nihil, distinctio praesentium dolores cupiditate dicta quas dolo-
13     vitae.</p>
14 </body>
15 </html>
```

```
views.py
demodjango > views.py > index
1 from django.http import HttpResponse
2 from django.shortcuts import render
3
4
5 def index(request):
6     # return HttpResponse('Welcome to my home page')
7     #แสดงผลไฟล์ index.html ที่สร้างไว้ในโฟลเดอร์ templates (ไม่ต้องระบุชื่อโฟลเดอร์)
8     return render(request, 'index.html')
9
```



Django Template Language (DTL)

สำหรับ Django Framework นั้น เราไม่สามารถแทรกโค้ดของภาษาไพธอนลงในเว็บเพจ (HTML) โดยตรงได้ แต่ต้องแทรกด้วยโค้ดที่มีรูปแบบเฉพาะของมันเอง ซึ่งเรียกว่า Django Template Language (DTL) โดยลักษณะการแทรกโค้ดแบ่งเป็น 3 ประเภท

{{ variable }} เป็นการแทรกค่าของตัวแปรลงในตำแหน่งที่ต้องการ เพื่อแสดงค่านั้นออกไปที่เบราว์เซอร์ (ค่าดังกล่าวจะกลายเป็นส่วนหนึ่งของเว็บเพจ)

{% tag %} เป็นการแทรกคำสั่ง เพื่อดำเนินการอย่างใดอย่างหนึ่ง เช่น กำหนดเงื่อนไข หรือการวนลูป หรือคำสั่งอื่นๆ (คำว่า tag ใน Django หมายถึงกลุ่มคำสั่งเฉพาะของมันเอง ซึ่งเป็นคนละอันกับแท็กของ HTML)

{{ variable|filter }} เป็นการกรองข้อมูลของตัวแปร โดยใช้เครื่องหมาย | เชื่อมระหว่างชื่อตัวแปรและชื่อฟิลเตอร์

การแทรกค่าของตัวแปร

The image shows a Django development environment with three code editors and a web browser.

urls.py:

```
demodjango > urls.py > ...  
19  
20 urlpatterns = [  
21     # การกำหนด path  
22     path('', views.index, name='index'),  
23     path('about', views.about),  
24     path('test', views.test, name='test'),  
25  
26
```

views.py:

```
demodjango > views.py > ...  
30 def test(request):  
31     data = {'title': 'Django Framework', 'message': 'Hello World'}  
32     return render(request, 'test.html', data)  
33
```

test.html:

```
templates > dj test.html  
1 <!DOCTYPE html>  
2 <html lang="en">  
3 <head>  
4     <meta charset="UTF-8">  
5     <meta http-equiv="X-UA-Compatible" content="IE=edge">  
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">  
7     <title>{{ title }}</title>  
8 </head>  
9 <body>  
10     <h2>{{ message }}</h2>  
11 </body>  
12 </html>
```

Browser:

Django Framework

127.0.0.1:8000/test

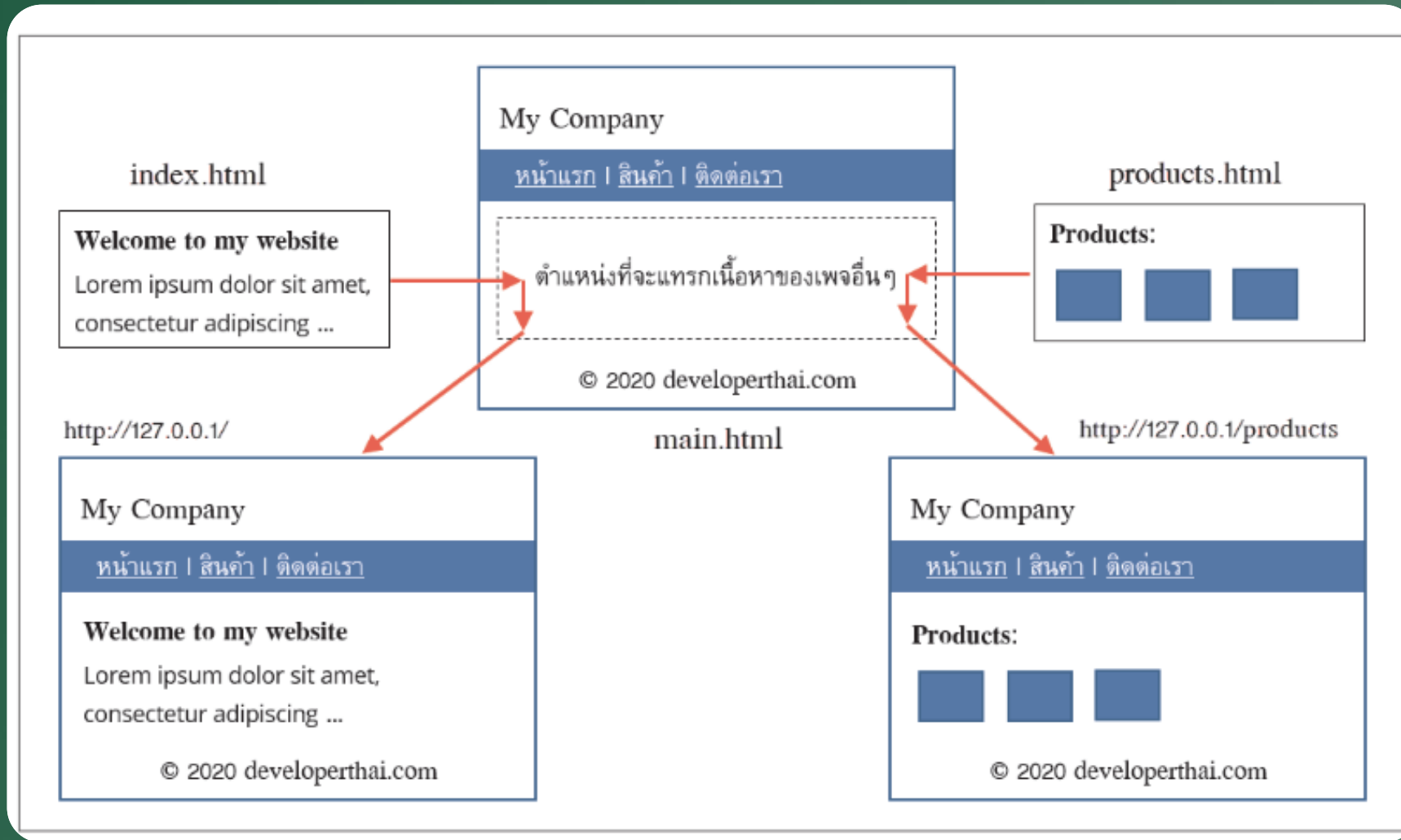
Hello World

คำสั่งประเภท Tag

```
{% tag %}  
...  
{% endtag %}
```

- tag คือชื่อแท็ก ซึ่งเป็นคำสั่งเฉพาะของแต่ละกรณี
- endtag ใช้กำหนดจุดสิ้นสุดของขอบเขตคำสั่ง โดยปกติจะเป็นคำว่า end ตามด้วยชื่อแท็ก เช่น กรณีแท็ก if จุดสิ้นสุดจะเป็นคำว่า endif หรือกรณีแท็ก with ก็ต้องระบุจุดสิ้นสุดด้วยคำว่า endwith เป็นต้น
- สิ่งที่อยู่ระหว่าง {% tag %} และ {% endtag %} อาจเป็นข้อความหรือโค้ด HTML ที่ต้องการแสดงผล หรือเป็นแท็กอื่นๆ ซ้อนกันลงไปอีกก็ได้

การใช้เท็มเพลตร่วมกัน



PYTHON django MVC

วันที่ **2**



5



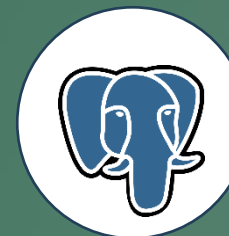
Samit Koyom
สถาบันไอทีจีเนียส



Day 2



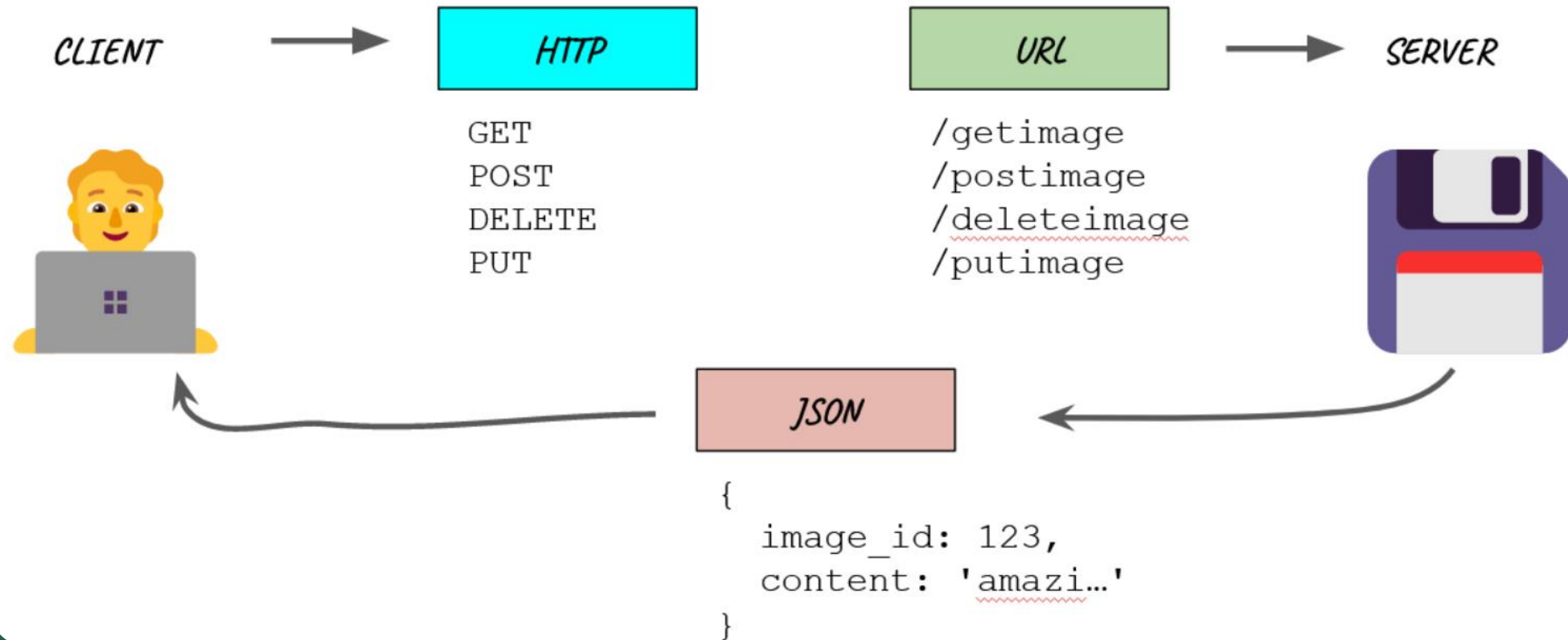
9. Django Rest Framework (DRF)
10. การทำ CRUD Rest API
11. การทำ Authentication (JWT) ใน DRF
12. การ Deploy Project Django แบบต่างๆ

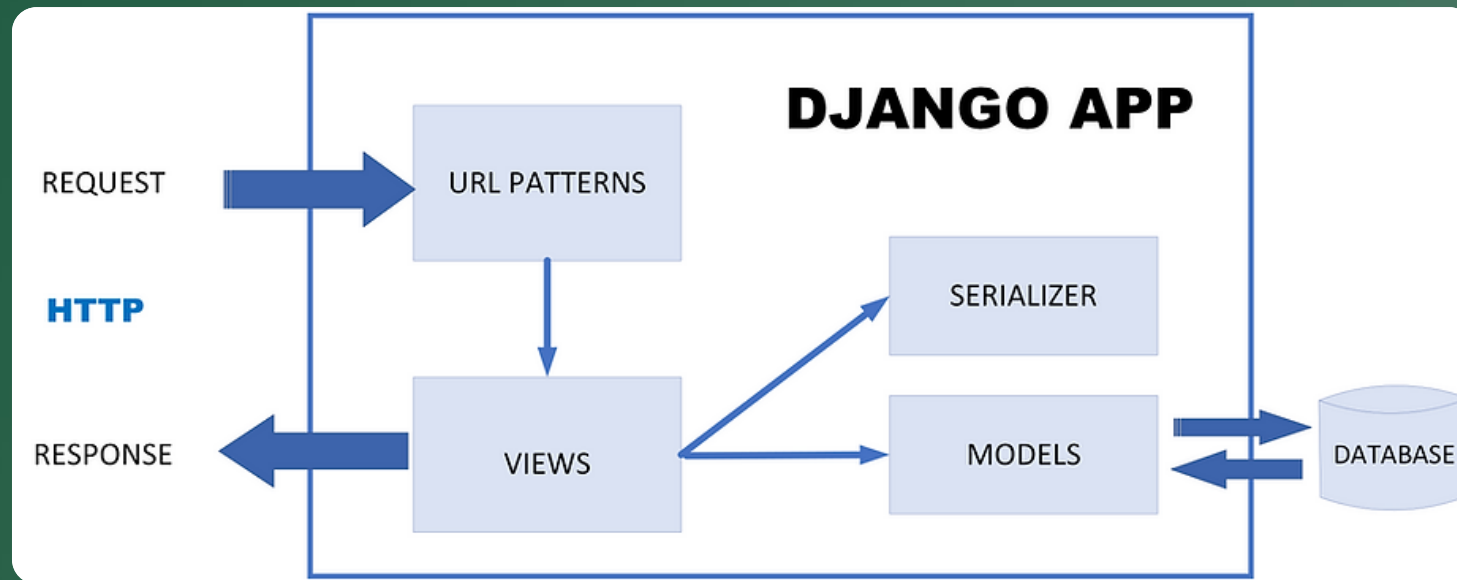


django

REST
framework

Django Building a REST API





องค์ประกอบหลักของ Django REST Framework มีดังนี้:

- **Requests (คำขอ):** คำขอที่ส่งจากฝั่งลูกค้า (client) ไปยัง API โดยคำขอเหล่านี้สามารถเป็นแบบ GET, POST, PUT หรือ DELETE และสามารถมีข้อมูลที่ส่งมาพร้อมในรูปแบบ JSON หรือรูปแบบอื่นๆ ได้
- **URL Patterns (รูปแบบ URL):** รูปแบบ URL ใช้สำหรับจับคู่คำขอ (request) กับวิธีการทำงานเฉพาะในระบบ (views)
- **Views (มุมมอง):** มุมมองทำหน้าที่จัดการคำขอที่เข้ามาและสร้างการตอบกลับ (response) ให้เหมาะสม
- **Serializers (ตัวแปลงข้อมูล):** ตัวแปลงข้อมูลทำหน้าที่แปลงข้อมูลระหว่างออบเจกต์ใน Python และข้อมูลในรูปแบบ JSON
- **Models (โมเดล):** โมเดลใน Django คือออบเจกต์ที่แทนข้อมูล (data) ที่ API ต้องการเปิดเผยหรือจัดการ

คุณสมบัติของ Django REST Framework

- **รองรับคำสั่ง HTTP ที่ใช้บ่อยในตัว:**

Django REST Framework มีการรองรับคำสั่ง HTTP ที่ใช้บ่อยใน RESTful API เช่น GET, POST, PUT และ DELETE ในตัว ทำให้สามารถดำเนินการ CRUD (Create, Read, Update, Delete) บนทรัพยากร API ของคุณได้อย่างง่ายดาย

- **การแปลงข้อมูลไปมาแบบอัตโนมัติระหว่าง JSON:**

Django REST Framework มีการจัดการการแปลงข้อมูลไปเป็น JSON และกลับมาเป็นข้อมูล Python อัตโนมัติ หมายความว่า你不จำเป็นต้องเขียนโค้ดเพื่อแปลงข้อมูลเอง เพราะระบบจะทำงานให้อัตโนมัติ

- **การรองรับการยืนยันตัวตนและการกำหนดสิทธิ์:**

Django REST Framework มีการรองรับระบบยืนยันตัวตน (Authentication) และการกำหนดสิทธิ์ (Authorization) ในตัว ช่วยให้สามารถป้องกันการเข้าถึงทรัพยากร API โดยไม่ได้รับอนุญาตได้อย่างง่ายดาย

- **การรองรับเวอร์ชันของ API:**

Django REST Framework มีการรองรับระบบเวอร์ชันของ API ในตัว ช่วยให้คุณสามารถออกเวอร์ชันใหม่ของ API ได้โดยไม่กระทบกับลูกค้าปัจจุบัน

- **การควบคุมปริมาณการใช้งาน (Throttling):**

Django REST Framework มีการรองรับระบบควบคุมปริมาณการใช้งาน API ช่วยจำกัดจำนวนคำขอที่ลูกค้าสามารถส่งมายัง API ภายในช่วงเวลาที่กำหนด

- **เอกสาร API แบบบราวเซอร์ได้:**

Django REST Framework มีเอกสาร API แบบที่สามารถเข้าถึงและเรียกดูผ่านเบราว์เซอร์ได้ในตัว ทำให้ลูกค้าสามารถเรียนรู้การใช้งาน API ของคุณได้ง่ายขึ้น

เว็บไซต์ยอดนิยมที่ใช้ Django REST Framework

- Instagram
- Pinterest
- Mozilla
- Eventbrite
- National Geographic
- Brown University
- The Guardian
- Public Radio International
- Disqus
- Sentry
- Read the Docs

Django REST Framework เป็นเครื่องมือที่ทรงพลังและยืดหยุ่นสำหรับการสร้าง Web API ด้วย Django โดยมีการใช้งานในเว็บไซต์และบริษัทที่มีชื่อเสียงมากมาย ทำให้เป็นตัวเลือกที่ยอดเยี่ยมสำหรับผู้ที่กำลังพัฒนา RESTful API ด้วย Django

Serializer ใน Django Framework

Serializer ใน Django คือเครื่องมือที่ช่วยในการแปลงข้อมูลระหว่าง Python objects และ รูปแบบข้อมูลที่สามารถถ่ายโอนได้ เช่น JSON หรือ XML เพื่อใช้ใน API โดยเฉพาะกับ RESTful API ใน Django REST Framework

บทบาทของ Serializer

1. แปลงข้อมูลจาก Python objects ไปเป็น JSON
ใช้เมื่อคุณต้องการส่งข้อมูลให้กับ client (เช่น เบร่าวเซอร์ หรือ แอปพลิเคชันมือถือ) ในรูปแบบ JSON
2. แปลงข้อมูลจาก JSON หรือรูปแบบอื่นๆ เป็น Python objects
ใช้เมื่อคุณต้องการรับข้อมูลจาก client และแปลงเป็นข้อมูลที่สามารถบันทึกลงฐานข้อมูลหรือประมวลผลต่อได้

การทำงานของ Serializer

Serializer จะทำหน้าที่เหมือนกับฟอร์มใน Django (Django Forms) แต่เน้นในด้านการแปลงข้อมูลสำหรับ API

- Validation: ตรวจสอบความถูกต้องของข้อมูลที่ได้รับเข้ามา เช่น ความยาวข้อมูล หรือรูปแบบอีเมล
- Serialization: แปลงข้อมูลให้อยู่ในรูปแบบที่เหมาะสมสำหรับการส่งออก
- Deserialization: แปลงข้อมูลที่ส่งเข้ามาให้อยู่ในรูปแบบ Python objects

Serializer ใน Django Framework

ตัวอย่างการใช้งาน Serializer

โมเดลตัวอย่าง

python

คัดลอกโค้ด

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=100)
    author = models.CharField(max_length=50)
    published_date = models.DateField()
```

Serializer สำหรับโมเดล

python

คัดลอกโค้ด

```
from rest_framework import serializers

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ['id', 'title', 'author', 'published_date']
```

Serializer ใน Django Framework

การใช้งานใน View

python

📄 คัดลอกโค้ด

```
from rest_framework.views import APIView
from rest_framework.response import Response
from .models import Book
from .serializers import BookSerializer

class BookListView(APIView):
    def get(self, request):
        books = Book.objects.all()
        serializer = BookSerializer(books, many=True)
        return Response(serializer.data)
```

ข้อดีของการใช้ Serializer

1. ลดความซับซ้อนในการจัดการข้อมูลระหว่างฐานข้อมูลและ API
2. รองรับการตรวจสอบข้อมูล (Validation) อัตโนมัติ
3. ใช้งานร่วมกับ Django REST Framework ได้ง่ายและยืดหยุ่น

Serializer เป็นองค์ประกอบที่สำคัญใน Django REST Framework ที่ช่วยให้นักพัฒนาสามารถพัฒนา RESTful API ได้อย่างมีประสิทธิภาพ

Getting Started with Django REST Framework

STEP 1: Install

Install Django and Django REST framework into the virtual environment

```
pip install django
pip install django_rest_framework
```

STEP 2: Create a Django Project and App:

```
django-admin startproject myproject
cd myproject
django-admin startapp myapp
```

Update your project's settings.py file to add the Django REST Framework to the INSTALLED_APPS list:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

Getting Started with Django REST Framework

STEP 3: Create a model

A model defines the structure of your data. In this example, you're creating a "Todo" model with attributes like "title," "description," and "completed." This model will determine how your data is stored in the database.

Write your Django model in the models.py file of your app. For example:

```
from django.db import models

class Todo(models.Model):
    title = models.CharField(max_length=255)
    description = models.TextField()
    completed = models.BooleanField(default=False)
```

Getting Started with Django REST Framework

STEP 4: Create a serializer

Serializers are responsible for converting data between Python objects and JSON. To create a serializer, create a class in the `serializers.py` file of your app that inherits from `rest_framework.serializers.ModelSerializer`.

For example:

Here you will create a **"TodoSerializer"** to facilitate this conversion for your **"Todo"** model.

```
from rest_framework import serializers
from myapp.models import Todo

class TodoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Todo
        fields = ['title', 'description', 'completed']
```

Getting Started with Django REST Framework

STEP 5: Create a view

Views are responsible for handling requests to your API. To create a view, create a class in the `views.py` file of your app that inherits from `rest_framework.views.APIView`.

For example:

Here, you will create a `"TodoListView"` class that will handle GET requests to fetch a list of todos. It uses the `"TodoSerializer"` to format the data.

```
from rest_framework.views import APIView
from rest_framework.response import Response
from myapp.models import Todo
from myapp.serializers import TodoSerializer

class TodoListView(APIView):
    def get(self, request):
        todos = Todo.objects.all()
        serializer = TodoSerializer(todos, many=True)
        return Response(serializer.data)
```

Getting Started with Django REST Framework

STEP 6: Configure your API

To make your API accessible, you need to define URL patterns. In this step, you're adding a URL pattern that maps to the "TodoListView" view. When a request is made to <http://localhost:8000/api/todos/>, it will be handled by the "TodoListView" class.

```
from django.urls import path
from myapp.views import TodoListView

urlpatterns = [
    ...
    path('api/todos/', TodoListView.as_view()),
]
```

Run the development server:

```
python manage.py runserver
```

Getting Started with Django REST Framework

STEP 7: Test your API

You can test your API using a tool like Postman. To create a new request, click the "New" button and select "POST". In the "Request URL" field, enter `http://localhost:8000/api/todos/`. In the "Body" tab, select "JSON" and enter the following JSON data:

```
{
  "title": "My first todo",
  "description": "This is my first todo."
}
```

Click the "Send" button to send the request. If the request is successful, you will see a response with the following JSON data:

```
{
  "id": 1,
  "title": "My first todo",
  "description": "This is my first todo.",
  "completed": false
}
```

โครงสร้างโปรเจกต์ Rest API “Inventory Store”

```
inventory_store/
├── api/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── serializers.py
│   ├── permissions.py
│   ├── filters.py
│   ├── pagination.py
│   ├── tasks.py
│   ├── urls.py
│   └── views.py
├── authn/
│   ├── __init__.py
│   ├── apps.py
│   ├── models.py
│   ├── serializers.py
│   ├── urls.py
│   └── views.py
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   ├── urls.py
│   ├── wsgi.py
│   ├── settings/
│   │   ├── __init__.py
│   │   ├── base.py
│   │   ├── dev.py
│   │   └── prod.py
├── env/
│   └── .env.example
├── manage.py
├── pyproject.toml / requirements.txt
└── README.md

# แอปหลัก รวม endpoint ทั้งหมด

# Product, Category, Supplier, Warehouse, PurchaseOrder
# DRF serializers ของแต่ละโมเดล
# เช่น IsStaffOrReadOnly, IsWarehouseUser
# django-filter สำหรับ list endpoints
# PageNumberPagination ค่าเริ่มต้น
# (ตัวเลือก) งาน background/สคริปต์
# รวม route ของ resource inventory
# ViewSets / APIViews

# แอปย่อยสำหรับ JWT และโปรไฟล์ผู้ใช้

# (ตัวเลือก) UserProfile, Role/Permission เสริมจาก Django
# RegisterSerializer, LoginSerializer, ChangePasswordView
# /api/auth/*
# RegisterView, TokenObtainPairView, TokenRefreshView

# โปรเจกต์ Django (settings แยก environment)

# รวม /api/, /api/auth/, /swagger/

# INSTALLED_APPS, REST_FRAMEWORK, SIMPLE_JWT, SPECTACULAR_API
# DEBUG=True, DB dev
# DEBUG=False, DB prod, CORS/SECURITY

# ไฟล์ตัวอย่าง environment
# DJANGO_SECRET_KEY, DB_*, DEBUG, ALLOWED_HOSTS, CORS

# django-rest-framework, django-rest-framework-simplejwt
```

API Endpoint ที่ต้องการ

1) Authentication (JWT)

- **POST** `/api/auth/register/` — สมัครผู้ใช้ใหม่ (staff/warehouse user)
- **POST** `/api/auth/login/` — รับ access & refresh token (ใช้ `TokenObtainPairView`)
- **POST** `/api/auth/refresh/` — ขอ access token ใหม่ด้วย refresh token
- **POST** `/api/auth/logout/` — ยกเลิก token (เซ็งลोजิกฝั่งแอป; หรือใช้ blacklist ถ้าเปิดใช้)
- **GET** `/api/auth/me/` — โปรไฟล์ผู้ใช้ปัจจุบัน

Header ที่ใช้หลังล็อกอิน: `Authorization: Bearer <access_token>`

2) Master Data

- Categories
 - GET `/api/categories/` — ทั้งหมด (รองรับค้นหา/ฟิลเตอร์ชื่อ)
 - POST `/api/categories/`
 - GET `/api/categories/{id}/`
 - PUT `/api/categories/{id}/`
 - DELETE `/api/categories/{id}/`
- Units (หน่วยนับ เช่น pcs, box)
 - GET/POST `/api/units/`
 - GET/PUT/DELETE `/api/units/{id}/`
- Suppliers
 - GET/POST `/api/suppliers/`
 - GET/PUT/DELETE `/api/suppliers/{id}/`
- Warehouses
 - GET/POST `/api/warehouses/`
 - GET/PUT/DELETE `/api/warehouses/{id}/`
- Products
 - GET/POST `/api/products/` — ฟิลเตอร์ `?sku=&name=&category=&supplier=`
 - GET/PUT/DELETE `/api/products/{id}/`

3) การจัดการสต็อก (Operations)

- Purchase Orders (PO) (สั่งซื้อเข้า)
 - GET/POST `/api/purchase-orders/` — ฟิลเดอร์ `?status=pending|received`
 - GET/PUT/DELETE `/api/purchase-orders/{id}/`
 - POST `/api/purchase-orders/{id}/receive/` — ปิด PO และสร้าง StockMovement รับเข้า
- Stock Movements (เคลื่อนไหวสต็อก: in/out/transfer/adjust)
 - GET/POST `/api/stock-movements/`
 - GET/PUT/DELETE `/api/stock-movements/{id}/`
 - ประเภท (type): IN , OUT , TRANSFER , ADJUST
 - ปลายทาง/ต้นทางคลัง: `source_warehouse` , `target_warehouse` (สำหรับ TRANSFER)
- Stock Counts (ตรวจนับ)
 - GET/POST `/api/stock-counts/` — เปิดรอบตรวจนับ
 - GET/PUT/DELETE `/api/stock-counts/{id}/`
 - POST `/api/stock-counts/{id}/finalize/` — ปิดรอบและสร้าง ADJUST movement อัตโนมัติ
- Inventory Snapshot / Balance
 - GET `/api/inventory/balance/` — คงเหลือปัจจุบันแบบรวม หรือ ฟิลเดอร์ `?warehouse=&product=`
 - GET `/api/inventory/ledger/` — สมุดรายวันสต็อกต่อสินค้า `?product=&from=&to=`

4) รายงาน (เบื้องต้น)

- GET `/api/reports/top-products/` — สินค้าเคลื่อนไหวสูงสุด ช่วงเวลา `?from=&to=`
 - GET `/api/reports/low-stock/` — ต่ำกว่าจุดสั่งซื้อ (reorder point)
-

5) เอกสาร API (Swagger / OpenAPI)

- GET `/swagger/` — Swagger UI
- GET `/api/schema/` — OpenAPI JSON (drf-spectacular)

ตัวอย่างโครงสร้าง URL (ตอนรัน dev)

text

Copy Edit

```
http://127.0.0.1:8000/api/products/  
http://127.0.0.1:8000/api/suppliers/  
http://127.0.0.1:8000/api/purchase-orders/  
http://127.0.0.1:8000/api/stock-movements/  
http://127.0.0.1:8000/api/inventory/balance/  
http://127.0.0.1:8000/api/reports/low-stock/  
  
http://127.0.0.1:8000/api/auth/register/  
http://127.0.0.1:8000/api/auth/login/  
http://127.0.0.1:8000/api/auth/refresh/  
http://127.0.0.1:8000/api/auth/me/  
  
http://127.0.0.1:8000/swagger/
```

เริ่มต้นสร้างโปรเจกต์ Rest API “inventory_store”

Step 1

1. สร้าง Virtual Environment



Bash

← Ask

📄 Copy

▶ Run

```
# สร้าง virtual environment
python -m venv env

# เปิดใช้งาน virtual environment
# บน Windows
env\Scripts\activate

# บน macOS/Linux
source env/bin/activate
```

Step 2 & 3

2. ติดตั้ง Django, Django Rest Framework และ psycopg2

 Bash

↶ Ask

📄 Copy

▶ Run

```
pip install django djangorestframework drf-yasg psycopg2-binary
```

3. สร้าง Django Project และ App

 Bash

↶ Ask

📄 Copy

▶ Run

```
# สร้างโปรเจกต์ Django
django-admin startproject inventory_store

# เข้าไปในโฟลเดอร์โปรเจกต์
cd inventory_store

# สร้างแอปพลิเคชัน
python manage.py startapp api
```

Step 4

4. ตั้งค่าใน settings.py

เปิดไฟล์ `inventory_store/settings.py`

เพิ่มแอปพลิเคชันและ REST framework เข้าไปใน **INSTALLED_APPS** :

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
    'drf_yasg',  
    'api',  
]
```

ตั้งค่าฐานข้อมูล PostgreSQL:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'your_db_name',  
        'USER': 'your_db_user',  
        'PASSWORD': 'your_db_password',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

Step 5

5. สร้าง Models

แก้ไขไฟล์ `api/models.py`:

```
from __future__ import annotations

import uuid
from decimal import Decimal
from django.conf import settings
from django.core.validators import MinValueValidator
from django.db import models
from django.utils import timezone

class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

Step 6

6. สร้าง Serializers

สร้างไฟล์ `api/serializers.py` และเพิ่มโค้ดต่อไปนี้:

```
# api/serializers.py
from __future__ import annotations

from decimal import Decimal
from django.db import transaction
from rest_framework import serializers

from .models import (
    Category,
    Unit,
    Supplier,
    Warehouse,
    Product,
    PurchaseOrder,
    PurchaseOrderItem,
    StockMovement,
    StockCount,
    StockCountItem,
    on_hand,
)
```

Step 7

7. สร้าง Views

สร้างไฟล์ `api/views.py` และเพิ่มโค้ดต่อไปนี้:

```
# api/views.py
from __future__ import annotations

from decimal import Decimal
from django.db import transaction
from django.db.models import Q
from django.utils import timezone

from rest_framework import permissions, viewsets
from rest_framework.decorators import action
from rest_framework.response import Response

from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter, OrderingFilter

from .models import (
    Category,
    Unit,
```

Step 8

8. ตั้งค่า URLs

แก้ไขไฟล์ `api/urls.py`:

```
from rest_framework.routers import DefaultRouter
from .views import (
    CategoryViewSet, UnitViewSet, SupplierViewSet, WarehouseViewSet, ProductView
    PurchaseOrderViewSet, StockMovementViewSet, StockCountViewSet, InventoryView
)

router = DefaultRouter()
router.register(r"categories", CategoryViewSet)
router.register(r"units", UnitViewSet)
router.register(r"suppliers", SupplierViewSet)
router.register(r"warehouses", WarehouseViewSet)
router.register(r"products", ProductViewSet)
router.register(r"purchase-orders", PurchaseOrderViewSet)
router.register(r"stock-movements", StockMovementViewSet)
router.register(r"stock-counts", StockCountViewSet)
router.register(r"inventory", InventoryViewSet, basename="inventory")
urlpatterns = router.urls
```

Step 9 & 10

9. ทำ Migration และรันเซิร์ฟเวอร์



Bash

← Ask

📄 Copy

▶ Run

```
# สร้างไฟล์ migration
python manage.py makemigrations

# ใช้ migration เพื่อสร้างตารางในฐานข้อมูล
python manage.py migrate

# รันเซิร์ฟเวอร์
python manage.py runserver
```

10. ทดสอบ API

เปิดเบราว์เซอร์และไปที่ <http://127.0.0.1:8000/swagger/> เพื่อดูเอกสาร Swagger ของ API



Django Rest API Auth with JWT

ติดตั้ง Library สำหรับ JWT / Cors

```
pip install djangorestframework-simplejwt django-cors-headers
```

```
33 INSTALLED_APPS = [  
34     'django.contrib.admin',  
35     'django.contrib.auth',  
36     'django.contrib.contenttypes',  
37     'django.contrib.sessions',  
38     'django.contrib.messages',  
39     'django.contrib.staticfiles',  
40     'rest_framework',  
41     'corsheaders',  
42     'drf_yasg',  
43     'api',  
44 ]
```

```
45  
46 REST_FRAMEWORK = {  
47     'DEFAULT_AUTHENTICATION_CLASSES': (  
48         'rest_framework_simplejwt.authentication.JWTAuthentication',  
49     ),  
50     'DEFAULT_PERMISSION_CLASSES': (  
51         'rest_framework.permissions.IsAuthenticated',  
52     ),  
53 }
```

```
54  
55 SIMPLE_JWT = {  
56     'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), # ระยะเวลาของ Access Token คือ 1 ชั่วโมง  
57     'REFRESH_TOKEN_LIFETIME': timedelta(days=1), # ระยะเวลาของ Refresh Token คือ 1 วัน  
58     'ROTATE_REFRESH_TOKENS': True, # หมุนเวลา Refresh Token  
59     'BLACKLIST_AFTER_ROTATION': True, # เปิดใช้งานการตรวจสอบ blacklist  
60     'ALGORITHM': 'HS256', # อัลกอริธึมสำหรับการเข้ารหัส  
61     'SIGNING_KEY': SECRET_KEY, # คีย์สำหรับการเข้ารหัส  
62     'AUTH_HEADER_TYPES': ('Bearer',), # ประเภทของส่วนหัวสำหรับการอนุญาต  
63     'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',), # คลาสของสิทธิ์การเข้าสู่ระบบ  
64     'UPDATE_LAST_LOGIN': True, # อัปเดตเวลาการเข้าสู่ระบบครั้งสุดท้าย  
65 }
```

```
66
67 MIDDLEWARE = [
68     'corsheaders.middleware.CorsMiddleware', # เพิ่ม CorsMiddleware ที่ตำแหน่งบนสุด
69     'django.middleware.security.SecurityMiddleware',
70     'django.contrib.sessions.middleware.SessionMiddleware',
71     'django.middleware.common.CommonMiddleware',
72     'django.middleware.csrf.CsrfViewMiddleware',
73     'django.contrib.auth.middleware.AuthenticationMiddleware',
74     'django.contrib.messages.middleware.MessageMiddleware',
75     'django.middleware.clickjacking.XFrameOptionsMiddleware',
76 ]
77
```

dekend > settings.py > [๑] TEMPLATES

```
135     },
136 ]
137
138 # ตั้งค่าการอนุญาต CORS
139 # อนุญาตทุกโดเมนให้เข้าถึง (เปิด CORS ทั้งหมด)
140 CORS_ALLOW_ALL_ORIGINS = True
141
142 # อนุญาตเฉพาะโดเมนที่ระบุให้เข้าถึง
143 # CORS_ALLOWED_ORIGINS = [
144 #     "https://example.com",
145 #     "https://sub.example.com",
146 #     "http://localhost:3000",
147 #     "http://127.0.0.1:3000",
148 # ]
149
150 # ถ้าต้องการกำหนด HTTP Methods ที่อนุญาต เช่น GET, POST, PUT, PATCH, DELETE
151 CORS_ALLOW_METHODS = [
152     "GET",
153     "POST",
154     "PUT",
155     "PATCH",
156     "DELETE",
157     "OPTIONS",
158 ]
159
160 # ถ้าต้องการกำหนด Headers ที่อนุญาต เช่น Authorization, Content-Type
161 CORS_ALLOW_HEADERS = [
162     "content-type",
163     "authorization",
164     "x-csrf-token",
165     "x-requested-with",
166 ]
167
168
```

dekend > settings.py > ...

```
160 # ถ้าต้องการกำหนด Headers ที่อนุญาต เช่น Authorization, Content-Type
161 CORS_ALLOW_HEADERS = [
162     "content-type",
163     "authorization",
164     "x-csrftoken",
165     "x-requested-with",
166 ]
167
168
169 # Internationalization
170 # https://docs.djangoproject.com/en/5.1/topics/i18n/
171
172 LANGUAGE_CODE = 'en-us'
173
174 TIME_ZONE = 'Asia/Bangkok'
175
176 USE_I18N = True
177
178 USE_TZ = True
179
180
181 # Static files (CSS, JavaScript, Images)
182 # https://docs.djangoproject.com/en/5.1/howto/static-files/
183
184 STATIC_URL = 'static/'
185
186 # Default primary key field type
187 # https://docs.djangoproject.com/en/5.1/ref/settings/#default-auto-field
188
189 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
190
```

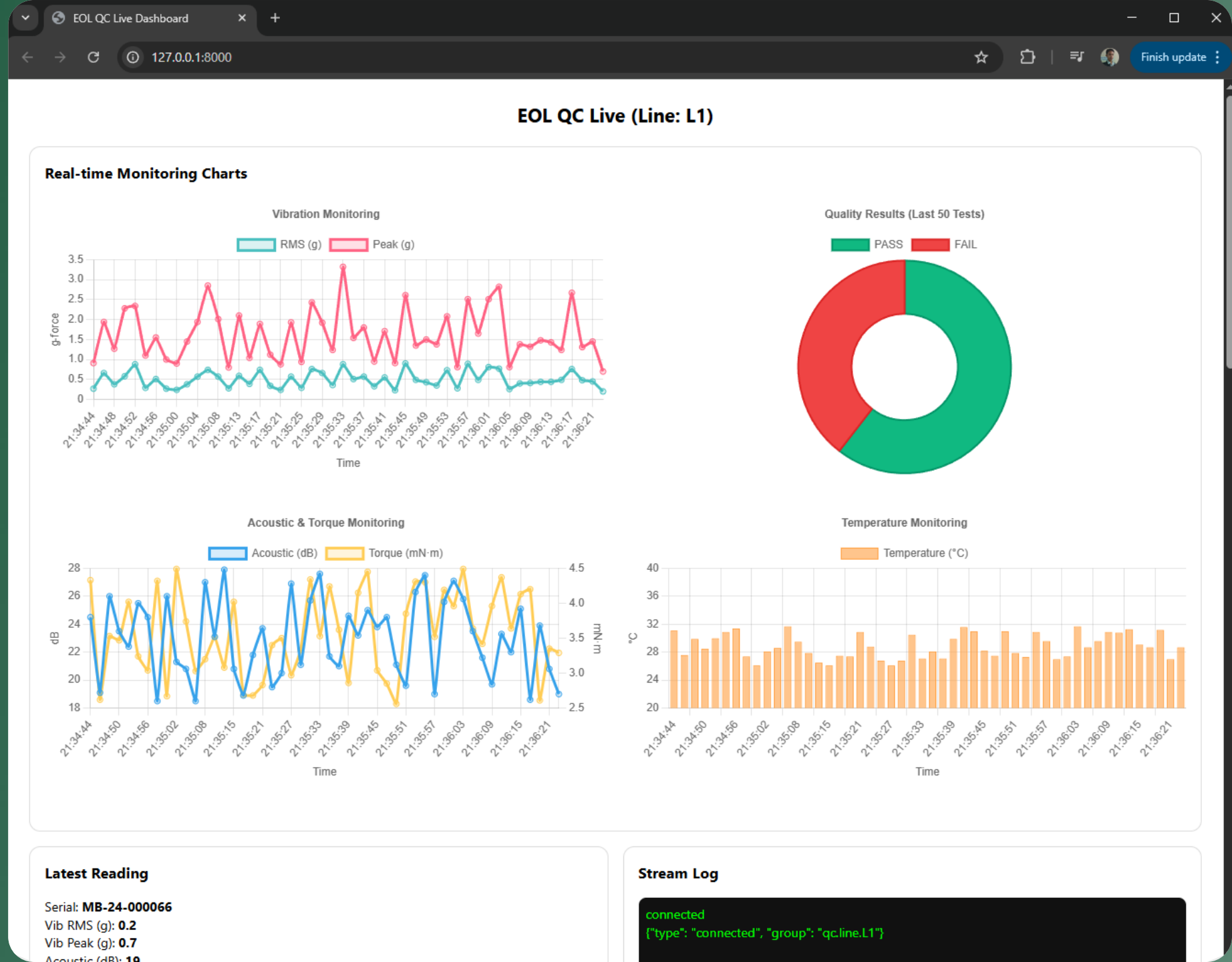
WORKSHOP

EOL QC Live



WebSocket

EOL QC Live



EOL QC Live



EOL QC Live Dashboard

127.0.0.1:8000

Finish update

Latest Reading

Serial: MB-24-000106
Vib RMS (g): 0.66
Vib Peak (g): 1.78
Acoustic (dB): 23.5
Torque (mN·m): 3.69
Temp (°C): 31
Result: PASS
Time: 14/08/2568 21:37:44

Stream Log

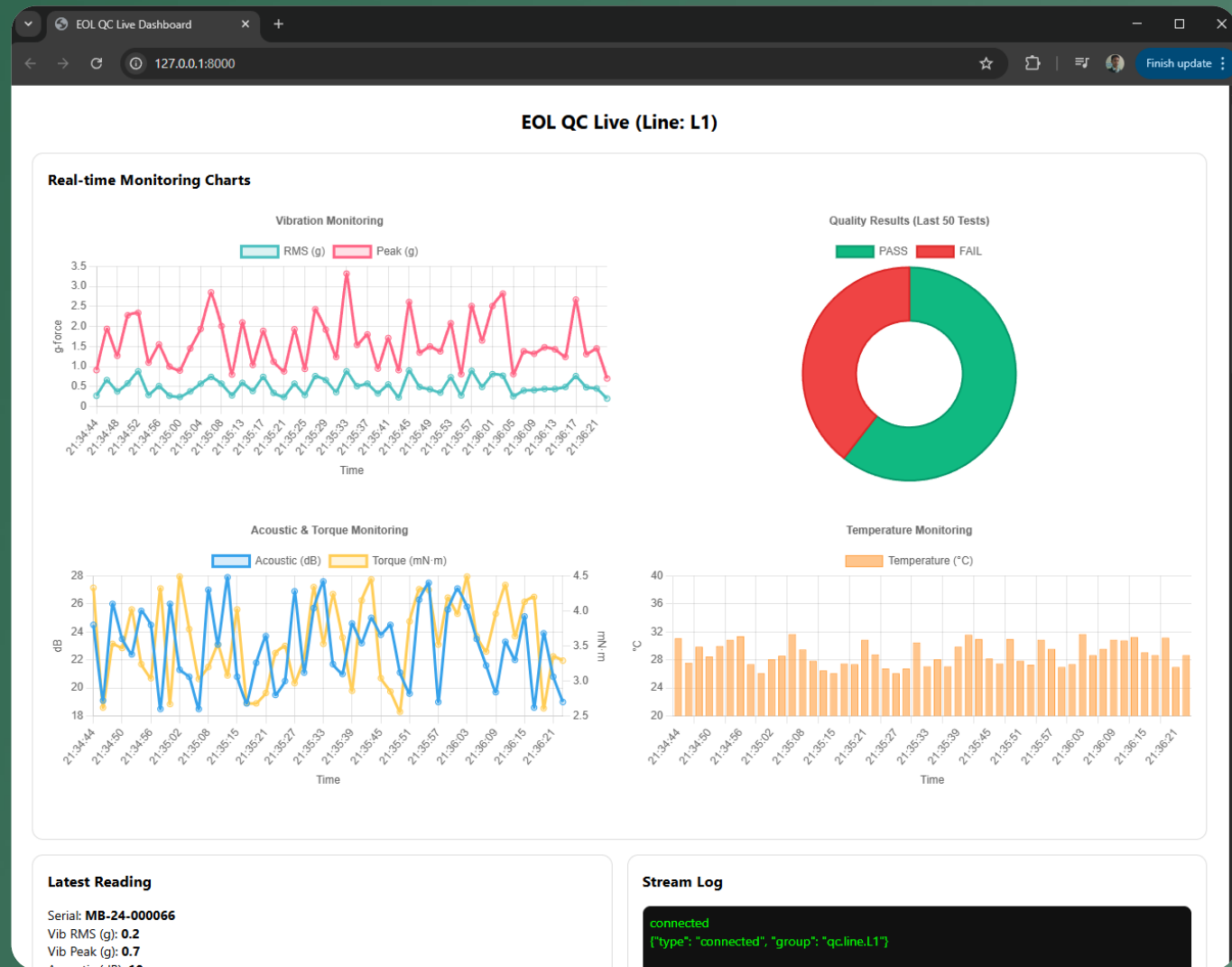
connected
{"type": "connected", "group": "qc.line.L1"}

แสดงข้อมูล: 50 / 50 รายการล่าสุด

TS	Serial	RMS	Peak	dB	Torque	Temp	Result
14/08/2568 21:37:44	MB-24-000106	0.66	1.78	23.5	3.69	31	PASS
14/08/2568 21:37:42	MB-24-000105	0.66	2.51	25	3.09	26.5	PASS
14/08/2568 21:37:40	MB-24-000104	0.32	0.92	24.5	3.42	28.6	PASS
14/08/2568 21:37:38	MB-24-000103	0.39	1.34	20.5	2.9	26	PASS
14/08/2568 21:37:36	MB-24-000102	0.23	0.74	22.3	2.88	27.5	PASS
14/08/2568 21:37:34	MB-24-000101	0.83	3.06	20.1	4.17	31.2	FAIL
14/08/2568 21:37:32	MB-24-000100	0.77	2.9	20.6	2.56	30	PASS
14/08/2568 21:37:30	MB-24-000099	0.73	2.37	26.6	4.46	30.9	FAIL
14/08/2568 21:37:28	MB-24-000098	0.4	1.5	25	4.39	30.3	FAIL
14/08/2568 21:37:26	MB-24-000097	0.61	1.55	24.8	2.69	26.7	PASS
14/08/2568 21:37:24	MB-24-000096	0.65	2.31	22.5	3.61	29.7	PASS
14/08/2568 21:37:22	MB-24-000095	0.36	1.16	22.8	3.41	28.5	PASS
14/08/2568 21:37:20	MB-24-000094	0.73	2.43	18.3	3.66	26.4	PASS
14/08/2568 21:37:18	MB-24-000093	0.8	3.07	26.8	3.95	27.6	FAIL
14/08/2568 21:37:16	MB-24-000092	0.55	1.49	21.6	3.06	27.7	PASS
14/08/2568 21:37:14	MB-24-000091	0.63	2.39	25.4	3.45	29.3	PASS
14/08/2568 21:37:12	MB-24-000090	0.74	2.51	21.4	2.91	26.7	PASS

Tech Stack

EOL QC Live



Tech Stack

EOL QC Live



Backend (ASGI / Realtime)

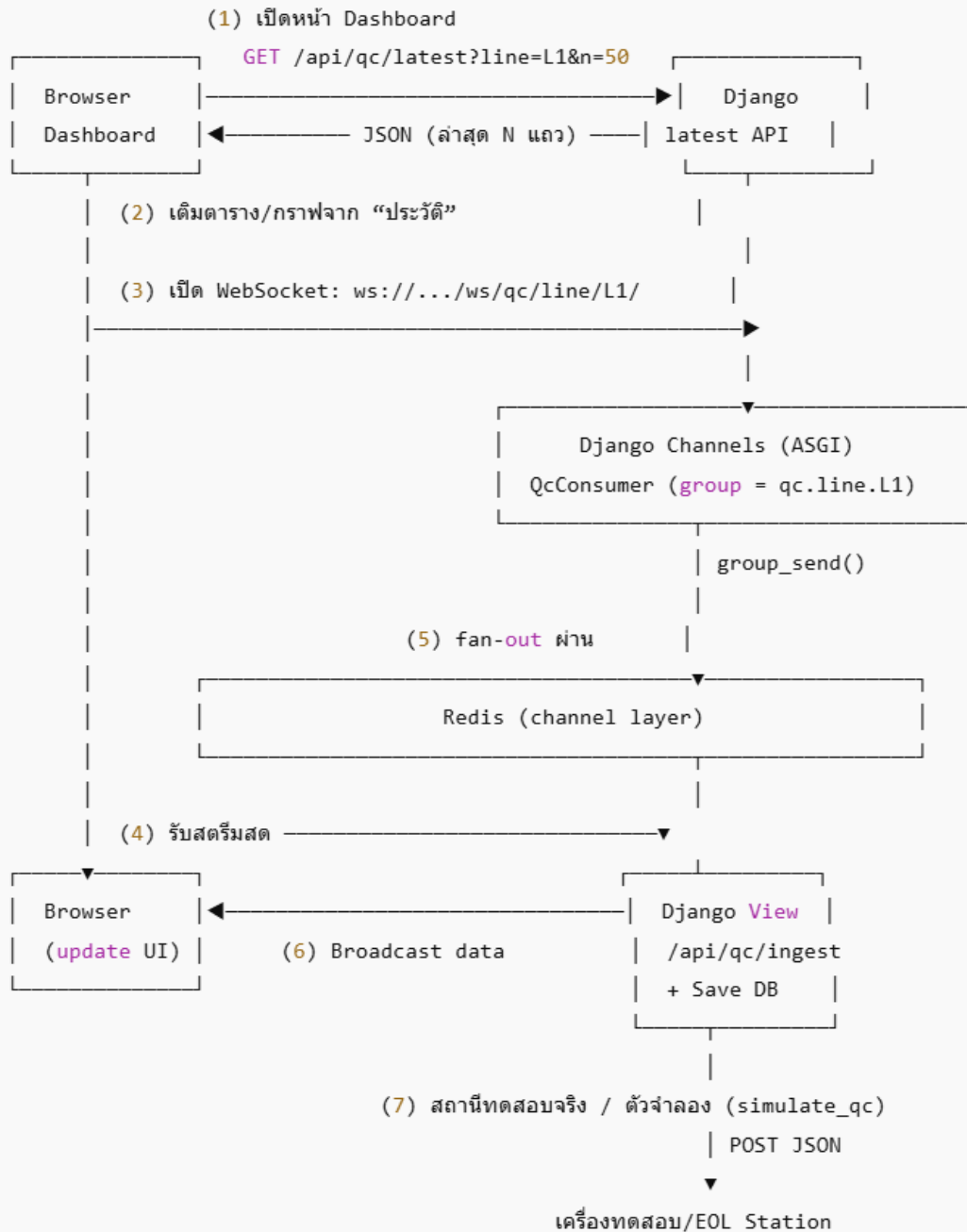
- Django 5.x + ASGI เป็นเว็บเฟรมเวิร์ก/โครงหลักของโปรเจกต์ และรองรับงาน async/WS ผ่านสเปก ASGI โดยตรง [Django Project +1](#)
- Django Channels 4.x เพิ่มความสามารถ WebSocket/กลุ่ม (group) และการส่งข้อความข้ามโปรเซสด้วย “channel layer” [channels.readthedocs.io](#)
- channels_redis (Redis channel layer) ใช้ Redis เป็นตัวกลางกระจายข้อความ (pub/sub) ไปยังทุก consumer/worker ที่อยู่ในกลุ่มเดียวกัน (เช่น `qc.line.L1`) — แนวทางที่เอกสารแนะนำสำหรับ production [channels.readthedocs.io](#)
- ASGI Server สำหรับ dev/poC ใช้ Uvicorn (ติดตั้งแบบ `uvicorn[standard]` เพื่อให้รองรับ WebSocket ผ่าน `websockets/wsproto`) หรือใช้ Daphne ก็ได้ [uvicorn.org +1](#)
[channels.readthedocs.io](#)
- Database (ถาวร/Traceability): เก็บผลทดสอบลงโมเดล `QcResult` (เช่น SQLite/PostgreSQL ตามแวดล้อม)

Frontend

- WebSocket API ฟังก์ชันเบราว์เซอร์ (JS) สำหรับรับสตรีมสดจาก Channels แบบสองทาง (full-duplex) [MDN Web Docs +2](#)
- Chart.js ใช้แสดงกราฟ realtime (Line/Doughnut/ ฯลฯ) บนแดชบอร์ด (โหลดผ่าน CDN ในหน้า `dashboard.html`) [chartjs.org +1](#)

Infra/Runtime

- Redis 6/7 เป็น channel layer backend (ข้อความสด at-most-once ในโหมด pub/sub) — ไม่ใช่ที่เก็บถาวรของข้อมูลทดสอบ แต่เป็น “ท่อส่งข้อความ” ให้ WS เท่านั้น [Redis](#)



System Flow

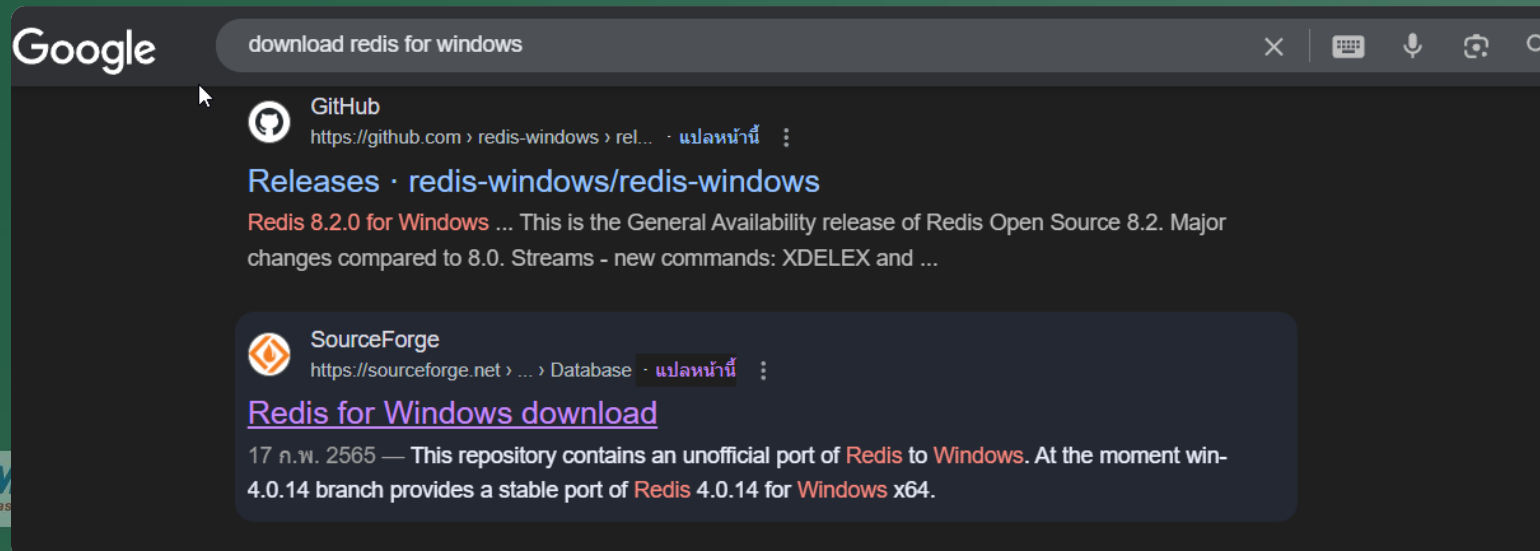
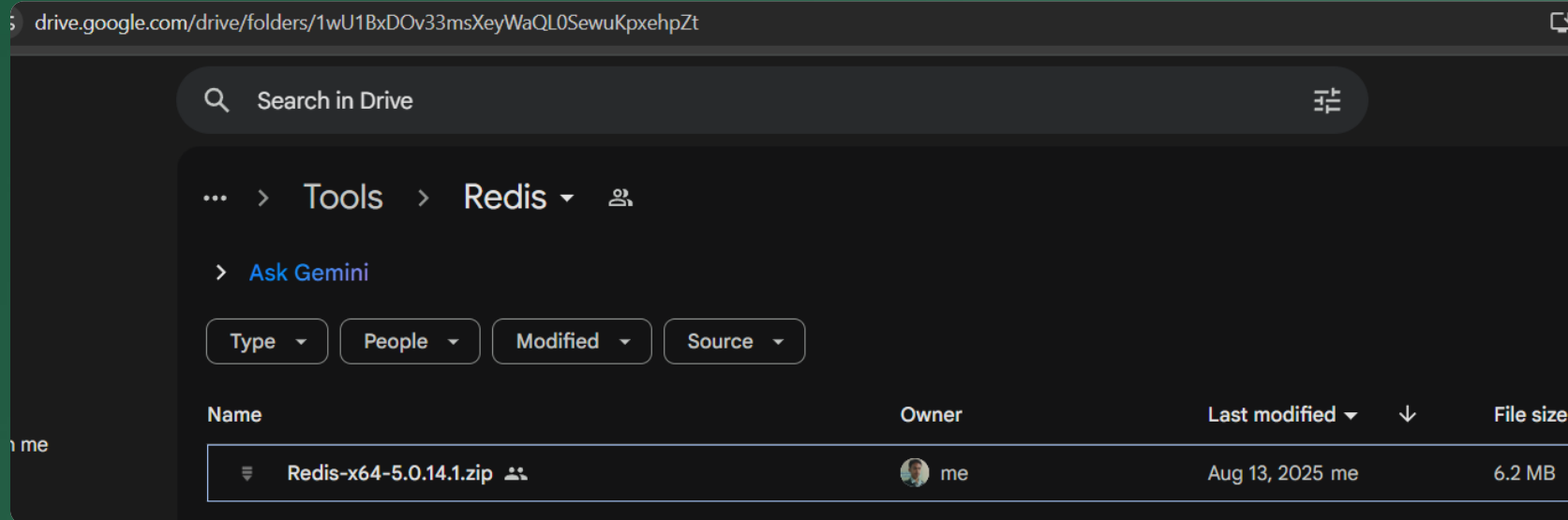
EOL QC Live

อธิบายลำดับเวลาโดยย่อ

1. หน้า Dashboard โหลด ย้อนหลัง ผ่าน REST GET /api/qc/latest → เติมตาราง/กราฟ
2. จากนั้นเปิด WebSocket ไปยัง ws/qc/line/L1/
3. ฟังก์ชันทดสอบ (จริง/จำลอง) ยิงผลทดสอบ → /api/qc/ingest
4. View บันทึกลง DB แล้ว group_send ไปยัง qc.line.L1
5. Redis (channel layer) กระจายข้อความถึงทุก consumer/worker ที่อยู่ในกลุ่ม
6. QcConsumer ส่งเฟรม WS ให้เบราว์เซอร์ → Dashboard อัปเดตแบบเรียลไทม์

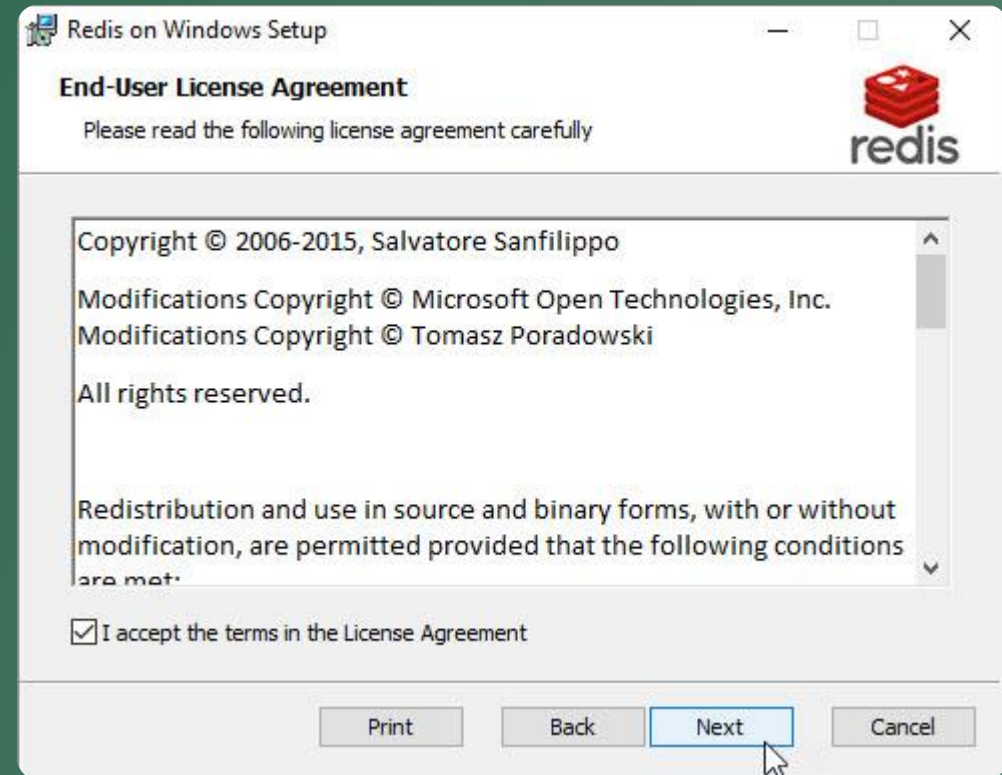
หมายเหตุ: กระบวนการ fan-out ผ่าน Redis เป็นกลไกของ Channels channel layer (แนะนำ production) และฝั่ง Unicorn ต้องติดตั้ง extras เพื่อรองรับ WebSocket เติมรูปแบบครบ channels.readthedocs.io +1 uvicorn.org

0. Setup “Redis” On Windows



สถาบันไอทีจีเนียส

เริ่มติดตั้ง Redis สำหรับ Windows กัน



กำหนด path และ port ในการรัน Redis (ใช้ตามค่าเริ่มต้น)

Redis on Windows Setup

Destination Folder

Click Next to install to the default folder or click Change to choose another.

Install Redis on Windows to:

C:\Program Files\Redis\

Change...

☒ Add the Redis installation folder to the PATH environment variable.

Back Next Cancel

Redis on Windows Setup

Port Number and Firewall Exception

Select whether to add an exception to the Windows Firewall for Redis.

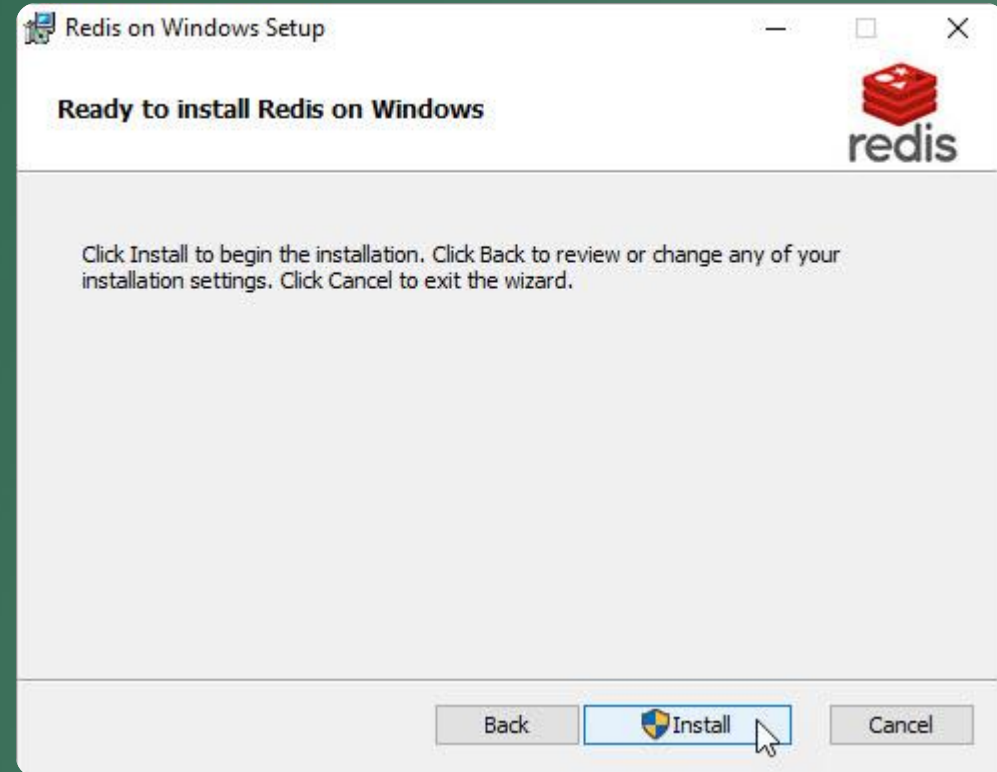
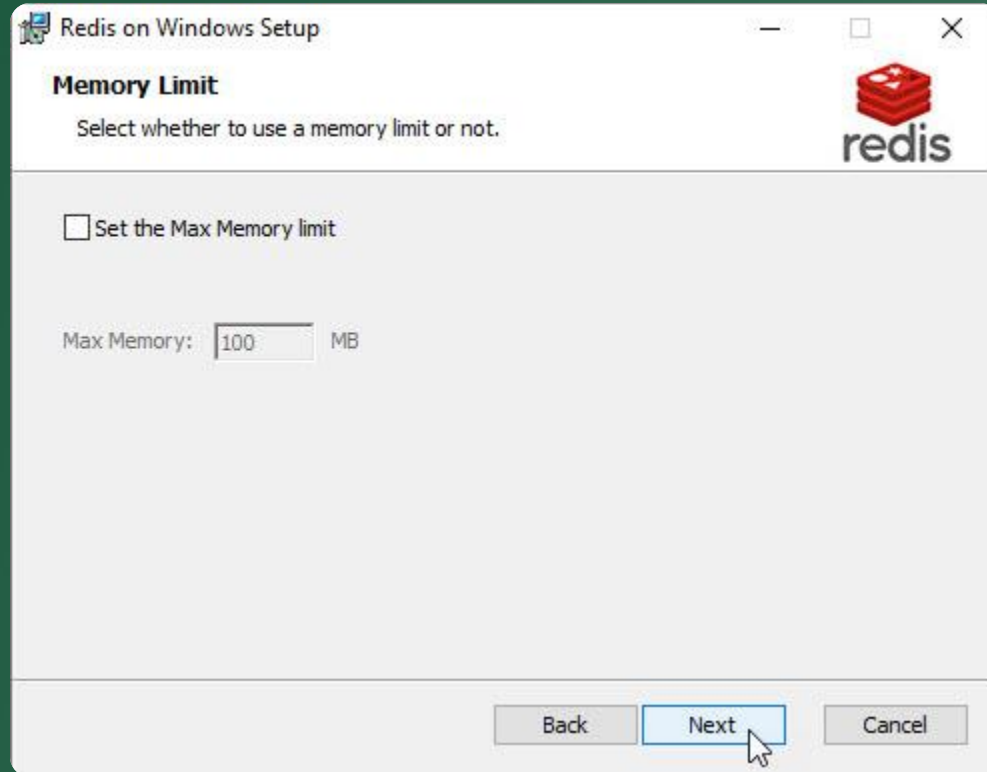
Port to run Redis on:

6379

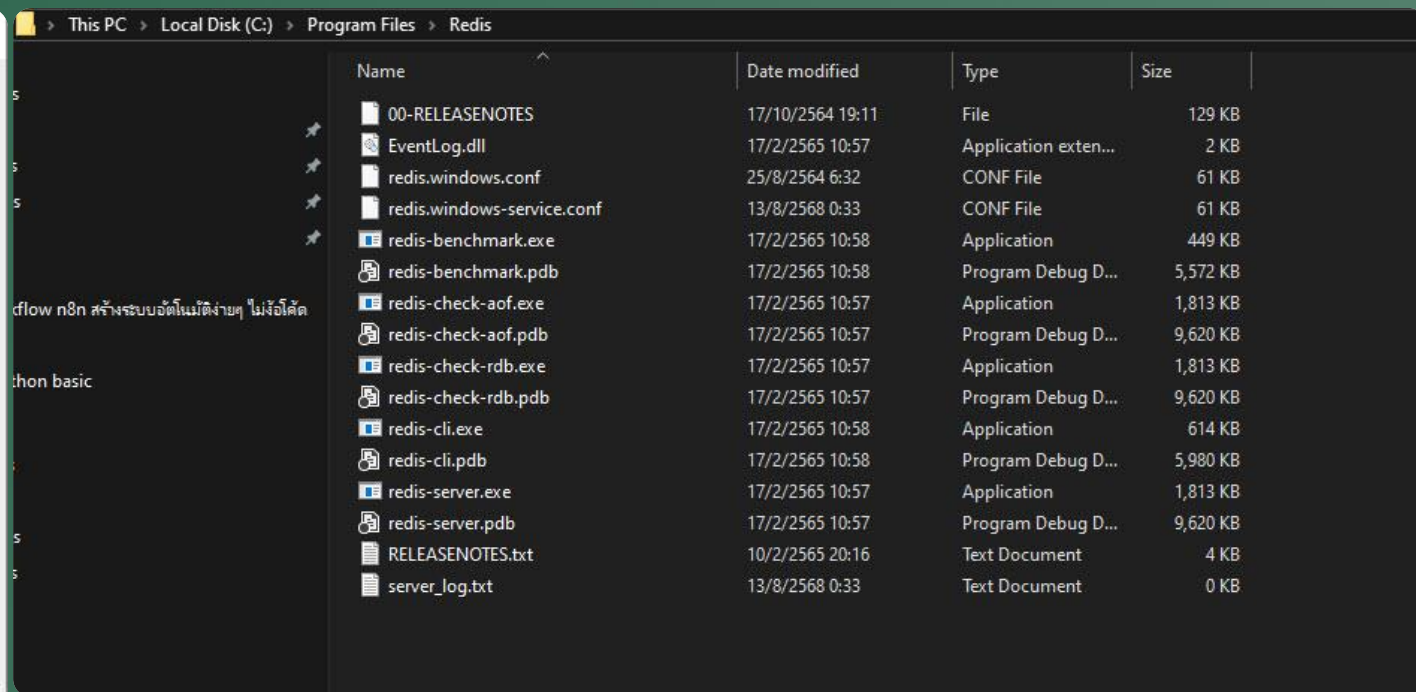
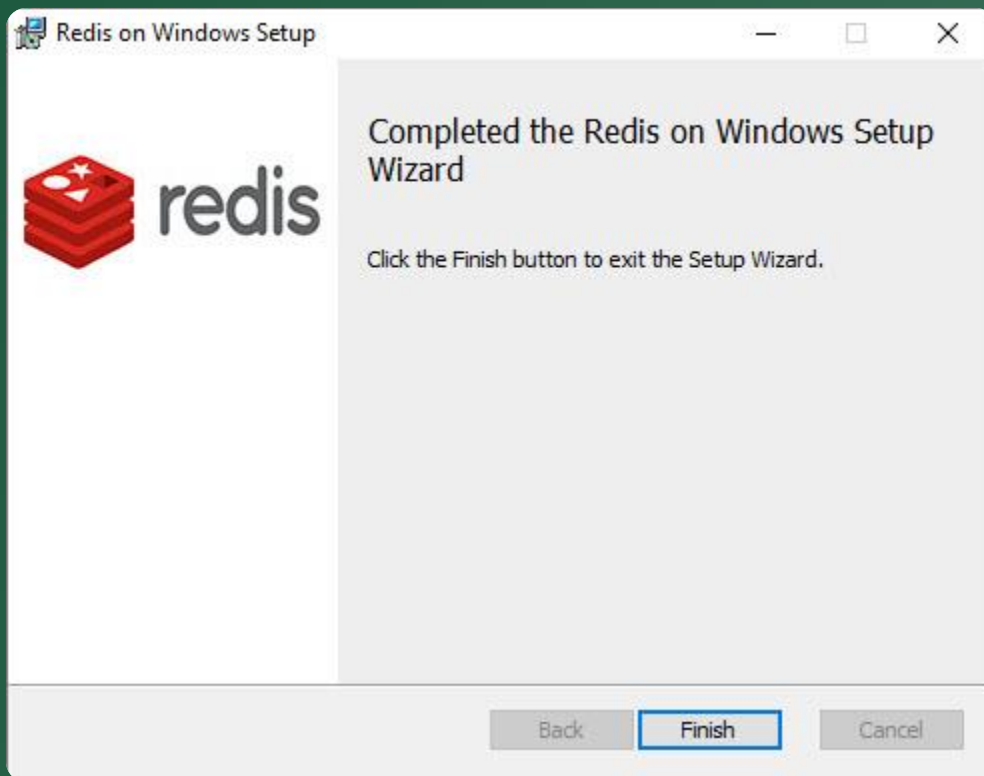
☒ Add an exception to the Windows Firewall.

Back Next Cancel

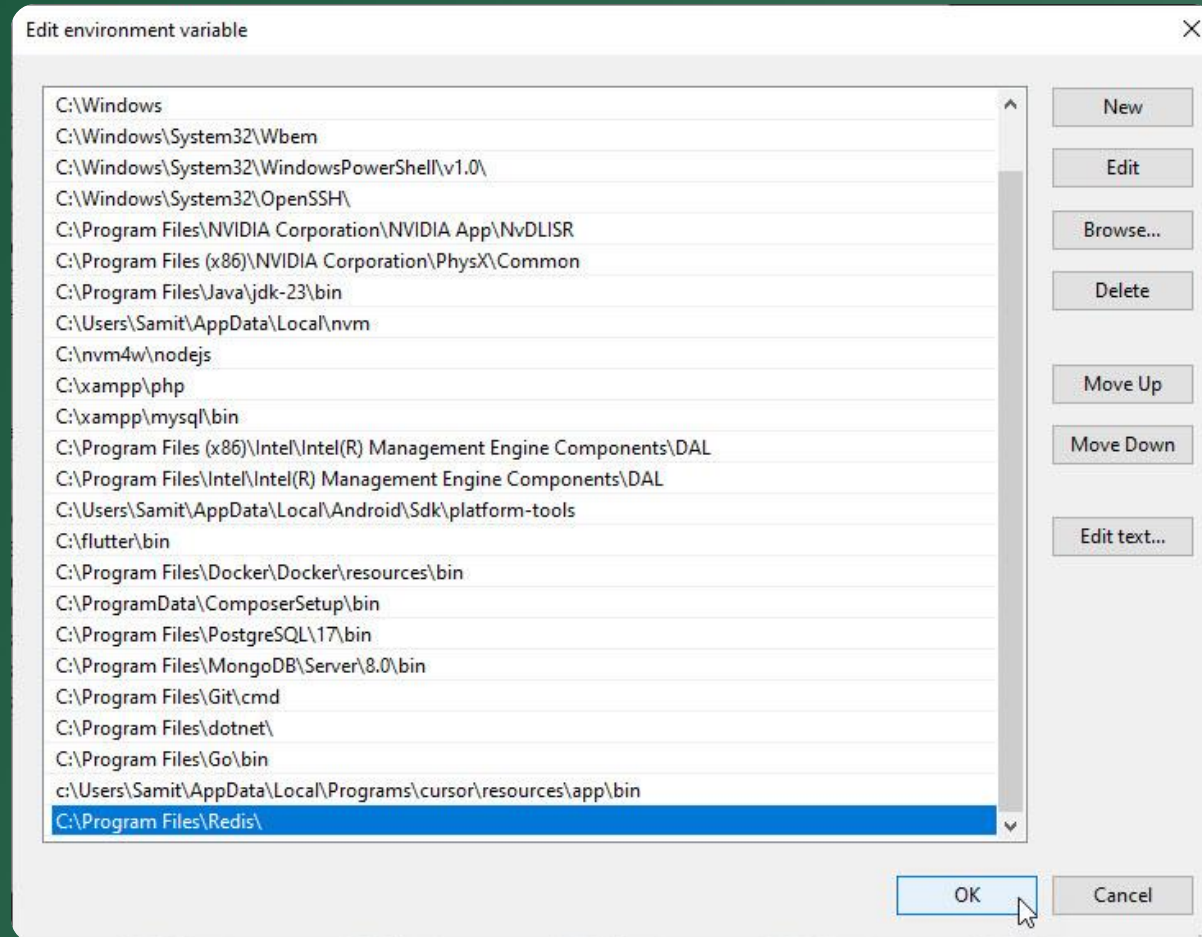
Next and Install ...



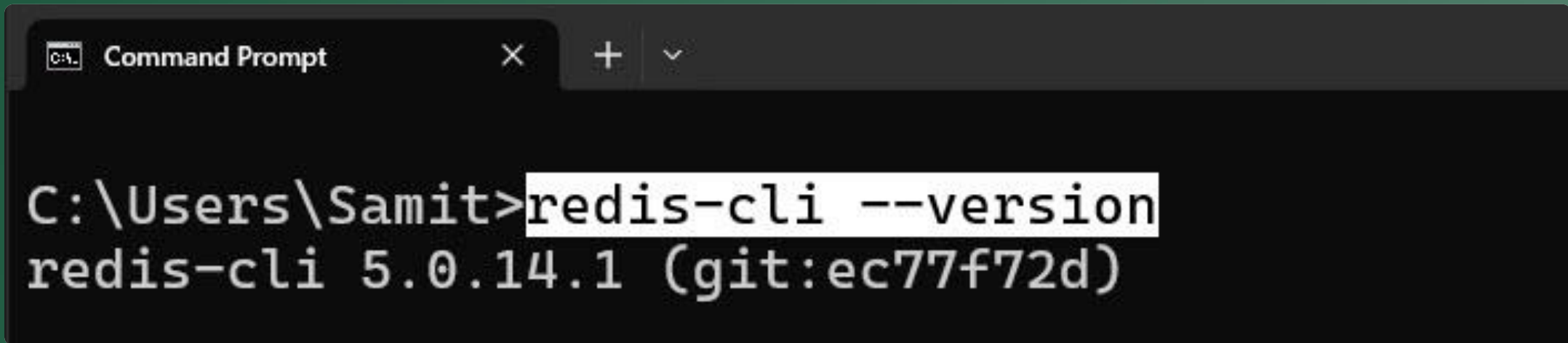
เสร็จแล้วพร้อมใช้



ถ้ามีปัญหาการเรียกใช้ลองเช็ค Path ใน env ของ Windows



เช็ค Version ของ Redis Cli ถ้าพบก็แสดงว่าใช้งานได้ปกติ

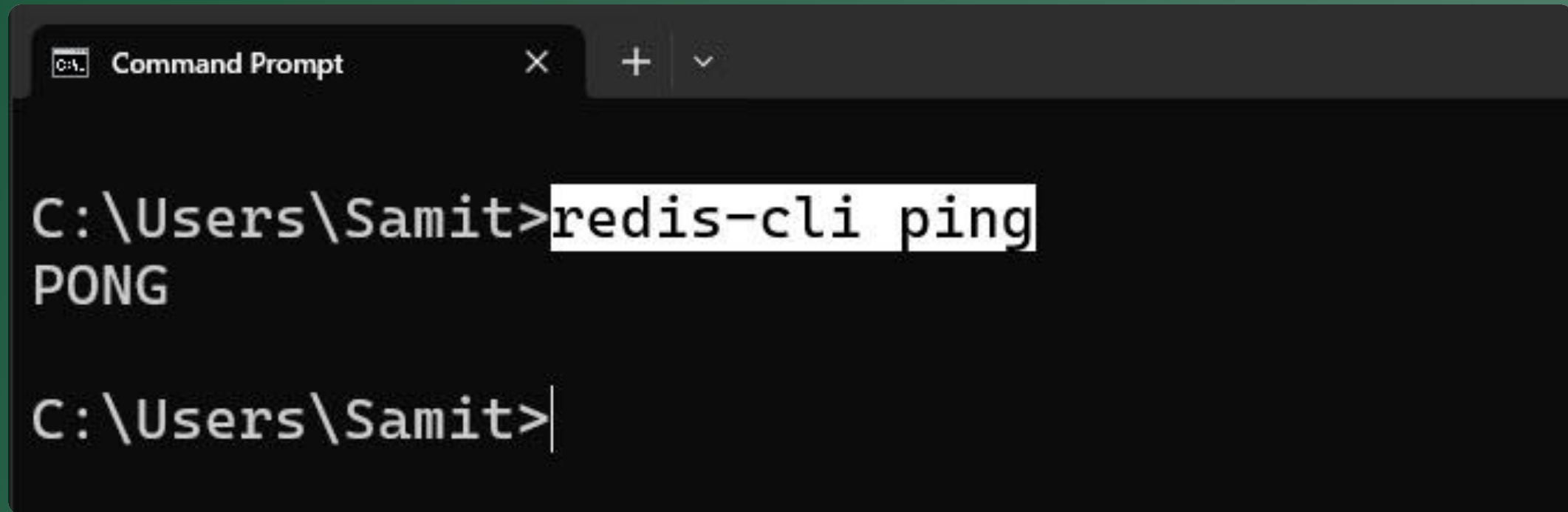


```
Command Prompt
C:\Users\Samit>redis-cli --version
redis-cli 5.0.14.1 (git:ec77f72d)
```

คำสั่งดูว่า Redis ติดตั้งไว้ที่ไหน ถ้าพบก็แสดงว่าใช้งานได้ปกติ

```
Command Prompt
C:\Users\Samit>where redis-cli
C:\Program Files\Redis\redis-cli.exe
```

ทดสอบ ping ถ้าตอบกลับมาว่า PONG ก็คุยกันได้แล้ว



```
C:\Users\Samit>redis-cli ping
PONG

C:\Users\Samit>
```

The image shows a Windows Command Prompt window with a dark background. The title bar at the top says "Command Prompt" and includes standard window controls (close, maximize, minimize). The command prompt shows the user "Samit" at the "C:\Users\Samit" directory. They have entered the command "redis-cli ping", which has been highlighted with a white selection box. The output of the command is "PONG", indicating a successful connection to the Redis server. The prompt is now ready for the next command.

คำสั่งพื้นฐานใน Redis ที่น่ารู้จัก

1 คำสั่งทั่วไป (General)

text

 Copy code

PING	# ทดสอบการเชื่อมต่อ (ตอบ PONG)
AUTH <password>	# ยืนยันตัวตนถ้าตั้ง requirepass
SELECT <db>	# เลือก database (0-15 ค่าเริ่มต้น)
DBSIZE	# จำนวน key ทั้งหมดใน DB ปัจจุบัน
KEYS <pattern>	# ค้นหา key (เช่น KEYS user:*)
SCAN 0	# ค้นหา key แบบไม่บล็อก (ใช้แทน KEYS ในระบบใหญ่)
FLUSHDB	# ลบข้อมูลใน DB ปัจจุบัน
FLUSHALL	# ลบข้อมูลทุก DB

คำสั่งพื้นฐานใน Redis ที่น่ารู้จัก

2 Key-Value พื้นฐาน

text

 Copy code

SET mykey "Hello"	# สร้าง key
GET mykey	# อ่านค่า
DEL mykey	# ลบ key
EXISTS mykey	# ตรวจสอบว่ามี key หรือไม่
TTL mykey	# ดูเวลาหมดอายุ (เป็นวินาที)
EXPIRE mykey 60	# กำหนดเวลาหมดอายุ 60 วินาที

คำสั่งพื้นฐานใน Redis ที่น่ารู้จัก

3 String (นับเลข / ต่อข้อความ)

text

 Copy code

INCR counter

เพิ่มค่า +1

DECR counter

ลดค่า -1

APPEND mykey " World"

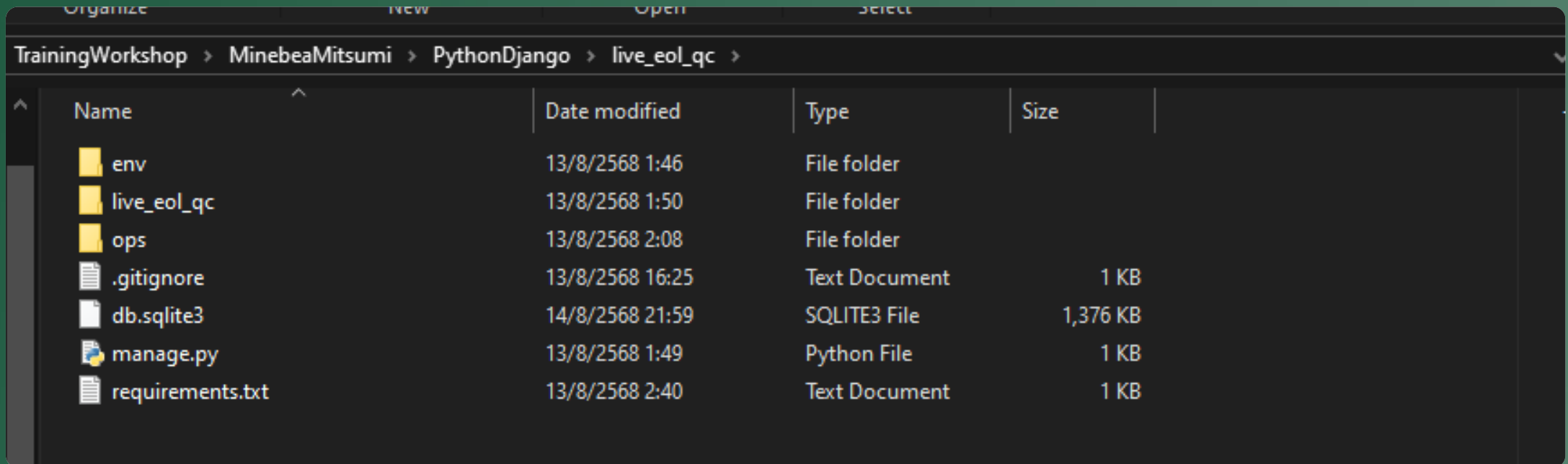
ต่อข้อความเดิม

GETRANGE mykey 0 4

อ่านบางส่วน (index เริ่มจาก 0)

1. Create project “live_eol_qc”

```
django-admin startproject live_eol_qc .
```

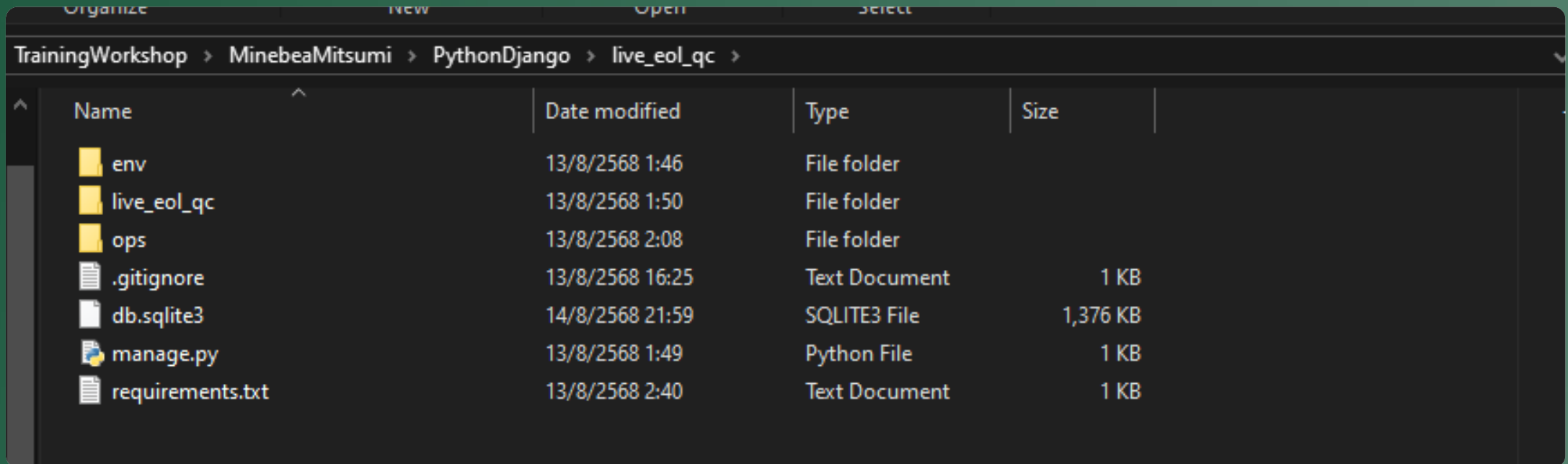


The screenshot shows a file explorer window with the path 'TrainingWorkshop > MinebeaMitsumi > PythonDjango > live_eol_qc'. The file list is as follows:

Name	Date modified	Type	Size
env	13/8/2568 1:46	File folder	
live_eol_qc	13/8/2568 1:50	File folder	
ops	13/8/2568 2:08	File folder	
.gitignore	13/8/2568 16:25	Text Document	1 KB
db.sqlite3	14/8/2568 21:59	SQLITE3 File	1,376 KB
manage.py	13/8/2568 1:49	Python File	1 KB
requirements.txt	13/8/2568 2:40	Text Document	1 KB

2. Create app “ops”

```
python manage.py startapp ops
```



Name	Date modified	Type	Size
env	13/8/2568 1:46	File folder	
live_eol_qc	13/8/2568 1:50	File folder	
ops	13/8/2568 2:08	File folder	
.gitignore	13/8/2568 16:25	Text Document	1 KB
db.sqlite3	14/8/2568 21:59	SQLITE3 File	1,376 KB
manage.py	13/8/2568 1:49	Python File	1 KB
requirements.txt	13/8/2568 2:40	Text Document	1 KB

3. ตั้งค่า Django + Channels + Redis

```
python

INSTALLED_APPS = [
    # ... apps เริ่มต้นของ Django ...
    'channels',
    'ops',
]

ASGI_APPLICATION = 'live_eol_qc.asgi.application'

import os
REDIS_URL = os.environ.get('REDIS_URL', 'redis://127.0.0.1:6379/0')

CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': { 'hosts': [REDIS_URL] },
    }
}

# Templates
TEMPLATES = [{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [],
    'APP_DIRS': True,
    'OPTIONS': { 'context_processors': [] },
}]
```

Copy Edit

4. ออกแบบฐานข้อมูล (Traceability)

python

Copy Edit

```
from django.db import models

class QcResult(models.Model):
    RESULT_CHOICES = [("PASS", "PASS"), ("FAIL", "FAIL")]

    serial = models.CharField(max_length=64, db_index=True)
    ts = models.DateTimeField(db_index=True)
    vib_rms_g = models.FloatField()
    vib_peak_g = models.FloatField()
    acoustic_db = models.FloatField()
    torque_mNm = models.FloatField()
    temp_c = models.FloatField()
    result = models.CharField(max_length=4, choices=RESULT_CHOICES)
    station_id = models.CharField(max_length=64, db_index=True)
    line_id = models.CharField(max_length=32, db_index=True)

    class Meta:
        indexes = [
            models.Index(fields=["line_id", "ts"]),
            models.Index(fields=["serial", "ts"])
```

5. เขียน Consumer สำหรับ WebSocket

python

Copy Edit

```
from channels.generic.websocket import AsyncJsonWebSocketConsumer

class QcConsumer(AsyncJsonWebSocketConsumer):
    """
    รองรับทั้ง 2 เส้นทาง:
    - ws/qc/line/<line_id>/
    - ws/qc/bearing/<serial>/
    จะจับ kwargs จาก URL แล้ว map เป็น group name
    """
    async def connect(self):
        params = self.scope['url_route']['kwargs']
        if 'line_id' in params:
            self.group = f"qc:line:{params['line_id']}"
        else:
            self.group = f"qc:serial:{params['serial']}"
        await self.channel_layer.group_add(self.group, self.channel_name)
        await self.accept()
        await self.send_json({"type": "connected", "group": self.group})

    async def disconnect(self, code):
        await self.channel_layer.group_discard(self.group, self.channel_name)
```

6. แดชบอร์ด (HTML + JS)

ops/urls.py

python

Copy Edit

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path("", TemplateView.as_view(template_name="ops/dashboard.html")),
]
```

live_eol_qc/urls.py

python

Copy Edit

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("", include("ops.urls")),
    path("admin/", admin.site.urls),
]
```

7. ช่องทางรับข้อมูลจริง (HTTP Ingest) + Broadcast

ops/views.py

python

Copy Edit

```
import json
from django.http import JsonResponse, HttpResponseBadRequest
from django.views.decorators.csrf import csrf_exempt
from django.utils import timezone
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from .models import QcResult

@csrf_exempt
def ingest_qc(request):
    if request.method != "POST":
        return HttpResponseBadRequest("POST only")

    try:
        payload = json.loads(request.body.decode("utf-8"))
        # คาดหวังฟิลด์ตาม payload ที่ตกลงกัน
        data = {
            "type": "qc",
            "serial": payload["serial"],
            "ts": payload.get("ts") or timezone.now().isoformat(),
            "vibration_rms_g": float(payload["vibration_rms_g"]),
            "vibration_peak_g": float(payload["vibration_peak_g"]),
            "acoustic_db": float(payload["acoustic_db"]),
            "torque_mNm": float(payload["torque_mNm"]),
            "temp_c": float(payload["temp_c"]),
            "result": payload["result"],
        }
```

8. ตัวจำลองสถานการณ์ทดสอบ (ส่งต่อเนื่องทุก 2 วิ)

ops/management/commands/simulate_qc.py

python

Copy Edit

```
import time, random
from django.core.management.base import BaseCommand
from django.utils import timezone
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from ops.models import QcResult

class Command(BaseCommand):
    help = "Simulate EOL QC stream (every 2s)"

    def add_arguments(self, parser):
        parser.add_argument("--line", default="L1")
        parser.add_argument("--station", default="EOL-01")
        parser.add_argument("--serial-prefix", default="MB-24-")

    def handle(self, *args, **opts):
        line_id = opts["line"]
        station = opts["station"]
        prefix = opts["serial-prefix"]
        channel_layer = get_channel_layer()
```

9. วิธีรันทดสอบ (dev)

1) เตรียมแวดล้อม

```
python -m venv env  
env\Scripts\activate (windows)  
source env/bin/activate (macos)  
pip install -r requirements.txt
```

2) ตั้ง Django

```
python manage.py migrate  
python -m uvicorn live_eol_qc.asgi:application --reload --host 0.0.0.0 --port 8000
```

3) เปิดอีกเทอร์มินัล

```
python manage.py simulate_qc --line L1 --station EOL-01
```

4) เปิดเบราว์เซอร์ที่

```
http://127.0.0.1:8000/
```



DJANGO

+

POSTGRES SQL

DEPLOYMENT ON RENDER

django



RENDER

LIVE



 WEB SERVICE

dekend-django-api-class

Python 3

Free

Upgrade your instance →

Connect 

Manual Deploy 

 iamsamitdev / dekend-day5  after2

https://dekend-django-api-class.onrender.com 

 Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. [Upgrade now](#)

	Deploy live for <u>adddb43</u>: update whitenoise February 18, 2025 at 12:22 AM	
	Deploy started for <u>adddb43</u>: update whitenoise New commit via Auto-Deploy February 18, 2025 at 12:21 AM	
	Deploy live for <u>f896745</u>: update static file for swagger February 17, 2025 at 11:58 PM	Rollback
	Deploy started for <u>f896745</u>: update static file for swagger New commit via Auto-Deploy February 17, 2025 at 11:56 PM	
	Deploy live for <u>1177927</u>: add staic file February 17, 2025 at 11:53 PM	Rollback
	Deploy started for <u>1177927</u>: add staic file	

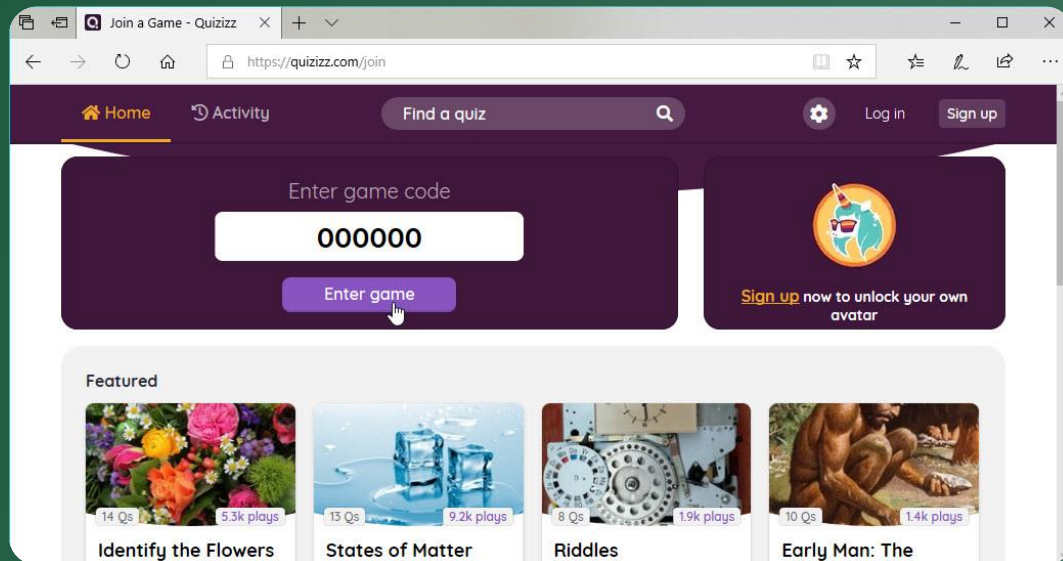
แบบทดสอบหลังอบรม



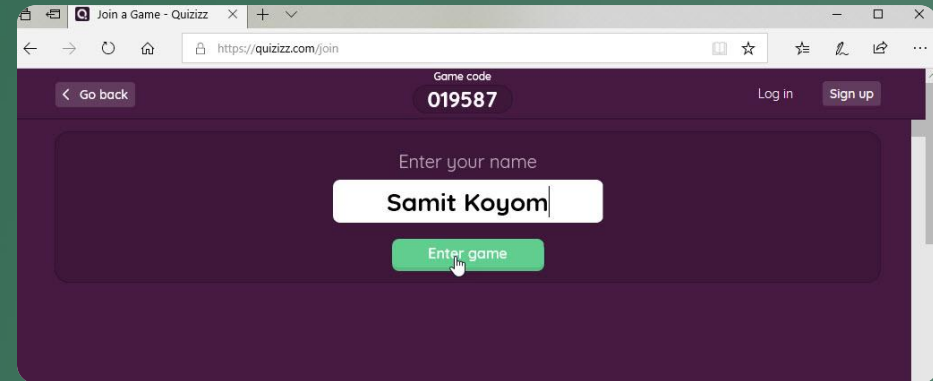
Pre-Test ทำแบบทดสอบหลังอบรม

STEP 1: เข้าทำแบบทดสอบที่ลิงก์ ป้อนรหัสเข้าห้องสอบ

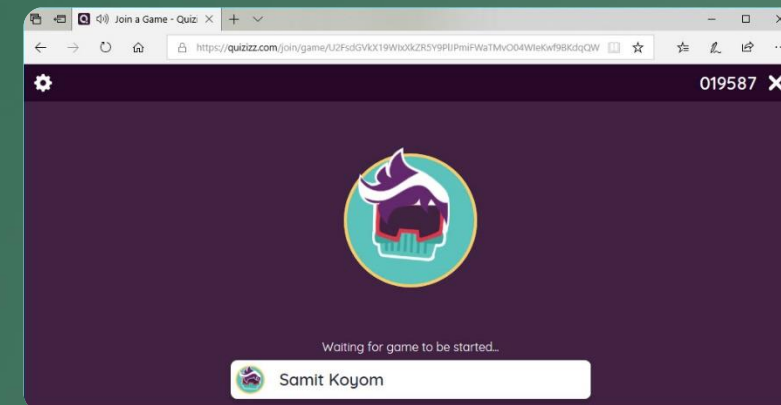
quizizz.com/join



STEP 2: ป้อนชื่อ



STEP 3: รอผู้สอน Start ข้อสอบ





อาจารย์สามิตร โกยม



samitkoyom@gmail.com



iamsamit



faeacebook.com/iamsamit



twitter.com/iamsamit



github.com/iamsamitdev



linkedin.com/in/samit-koyom-72173348